

Hub Éducatif & Créatif Multimodal

Dossier technique complet

Commandes utilisées · Entités et base de données · Authentification et rôles · Code et explication de chaque page

Projet universitaire — Django 5.2 · Python 3.12 · SQLite · Bootstrap 5

Sommaire

1. Présentation du projet et répartition du travail
2. Commandes utilisées, de zéro jusqu'au site qui tourne
3. Arborescence du projet
4. Comment Django fonctionne (MVT) — à expliquer au professeur
5. Configuration (settings.py et urls.py)
6. Entités et base de données (modèles, migrations, tables)
7. Authentification, rôles et permissions (Administrateur, Enseignant, Apprenant)
8. Pages du module commun : connexion, inscription, profil, utilisateurs, rôles
9. Pages du module pédagogique : cours et exercices
10. Pages statiques : coaching, ressources, objectifs
11. Accueil public et tableau de bord
12. Gabarit de base, composants, CSS et JavaScript
13. Tests effectués
14. Questions probables du professeur (avec réponses)

1. Présentation du projet et répartition du travail

Hub Éducatif & Créatif Multimodal est une plateforme éducative web : on y gère des cours et des exercices, avec des comptes utilisateurs et des rôles. Elle est développée par quatre étudiants ; chacun possède un module.

Module (app Django)	Entités	Qui / état	Ce qui est livré
`accounts` (commun)	User, Role	Commun à l'équipe	CRUD fonctionnel
`pedagogie`	Cours, Exercice	**Mon module**	CRUD fonctionnel
`coaching`	SessionCoaching, ConseilPedagogique	Collègue	Pages statiques uniquement
`ressources`	CategorieRessource, RessourcePedagogique	Collègue	Pages statiques uniquement
`objectifs`	ObjectifApprentissage, TacheApprentissage	Collègue	Pages statiques uniquement

> **Règle respectée :** pour les 6 entités des collègues il n'y a **aucun modèle, aucune migration, aucun formulaire, aucun CRUD**. Seules des pages de présentation avec des données de démonstration (fichiers `demo.py`) existent, clairement marquées « Démonstration ».

Technologies

Technologie	Rôle dans le projet
Python 3.12 + Django 5.2	Langage et framework (architecture MVT)
Django ORM	Accès à la base de données sans écrire de SQL
SQLite (`db.sqlite3`)	Base de données de développement (un simple fichier)
Django Auth	Utilisateurs, mots de passe hachés, groupes (rôles), permissions
Django Templates	Pages HTML dynamiques (`{% … %}` et `{{ … }}`)
Bootstrap 5.3 + Bootstrap Icons	Grille responsive, composants, icônes (chargés par CDN)
CSS (`hub.css`)	Design system : couleurs, cartes, badges, sidebar…
JavaScript (`hub.js`)	Sidebar repliable, modale de suppression, liste déroulante de la bonne réponse, filtres de démonstration
Chart.js	Graphique du tableau de bord (données réelles)

Charte graphique

Couleur	Code	Usage
Indigo	`#3D348B`	Couleur principale (boutons, page active)
Turquoise	`#00B8A9`	Couleur secondaire (accents, succès)
Ambre	`#F9A826`	Accent (badges « Démonstration »)
Texte	`#231D45`	Texte principal

2. Commandes utilisées, de zéro jusqu'au site qui tourne

Toutes les commandes se tapent dans un terminal (PowerShell). Voici l'ordre exact, avec ce que fait chaque commande.

#	Commande	Explication
1	`pip install django`	Installe Django (ou `pip install -r requirements.txt`).
2	`mkdir hub_educatif ; cd hub_educatif`	Crée et ouvre le dossier du projet.
3	`django-admin startproject config .`	Crée le projet : `manage.py` et le dossier `config/` (settings.py, urls.py, wsgi.py). Le point final = dans le dossier courant.
4	`python manage.py startapp accounts`	Crée l'application des comptes (utilisateurs, rôles).
5	`python manage.py startapp pedagogie`	Application de mon module (cours, exercices).
6	`python manage.py startapp coaching` (idem `ressources`, `objectifs`, `dashboard`)	Applications des pages statiques et du tableau de bord.
7	`python manage.py makemigrations pedagogie`	Lit `pedagogie/models.py` et génère le fichier de migration `0001_initial.py` (les instructions de création des tables).
8	`python manage.py sqlmigrate pedagogie 0001`	(Optionnel) Affiche le SQL réel que la migration va exécuter.
9	`python manage.py migrate`	Exécute toutes les migrations : crée les tables dans `db.sqlite3` (utilisateurs, groupes, permissions, cours, exercices…). Crée aussi les 3 rôles système grâce au signal `post_migrate`.
10	`python manage.py seed_demo`	Commande personnalisée : crée 3 comptes (admin, prof, élève), 3 cours et 4 exercices de démonstration.
11	`python manage.py createsuperuser`	(Optionnel) Crée un compte administrateur de son choix.
12	`python manage.py check`	Vérifie qu'il n'y a aucune erreur de configuration.
13	`python manage.py runserver`	Lance le serveur de développement sur http://127.0.0.1:8000/ (Ctrl+C pour l'arrêter).
14	`python manage.py shell`	Ouvre une console Python avec Django chargé, pour interroger la base (exemples en section 6).

Comptes de démonstration

Identifiant	Mot de passe	Rôle	Peut faire
admin	Admin1234!	Administrateur	Tout : utilisateurs, rôles, cours, exercices
prof	Prof1234!	Enseignant / Coach	Créer/modifier/supprimer ses cours et exercices
eleve	Eleve1234!	Apprenant	Consulter les cours/exercices et vérifier ses réponses

> **Important :** ces mots de passe sont uniquement pour la démonstration. Ils sont stockés **hachés** (PBKDF2)

> dans la base, jamais en clair.

3. Arborescence du projet

Structure des dossiers

```
hub_educatif/
├─ manage.py                      point d'entrée des commandes Django
├─ requirements.txt               dépendances (Django)
├─ db.sqlite3                     base de données SQLite (créée par migrate)
├─ README.md  GUIDE_INTEGRATION.md
├─ config/                        configuration globale
│   ├─ settings.py                réglages (apps, base, langue, auth…)
│   ├─ urls.py                    routes principales
│   ├─ forms.py                   BootstrapFormMixin (classes CSS des formulaires)
│   ├─ mixins.py                  QueryStringMixin (pagination + filtres)
│   └─ context.py                 version des fichiers CSS/JS (anti-cache)
├─ accounts/                      UTILISATEURS + RÔLES (CRUD)
│   ├─ roles.py                   constantes de rôles, is_admin(), création des rôles
│   ├─ apps.py                    signal post_migrate -> crée les rôles
│   ├─ forms.py  views.py  urls.py
│   └─ management/commands/seed_demo.py
├─ pedagogie/                     COURS + EXERCICES (CRUD)
│   ├─ models.py  forms.py  views.py  urls.py  migrations/
│   └─ templatetags/pedagogie_tags.py
├─ coaching/ ressources/ objectifs/   pages STATIQUES (views.py, urls.py, demo.py)
├─ dashboard/                     accueil public + tableau de bord
├─ templates/
│   ├─ base.html                  gabarit principal (sidebar + topbar)
│   ├─ auth_base.html             gabarit connexion/inscription
│   ├─ components/                morceaux réutilisables
│   └─ accounts/ pedagogie/ coaching/ ressources/ objectifs/ dashboard/
└─ static/  css/hub.css   js/hub.js   images/logo.svg
```

4. Comment Django fonctionne (MVT) — à expliquer au professeur

Django suit l'architecture **MVT : Model – View – Template**. Quand l'utilisateur ouvre une adresse, voici ce qui se passe :

Cycle d'une requête

```
Navigateur  ->  urls.py  ->  View (views.py)  ->  Model (models.py + base SQLite)
                                    |
                                    v
                             Template (HTML)  ->  page envoyée au navigateur
```

Élément	Fichier	Rôle
URL	`urls.py`	Associe une adresse (`/pedagogie/cours/`) à une vue.
Model	`models.py`	Décrit les données (une classe = une table). Django génère les tables avec les migrations.
View	`views.py`	Contient la logique : lit la base, applique recherche/filtres/permissions, choisit le gabarit.
Template	`templates/…html`	Le HTML de la page, avec des variables `{{ cours.titre }}` et des boucles `{% for … %}`.
Form	`forms.py`	Génère et valide les formulaires à partir d'un modèle (`ModelForm`).

Nous utilisons les **vues basées sur des classes** de Django : `ListView` (liste), `DetailView` (détail), `CreateView` (ajout), `UpdateView` (modification), `DeleteView` (suppression). Elles réalisent le CRUD avec très peu de code.

Lettre CRUD	Signification	Vue Django	Méthode HTTP
C	Create (ajouter)	CreateView	GET (formulaire) puis POST (enregistrer)
R	Read (lire)	ListView / DetailView	GET
U	Update (modifier)	UpdateView	GET puis POST
D	Delete (supprimer)	DeleteView	POST (après confirmation)

5. Configuration (settings.py et urls.py)

Voici le fichier `config/settings.py` (sans les commentaires automatiques). Les points que j'ai modifiés sont expliqués après le code.

config/settings.py

```
"""
Django settings for config project.
Generated by 'django-admin startproject' using Django 5.2.
For more information on this file, see
https://docs.djangoproject.com/en/5.2/topics/settings/
For the full list of settings and their values, see
https://docs.djangoproject.com/en/5.2/ref/settings/
"""
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent
SECRET_KEY = 'django-insecure-+k9coov19&n(6uf()n&+_rtbd9hp$04t5wd5f(&fthzn$44rsi'
DEBUG = True
ALLOWED_HOSTS = []
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'accounts',
    'pedagogie',
    'coaching',
    'ressources',
    'objectifs',
    'dashboard',
]
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.auth.middleware.LoginRequiredMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
ROOT_URLCONF = 'config.urls'
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
                'config.context.assets',
            ],
        },
    },
]
WSGI_APPLICATION = 'config.wsgi.application'
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
```

```
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
LANGUAGE_CODE = 'fr'
TIME_ZONE = 'Africa/Tunis'
USE_I18N = True
USE_TZ = True
STATIC_URL = 'static/'
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
STATICFILES_DIRS = [BASE_DIR / 'static']
LOGIN_URL = 'accounts:login'
LOGIN_REDIRECT_URL = 'dashboard:home'
LOGOUT_REDIRECT_URL = 'dashboard:landing'
from django.contrib.messages import constants as _m
MESSAGE_TAGS = {_m.ERROR: 'danger'}
import time
ASSET_VERSION = int(time.time())  # change à chaque démarrage : évite les CSS/JS en cache
```

Ce qui a été modifié et pourquoi

1. **INSTALLED_APPS** : j'ai ajouté `accounts`, `pedagogie`, `coaching`, `ressources`, `objectifs`, `dashboard`. Sans cela Django ne les connaît pas (modèles, templates, commandes).
2. **MIDDLEWARE → LoginRequiredMiddleware** : oblige à être connecté pour **toutes** les pages. Les seules exceptions sont les vues décorées par `@login_not_required` (accueil, connexion, inscription).
3. **TEMPLATES → DIRS** : indique le dossier global `templates/`. **context_processors** : `config.context.assets` ajoute la variable `ASSET_V` aux gabarits.
4. **LANGUAGE_CODE = 'fr'** et **TIME_ZONE = 'Africa/Tunis'** : interface en français, dates à l'heure locale.
5. **STATICFILES_DIRS** : dossier `static/` (CSS, JS, images).
6. **LOGIN_URL / LOGIN_REDIRECT_URL / LOGOUT_REDIRECT_URL** : où envoyer l'utilisateur non connecté, après connexion et après déconnexion.
7. **MESSAGE_TAGS** : transforme le niveau « error » de Django en classe Bootstrap « danger » pour les alertes.
8. **ASSET_VERSION** : nombre (heure de démarrage) ajouté à `hub.css?v=…` pour que le navigateur recharge toujours le dernier CSS/JS.

config/urls.py — les routes principales

config/urls.py

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('', include('dashboard.urls')),
    path('comptes/', include('accounts.urls')),
    path('pedagogie/', include('pedagogie.urls')),
    path('coaching/', include('coaching.urls')),
    path('ressources/', include('ressources.urls')),
    path('objectifs/', include('objectifs.urls')),
    path('admin/', admin.site.urls),
]
```

Chaque application possède son propre `urls.py` inclus avec un préfixe : `comptes/`, `pedagogie/`, `coaching/`, `ressources/`, `objectifs/`. L'accueil et le tableau de bord (`dashboard`) sont à la racine.

Outils communs

config/forms.py

```
from django import forms


class BootstrapFormMixin:
    """Ajoute les classes Bootstrap à tous les widgets d'un formulaire."""

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        for f in self.fields.values():
            w = f.widget
            if isinstance(w, forms.CheckboxInput):
                w.attrs['class'] = 'form-check-input'
            elif isinstance(w, (forms.Select, forms.SelectMultiple)):
                w.attrs['class'] = 'form-select'
            elif not isinstance(w, forms.CheckboxSelectMultiple):
                w.attrs['class'] = 'form-control'
```

BootstrapFormMixin : parcourt tous les champs d'un formulaire et ajoute la classe CSS Bootstrap adaptée (`form-control`, `form-select`, `form-check-input`). Ainsi, on n'écrit pas les classes à la main pour chaque champ.

config/mixins.py

```
class QueryStringMixin:
    """Expose `querystring` (GET sans `page`) pour conserver filtres + pagination."""

    def get_context_data(self, **kwargs):
        ctx = super().get_context_data(**kwargs)
        q = self.request.GET.copy()
        q.pop('page', None)
        ctx['querystring'] = q.urlencode()
        return ctx
```

QueryStringMixin : donne au gabarit la variable `querystring` (les paramètres de l'URL sans `page`). Les liens de pagination l'utilisent : en passant à la page 2, la recherche et les filtres sont conservés.

config/context.py

```
from django.conf import settings


def assets(request):
    return {'ASSET_V': settings.ASSET_VERSION}
```

Ce « context processor » rend `ASSET_V` disponible dans tous les gabarits (anti-cache CSS/JS).

6. Entités et base de données

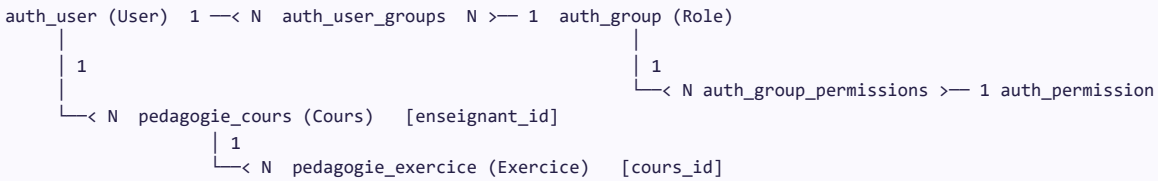
La base est un fichier `db.sqlite3` à la racine du projet. Les tables sont créées automatiquement par Django à partir des **modèles** (classes Python) avec les migrations. Voici où se trouve chaque entité :

Entité du sujet	Où elle est définie	Table SQLite
User	Modèle natif Django `django.contrib.auth.models.User` (pas de modèle à écrire)	`auth_user`
Role	Modèle natif `Group` (un rôle = un groupe)	`auth_group`
Lien User ↔ Role	Relation native	`auth_user_groups`
Permissions d'un rôle	Modèle natif `Permission`	`auth_permission`, `auth_group_permissions`
Cours	`pedagogie/models.py` (écrit par nous)	`pedagogie_cours`
Exercice	`pedagogie/models.py` (écrit par nous)	`pedagogie_exercice`

> **Choix de conception :** pour User et Role on réutilise le système natif de Django (comme demandé « utiliser de préférence les groupes et permissions natifs »). Le champ « role » de l'utilisateur est donc son **groupe**. On ne remplace pas le modèle User, ce qui évite les problèmes de migration.

6.1 Schéma des relations

Un enseignant a plusieurs cours ; un cours a plusieurs exercices



6.2 Le code des modèles : pedagogie/models.py

pedagogie/models.py

```
from django.conf import settings
from django.core.exceptions import ValidationError
from django.db import models
from django.urls import reverse


class Cours(models.Model):
    NIVEAUX = [('debutant', 'Débutant'), ('intermediaire', 'Intermédiaire'), ('avance', 'Avancé')]

    titre = models.CharField(max_length=200)
    description = models.TextField('Description courte')
    matiere = models.CharField('Matière', max_length=100)
    niveau = models.CharField(max_length=20, choices=NIVEAUX, default='debutant')
    contenu = models.TextField('Contenu du cours')
    date_creation = models.DateTimeField(auto_now_add=True)
    enseignant = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.PROTECT, related_name='cours')

    class Meta:
        ordering = ['-date_creation']
        verbose_name_plural = 'cours'

    def __str__(self):
        return self.titre
```

```
    def get_absolute_url(self):
        return reverse('pedagogie:cours_detail', args=[self.pk])


class Exercice(models.Model):
    TYPES = [('qcm', 'QCM'), ('vf', 'Vrai / Faux'), ('ouverte', 'Question ouverte')]
    DIFFICULTES = [('facile', 'Facile'), ('moyen', 'Moyen'), ('difficile', 'Difficile')]

    titre = models.CharField(max_length=200)
    question = models.TextField()
    type_exercice = models.CharField('Type', max_length=10, choices=TYPES, default='qcm')
    difficulte = models.CharField('Difficulté', max_length=10, choices=DIFFICULTES, default='facile')
    propositions = models.TextField('Propositions (QCM)', blank=True, help_text='Une proposition par ligne.')
    bonne_reponse = models.TextField('Bonne réponse')
    cours = models.ForeignKey(Cours, on_delete=models.CASCADE, related_name='exercices')

    class Meta:
        ordering = ['cours__titre', 'titre']

    def __str__(self):
        return self.titre

    def get_absolute_url(self):
        return reverse('pedagogie:exercice_detail', args=[self.pk])

    @property
    def enseignant(self):
        return self.cours.enseignant

    @property
    def liste_propositions(self):
        return [p.strip() for p in self.propositions.splitlines() if p.strip()]

    def clean(self):
        rep = (self.bonne_reponse or '').strip()
        if self.type_exercice == 'qcm':
            props = self.liste_propositions
            if len(props) < 2:
                raise ValidationError({'propositions': 'Un QCM nécessite au moins deux propositions.'})
            match = next((p for p in props if p.lower() == rep.lower()), None)
            if match is None:
                raise ValidationError({'bonne_reponse': 'La bonne réponse doit correspondre à une proposition.'})
            self.propositions, rep = '\n'.join(props), match
        elif self.type_exercice == 'vf':
            if rep.lower() not in ('vrai', 'faux'):
                raise ValidationError({'bonne_reponse': 'Saisissez « Vrai » ou « Faux ».'})
            self.propositions, rep = '', rep.capitalize()
        else:
            self.propositions = ''
        self.bonne_reponse = rep
```

Explication ligne par ligne

1. `class Cours(models.Model)` : une classe qui hérite de `models.Model` devient une table. Chaque attribut est une colonne.
2. `CharField(max_length=200)` = texte court ; `TextField` = texte long ; `DateTimeField(auto_now_add=True)` = date remplie automatiquement à la création.
3. `niveau` utilise `choices` : liste de couples (valeur stockée, libellé affiché). Idem pour le type et la difficulté de l'exercice.
4. `enseignant = ForeignKey(User, on_delete=PROTECT)` : **clé étrangère** vers l'utilisateur. `PROTECT` interdit de supprimer un utilisateur qui a des cours. `related_name='cours'` permet `user.cours.all()`.
5. `Exercice.cours = ForeignKey(Cours, on_delete=CASCADE, related_name='exercices')` : si on supprime un cours, ses exercices sont supprimés. `cours.exercices.all()` donne les exercices d'un cours.
6. `propositions` : texte avec **une proposition par ligne** (c'est la façon simple de saisir plusieurs choix d'un QCM). Ce champ s'ajoute à la liste du sujet.
7. `Meta.ordering` : ordre par défaut des listes. `__str__` : texte affiché pour un objet. `get_absolute_url` : adresse de la page de détail.

8. `Exercice.clean()` : **validation métier** exécutée automatiquement par le formulaire. QCM : au moins 2 propositions et la bonne réponse doit en faire partie. Vrai/Faux : « Vrai » ou « Faux ». Question ouverte : texte libre. En cas d'erreur, un message s'affiche sous le champ concerné.
9. `@property enseignant` sur Exercice : renvoie l'enseignant du cours, pour vérifier facilement qui est propriétaire d'un exercice.

6.3 La migration générée

Après avoir écrit le modèle, la commande `python manage.py makemigrations pedagogie` crée `pedagogie/migrations/0001_initial.py`. La commande `python manage.py sqlmigrate pedagogie 0001` montre le SQL exécuté par `migrate` :

SQL réel généré (sqlmigrate)

```
BEGIN;
--
-- Create model Cours
--
CREATE TABLE "pedagogie_cours" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "titre" varchar(200) NOT NULL, "description"
text NOT NULL, "matiere" varchar(100) NOT NULL, "niveau" varchar(20) NOT NULL, "contenu" text NOT NULL, "date_creation" datetime
NOT NULL, "enseignant_id" integer NOT NULL REFERENCES "auth_user" ("id") DEFERRABLE INITIALLY DEFERRED);
--
-- Create model Exercice
--
CREATE TABLE "pedagogie_exercice" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "titre" varchar(200) NOT NULL, "question"
text NOT NULL, "type_exercice" varchar(10) NOT NULL, "difficulte" varchar(10) NOT NULL, "propositions" text NOT NULL,
"bonne_reponse" text NOT NULL, "cours_id" bigint NOT NULL REFERENCES "pedagogie_cours" ("id") DEFERRABLE INITIALLY DEFERRED);
CREATE INDEX "pedagogie_cours_enseignant_id_4faab17f" ON "pedagogie_cours" ("enseignant_id");
CREATE INDEX "pedagogie_exercice_cours_id_24b5c3d7" ON "pedagogie_exercice" ("cours_id");
COMMIT;
```

6.4 Tables de la base

Table	Colonnes principales
`auth_user`	id, username, first_name, last_name, email, password (haché), is_active, is_staff, is_superuser, date_joined, last_login
`auth_group`	id, name (nom du rôle)
`auth_user_groups`	id, user_id → auth_user, group_id → auth_group (le rôle de l'utilisateur)
`auth_permission`	id, name, content_type_id, codename (ex. `add_cours`)
`auth_group_permissions`	id, group_id, permission_id (permissions d'un rôle)
`pedagogie_cours`	id, titre, description, matiere, niveau, contenu, date_creation, enseignant_id → auth_user
`pedagogie_exercice`	id, titre, question, type_exercice, difficulte, propositions, bonne_reponse, cours_id → pedagogie_cours

Django crée automatiquement 4 permissions par modèle : `add_`, `change_`, `delete_`, `view_` (ex. `add_cours`, `change_exercice`, `view_user`, `delete_group`).

6.5 Interroger la base avec le shell Django

Exemples de requêtes ORM

```
python manage.py shell

>>> from pedagogie.models import Cours, Exercice
>>> Cours.objects.count()                              # nombre de cours
```

```
>>> Cours.objects.filter(matiere="Informatique")           # filtrer
>>> c = Cours.objects.get(titre="Introduction à Python")
>>> c.exercices.all()                                     # exercices du cours (relation inverse)
>>> c.enseignant.username                                 # 'prof'
>>> from django.contrib.auth.models import User, Group
>>> User.objects.get(username="eleve").groups.all()       # rôle de l'élève
>>> Group.objects.get(name="Enseignant / Coach").permissions.all()
```

7. Authentification, rôles et permissions

Cette section explique **comment fonctionnent la connexion et la différence entre Administrateur, Enseignant et Apprenant**.

7.1 Les trois rôles

Un rôle est un **groupe Django**. Un groupe possède une liste de **permissions**. Un utilisateur appartient à un groupe : il reçoit ses permissions.

accounts/roles.py

```
from django.contrib.auth.models import Group, Permission

ADMIN = 'Administrateur'
TEACHER = 'Enseignant / Coach'
LEARNER = 'Apprenant'
SYSTEM_ROLES = (ADMIN, TEACHER, LEARNER)


def is_admin(user):
    return user.is_superuser or user.groups.filter(name=ADMIN).exists()


def _perms(models, actions):
    codes = [f'{a}_{m}' for m in models for a in actions]
    return Permission.objects.filter(codename__in=codes, content_type__app_label__in=['auth', 'pedagogie'])


def ensure_system_roles():
    """Crée les 3 rôles système (une seule fois) avec leurs permissions par défaut."""
    crud = ('add', 'change', 'delete', 'view')
    defaults = {
        ADMIN: _perms(['user', 'group', 'cours', 'exercice'], crud),
        TEACHER: _perms(['cours', 'exercice'], crud),
        LEARNER: _perms(['cours', 'exercice'], ('view',)),
    }
    for name, perms in defaults.items():
        group, created = Group.objects.get_or_create(name=name)
        if created:
            group.permissions.set(perms)
```

1. Les constantes `ADMIN`, `TEACHER`, `LEARNER` contiennent les noms des 3 rôles ; `SYSTEM_ROLES` est la liste des rôles protégés (non supprimables, non renommables).
2. `is_admin(user)` : vrai si l'utilisateur est superutilisateur ou membre du groupe « Administrateur ».
3. `ensure_system_roles()` crée les 3 groupes **s'ils n'existent pas** et leur donne leurs permissions par défaut : Administrateur = tout (utilisateurs, rôles, cours, exercices) ; Enseignant = ajouter/modifier/supprimer/consulter cours et exercices ; Apprenant = consulter seulement. Les permissions ne sont posées qu'à la création, donc les modifications faites ensuite dans l'écran « Rôles » ne sont pas écrasées.

accounts/apps.py

```
from django.apps import AppConfig
from django.db.models.signals import post_migrate


def _create_roles(sender, **kwargs):
    from django.apps import apps
    from django.contrib.auth.management import create_permissions

    for label in ('auth', 'pedagogie'):
        create_permissions(apps.get_app_config(label), verbosity=0)
    from .roles import ensure_system_roles
    ensure_system_roles()
```

```
class AccountsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'accounts'

    def ready(self):
        post_migrate.connect(_create_roles, sender=self)
```

Ce fichier **connecte** `ensure_system_roles` au signal `post_migrate` : dès que `python manage.py migrate` se termine, les rôles existent. C'est indispensable car l'inscription attribue le rôle « Apprenant ».

7.2 Tableau « qui peut faire quoi »

Action	Administrateur	Enseignant / Coach	Apprenant
Voir le tableau de bord, cours, exercices	✓	✓	✓
Vérifier sa réponse à un exercice	✓	✓	✓
Voir la bonne réponse dans la fiche d'un exercice	✓	✓ (les siens)	✗ (après essai seulement)
Créer un cours / un exercice	✓	✓	✗ (403)
Modifier / supprimer un cours ou exercice	✓ (tous)	✓ (**seulement les siens**)	✗
Gérer les utilisateurs (liste, ajout, modif., désactivation, suppression)	✓	✗	✗
Gérer les rôles et leurs permissions	✓	✗	✗
Voir et modifier son profil, changer son mot de passe	✓	✓	✓
Pages de démonstration (coaching, ressources, objectifs)	✓	✓	✓

7.3 Comment la sécurité est appliquée dans le code (3 niveaux)

1. **Connexion obligatoire** : `LoginRequiredMiddleware` (settings.py) redirige vers la page de connexion toute personne non connectée.
2. **Permission Django** : chaque vue utilise `PermissionRequiredMixin` avec `permission_required = 'pedagogie.add_cours'` (par exemple). Si le rôle n'a pas cette permission : erreur 403 « Accès refusé ».
3. **Propriété de l'objet** : pour modifier/supprimer, `OwnerOrAdminMixin` vérifie que l'objet appartient à l'utilisateur (ou qu'il est admin).

pedagogie/views.py — OwnerOrAdminMixin

```
class OwnerOrAdminMixin:
    """Modifier/supprimer : seulement le propriétaire (enseignant du cours) ou un administrateur."""

    def get_object(self, queryset=None):
        obj = super().get_object(queryset)
        if not is_admin(self.request.user) and obj.enseignant != self.request.user:
            raise PermissionDenied
        return obj
```

pedagogie/templatetags/pedagogie_tags.py

```
from django import template

from accounts.roles import is_admin

register = template.Library()
```

```
def _allowed(obj, user, action):
    perm = f'pedagogie.{action}_{obj._meta.model_name}'
    return user.has_perm(perm) and (is_admin(user) or obj.enseignant == user)


@register.filter
def can_edit(obj, user):
    return _allowed(obj, user, 'change')


@register.filter
def can_delete(obj, user):
    return _allowed(obj, user, 'delete')
```

Dans les gabarits, les filtres `can_edit` et `can_delete` servent à **masquer** les boutons non autorisés (ex. `{% if c|can_edit:user %}`). Le masquage est un confort visuel ; la vraie protection reste dans les vues (même en tapant l'adresse à la main, un apprenant reçoit un 403).

7.4 La page de connexion (login)

La connexion utilise la `LoginView` fournie par Django : elle vérifie le nom d'utilisateur et le mot de passe (comparé au hachage). Nous ne personnalisons que la vue (décorateur « sans connexion requise ») et le gabarit.

accounts/views.py — HubLoginView

```
class HubLoginView(LoginView):
    template_name = 'accounts/login.html'
    redirect_authenticated_user = True
```

accounts/urls.py (extrait)

```
path('connexion/', v.HubLoginView.as_view(), name='login'),
path('deconnexion/', LogoutView.as_view(), name='logout'),
```

La **déconnexion** est un formulaire POST avec jeton CSRF (obligatoire depuis Django 5), présent dans la sidebar et le menu du compte de `base.html`. Après connexion, l'utilisateur est envoyé vers `dashboard:home` (`LOGIN_REDIRECT_URL`).

7.5 Comment le rôle apparaît dans l'interface

Dans les gabarits

```
{{ user.groups.first.name|default:"Utilisateur" }}      {# nom du rôle de l'utilisateur connecté #}

{% if perms.auth.view_user %}  ... lien "Utilisateurs"  {% endif %}      {# visible seulement avec la permission #}
{% if perms.pedagogie.add_cours %}  ... bouton "Ajouter un cours"  {% endif %}
```

L'objet `perms` est fourni automatiquement par Django : il contient les permissions de l'utilisateur connecté. C'est ainsi que la sidebar affiche « Administration » seulement à l'administrateur, et le bouton « Ajouter » seulement à ceux qui peuvent créer.

7.6 Création des comptes de démonstration : seed_demo

accounts/management/commands/seed_demo.py

```
from django.contrib.auth.models import Group, User
from django.core.management.base import BaseCommand

from accounts.roles import ADMIN, LEARNER, TEACHER, ensure_system_roles
from pedagogie.models import Cours, Exercice

USERS = [  # (username, prénom, nom, rôle, mot de passe) — comptes de démonstration
    ('admin', 'Amel', 'Admin', ADMIN, 'Admin1234!'),
```

```
    ('prof', 'Karim', 'Enseignant', TEACHER, 'Prof1234!'),
    ('eleve', 'Sarra', 'Apprenante', LEARNER, 'Eleve1234!'),
]

COURS = [
    ('Introduction à Python', 'Les bases du langage : variables, types et boucles.', 'Informatique', 'debutant',
     'Python est un langage simple et lisible.\n\n1. Variables et types\n2. Conditions\n3. Boucles for et while'),
    ('Bases des mathématiques', 'Fractions, proportions et calcul mental.', 'Mathématiques', 'debutant',
     'Une fraction représente une part d\'un tout : a/b.\n\nPour additionner deux fractions, on les met au même dénominateur.'),
    ('Anglais débutant', 'Vocabulaire et phrases essentielles du quotidien.', 'Langues', 'debutant',
     'Greetings: Hello, Good morning, Goodbye.\nIntroductions: My name is..., I am from...'),
]

EXERCICES = [
    ('Variables Python', 0, 'qcm', 'facile', 'Quelle instruction crée une variable x valant 5 ?', 'x = 5', 'x := 5\nx = 5\nint x
= 5\nlet x == 5'),
    ('Boucles Python', 0, 'vf', 'moyen', 'Une boucle while s\'exécute au moins une fois.', 'Faux', ''),
    ('Calcul de fractions', 1, 'ouverte', 'moyen', 'Calculez 1/2 + 1/3.', '5/6', ''),
    ('Vocabulaire anglais', 2, 'qcm', 'facile', 'Comment dit-on « Bonjour » en anglais ?', 'Hello',
'Hello\nGoodbye\nThanks\nSorry'),
]


class Command(BaseCommand):
    help = 'Crée les rôles, 3 comptes et quelques cours/exercices de démonstration.'

    def handle(self, *args, **opts):
        ensure_system_roles()
        for username, first, last, role, pwd in USERS:
            u, created = User.objects.get_or_create(username=username, defaults={
                'first_name': first, 'last_name': last, 'email': f'{username}@hub.local',
                'is_staff': role == ADMIN, 'is_superuser': role == ADMIN})
            if created:
                u.set_password(pwd)
                u.save()
            u.groups.set([Group.objects.get(name=role)])
        prof = User.objects.get(username='prof')
        cours = [Cours.objects.get_or_create(titre=t, defaults=dict(
            description=d, matiere=m, niveau=n, contenu=c, enseignant=prof))[0] for t, d, m, n, c in COURS]
        for titre, ci, typ, diff, q, rep, props in EXERCICES:
            ex, created = Exercice.objects.get_or_create(titre=titre, cours=cours[ci], defaults=dict(
                type_exercice=typ, difficulte=diff, question=q, bonne_reponse=rep, propositions=props))
        self.stdout.write(self.style.SUCCESS('Données de démonstration prêtes.'))
```

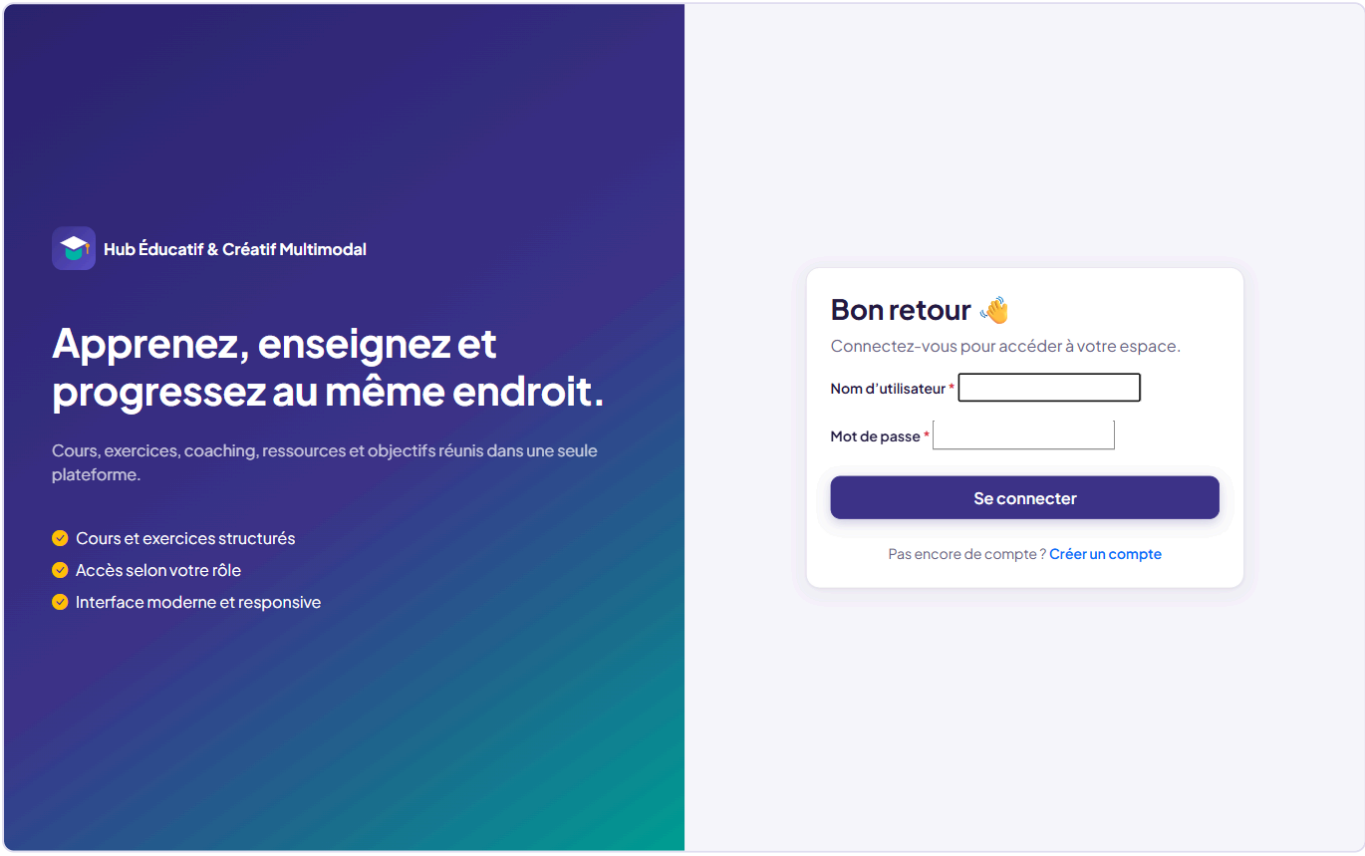
1. Le fichier est placé dans `management/commands/` : Django le reconnaît comme la commande `python manage.py seed_demo`.
2. `ensure_system_roles()` garantit que les rôles existent.
3. Pour chaque compte : `get_or_create` (ne crée pas de doublon), `set_password` (hache le mot de passe), puis `groups.set([...])` attribue le rôle.
4. Les cours sont créés pour l'enseignant « prof » et les exercices sont rattachés aux cours. La commande peut être relancée sans danger.

8. Pages du module commun : connexion, inscription, profil, utilisateurs, rôles

Pour chaque page : **adresse** (après `python manage.py runserver`), capture, **code de la vue**, **code de l'interface (gabarit HTML)** et explication.

8.1 Page de connexion

Commande / adresse :
`http://127.0.0.1:8000/comptes/connexion/` (nom d'URL : `accounts:login`)



Page de connexion

Fichiers : `accounts/views.py` (HubLoginView, vu en 7.4), `templates/auth_base.html`, `templates/accounts/login.html`, `templates/components/field.html`.

`templates/auth_base.html`

```
{% load static %}<!doctype html>
<html lang="fr">
<head>{% include "components/head.html" %}<title>{% block title %}{% endblock %} · Hub Éducatif</title></head>
<body>
<div class="auth-wrap">
  <aside class="auth-side">
    <a href="{% url 'dashboard:landing' %}" class="d-flex align-items-center gap-2 text-white fw-bold mb-5"><img src="{% static
'images/logo.svg' %}" width="44" alt=""> Hub Éducatif & Créatif Multimodal</a>
    <h2 class="display-6 fw-bold">Apprenez, enseignez et progressez au même endroit.</h2>
    <p class="opacity-75 mt-3">Cours, exercices, coaching, ressources et objectifs réunis dans une seule plateforme.</p>
    <ul class="list-unstyled mt-4 opacity-90">
      <li class="mb-2"><i class="bi bi-check-circle-fill text-warning me-2"></i>Cours et exercices structurés</li>
      <li class="mb-2"><i class="bi bi-check-circle-fill text-warning me-2"></i>Accès selon votre rôle</li>
      <li><i class="bi bi-check-circle-fill text-warning me-2"></i>Interface moderne et responsive</li>
    </ul>
  </aside>
  <section class="auth-form"><div class="auth-card">
```

```
    {% for m in messages %}<div class="alert alert-{{ m.tags|default:'info' }}">{{ m }}</div>{% endfor %}
    {% block content %}{% endblock %}
  </div></section>
</div>
{% include "components/scripts.html" %}
</body>
</html>
```

templates/accounts/login.html

```
{% extends "auth_base.html" %}
{% block title %}Connexion{% endblock %}
{% block content %}
<div class="card-x card-pad p-4">
  <h3>Bon retour 👋</h3><p class="text-muted">Connectez-vous pour accéder à votre espace.</p>
  <form method="post" novalidate>{% csrf_token %}
    {% if form.non_field_errors %}<div class="alert alert-danger">{{ form.non_field_errors.0 }}</div>{% endif %}
    {% include "components/field.html" with field=form.username %}
    {% include "components/field.html" with field=form.password %}
    <input type="hidden" name="next" value="{{ next }}">
    <button class="btn btn-primary w-100 mt-2">Se connecter</button>
  </form>
  <p class="text-center text-muted small mt-4 mb-0">Pas encore de compte ? <a href="{% url 'accounts:register' %}" class="fw-
semibold">Créer un compte</a></p>
</div>
{% endblock %}
```

templates/components/field.html

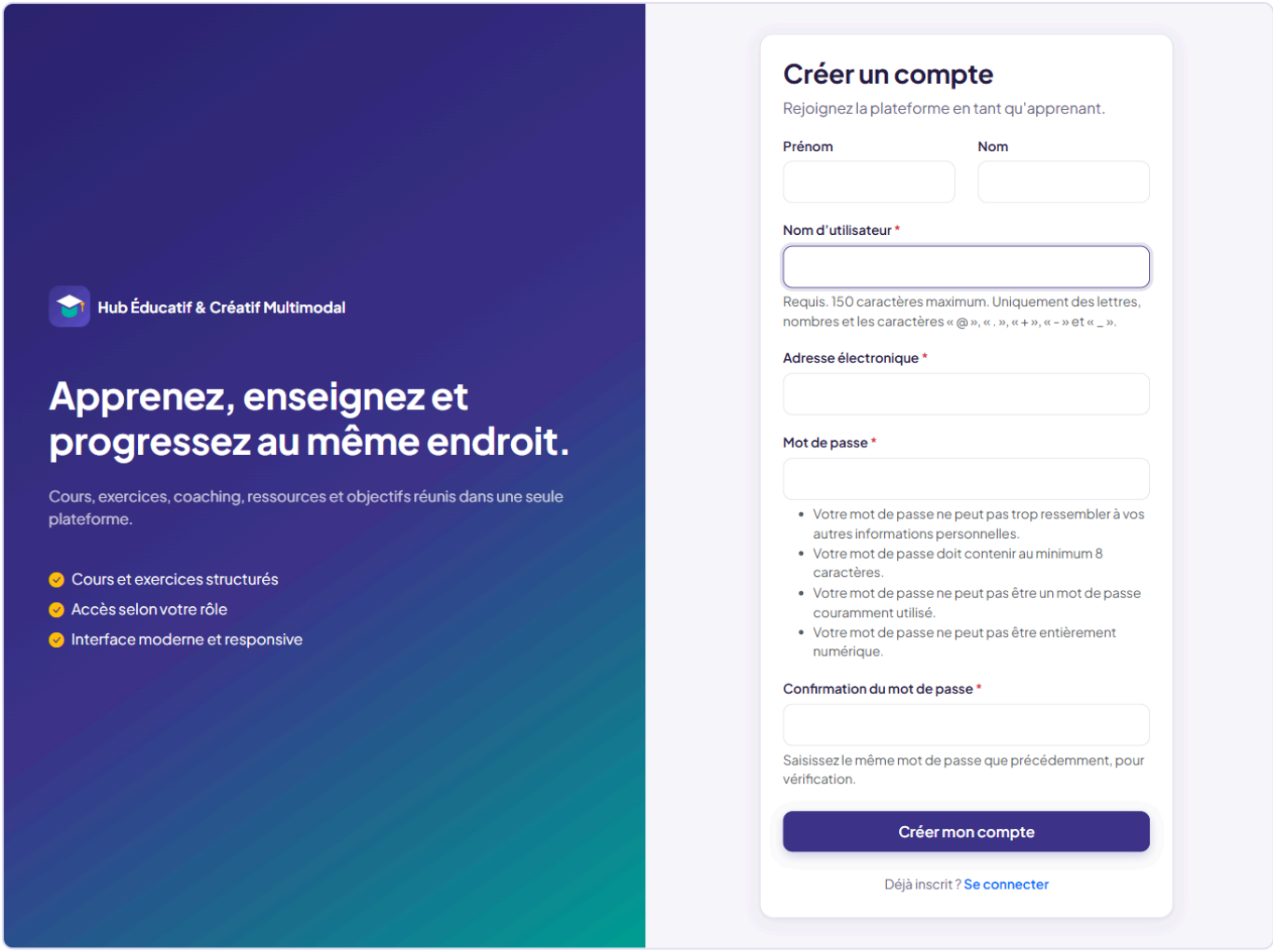
```
{# Usage : {% include "components/field.html" with field=form.titre %} (+ hint="..." optionnel) #}
<div class="field mb-3">
  {% if field.field.widget.input_type == "checkbox" %}
    <div class="form-check form-switch">{{ field }} <label class="form-check-label" for="{{ field.id_for_label }}">{{
field.label }}</label></div>
  {% else %}
    <label class="form-label" for="{{ field.id_for_label }}">{{ field.label }}{% if field.field.required %} <span class="text-
danger">*</span>{% endif %}</label>
    {{ field }}
  {% endif %}
  {% if hint %}<div class="form-text" id="answer-hint">{{ hint }}</div>{% elif field.help_text %}<div class="form-text">{{
field.help_text }}</div>{% endif %}
  {% for e in field.errors %}<div class="invalid-feedback">{{ e }}</div>{% endfor %}
</div>
```

1. `login.html` **étend** `auth_base.html` (`{% extends %}`) et remplit le bloc `content`.
2. `{% csrf_token %}` : jeton de sécurité obligatoire dans tout formulaire POST (protection CSRF).
3. `form.username` et `form.password` viennent du `AuthenticationForm` de Django ; `field.html` affiche chaque champ avec son libellé, son étoile « requis » et ses messages d'erreur.
4. `form.non_field_errors` affiche « identifiant ou mot de passe incorrect ».
5. `auth_base.html` : mise en page en deux colonnes ; la colonne de présentation est masquée sur petits écrans (CSS `.auth-wrap`).

8.2 Page d'inscription

Commande / adresse :

`http://127.0.0.1:8000/comptes/inscription/` (accounts:register)



Page d'inscription

accounts/views.py — RegisterView

```
class RegisterView(CreateView):
    form_class = RegisterForm
    template_name = 'accounts/register.html'
    success_url = reverse_lazy('accounts:login')

    def form_valid(self, form):
        messages.success(self.request, 'Compte créé : vous pouvez vous connecter.')
        return super().form_valid(form)
```

accounts/forms.py — RegisterForm

```
class RegisterForm(BootstrapFormMixin, UserCreationForm):
    class Meta:
        model = User
        fields = ('first_name', 'last_name', 'username', 'email')

    def __init__(self, *a, **kw):
        super().__init__(*a, **kw)
        self.fields['email'].required = True

    def save(self, commit=True):
        user = super().save(commit)
        user.groups.add(Group.objects.get(name=LEARNER))
        return user
```

templates/accounts/register.html

```
{% extends "auth_base.html" %}
{% block title %}Inscription{% endblock %}
{% block content %}
<div class="card-x card-pad p-4">
  <h3>Créer un compte</h3><p class="text-muted">Rejoignez la plateforme en tant qu'apprenant.</p>
```

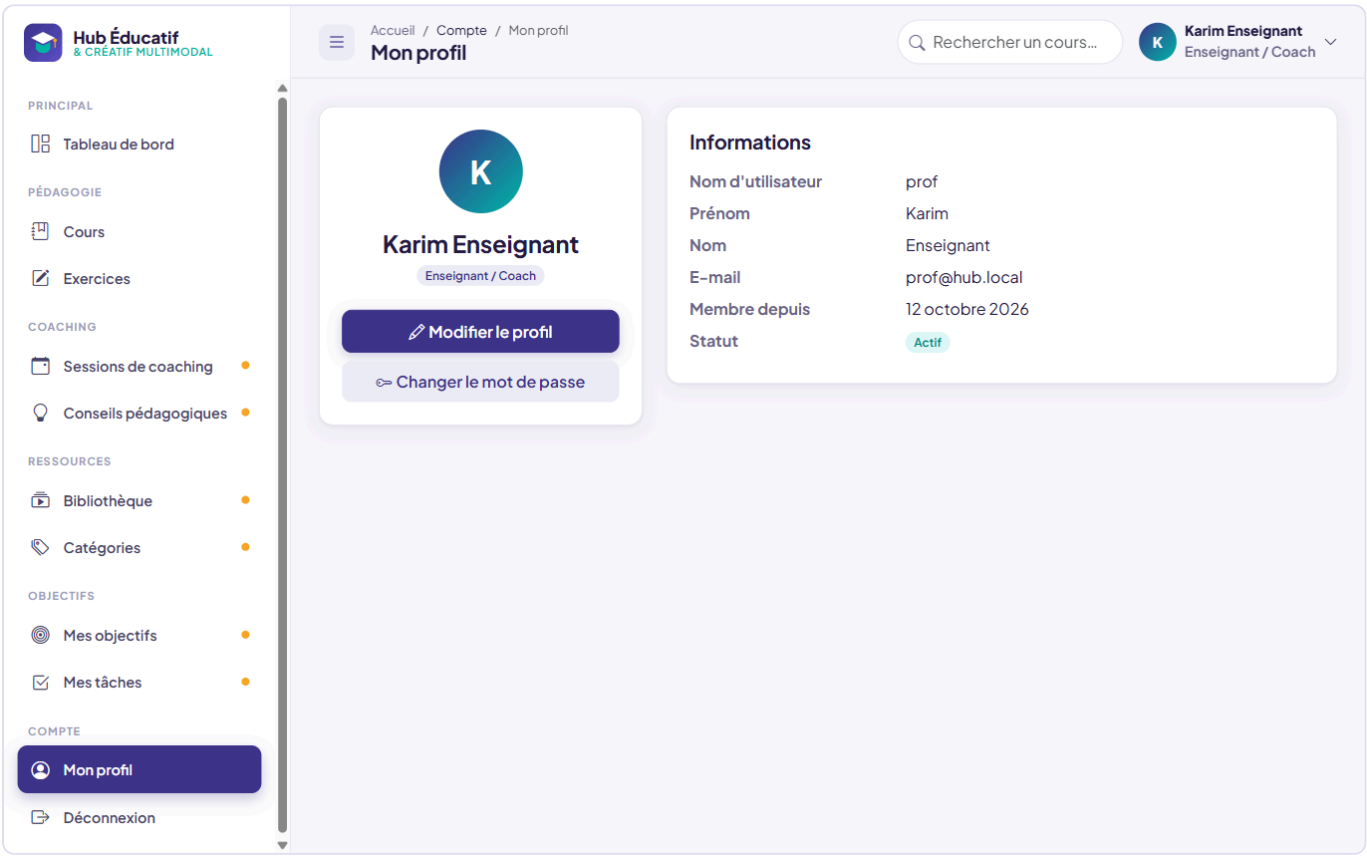
```
  <form method="post" novalidate>{% csrf_token %}
    <div class="row"><div class="col-6">{% include "components/field.html" with field=form.first_name %}</div><div class="col-
6">{% include "components/field.html" with field=form.last_name %}</div></div>
    {% include "components/field.html" with field=form.username %}
    {% include "components/field.html" with field=form.email %}
    {% include "components/field.html" with field=form.password1 %}
    {% include "components/field.html" with field=form.password2 %}
    <button class="btn btn-primary w-100 mt-2">Créer mon compte</button>
  </form>
  <p class="text-center text-muted small mt-4 mb-0">Déjà inscrit ? <a href="{% url 'accounts:login' %}" class="fw-semibold">Se
connecter</a></p>
</div>
{% endblock %}
```

1. `RegisterView` hérite de `CreateView` : affiche le formulaire (GET) puis enregistre (POST). `success_url` = page de connexion.
2. `RegisterForm` hérite de `UserCreationForm` : il valide que les deux mots de passe sont identiques, respectent les règles de sécurité Django et **hache** le mot de passe.
3. Dans `save()`, après la création, l'utilisateur est ajouté au groupe **Apprenant** : toute personne qui s'inscrit est apprenante (l'administrateur peut changer son rôle ensuite).
4. `email` est rendu obligatoire dans `__init__`.

8.3 Profil, modification du profil et mot de passe

Commande / adresse :

```
http://127.0.0.1:8000/comptes/profil/
http://127.0.0.1:8000/comptes/profil/modifier/
http://127.0.0.1:8000/comptes/profil/mot-de-passe/
```



Page « Mon profil »

accounts/views.py — ProfileView, ProfileEditView, PasswordView

```
class ProfileView(TemplateView):
    template_name = 'accounts/profile.html'
```

```
class ProfileEditView(UpdateView):
    form_class = ProfileForm
    template_name = 'accounts/profile_form.html'
    success_url = reverse_lazy('accounts:profile')

    def get_object(self):
        return self.request.user

    def form_valid(self, form):
        messages.success(self.request, 'Profil mis à jour.')
        return super().form_valid(form)
class PasswordView(PasswordChangeView):
    form_class = PasswordForm
    template_name = 'accounts/password_form.html'
    success_url = reverse_lazy('accounts:profile')

    def form_valid(self, form):
        messages.success(self.request, 'Mot de passe modifié.')
        return super().form_valid(form)
```

accounts/forms.py — ProfileForm, PasswordForm

```
class ProfileForm(BootstrapFormMixin, forms.ModelForm):
    class Meta:
        model = User
        fields = ('first_name', 'last_name', 'username', 'email')
class PasswordForm(BootstrapFormMixin, PasswordChangeForm):
    pass
```

templates/accounts/profile.html

```
{% extends "base.html" %}
{% block title %}Mon profil{% endblock %}
{% block page_title %}Mon profil{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Compte</li><li class="breadcrumb-item active">Mon profil</li>{% endblock %}
{% block content %}
<div class="row g-4">
  <div class="col-lg-4"><div class="card-x card-pad text-center">
    <span class="avatar lg mb-3">{{ user.first_name|default:user.username|first|upper }}</span>
    <h4 class="mb-1">{{ user.get_full_name|default:user.username }}</h4>
    <span class="badge badge-soft badge-primary">{{ user.groups.first.name|default:"Sans rôle" }}</span>
    <div class="d-grid gap-2 mt-4">
      <a href="{% url 'accounts:profile_edit' %}" class="btn btn-primary"><i class="bi bi-pencil me-1"></i>Modifier le
profil</a>
      <a href="{% url 'accounts:password' %}" class="btn btn-soft"><i class="bi bi-key me-1"></i>Changer le mot de passe</a>
    </div>
  </div></div>
  <div class="col-lg-8"><div class="card-x card-pad">
    <h5 class="mb-3">Informations</h5>
    <dl class="row mb-0">
      <dt class="col-sm-4 text-muted">Nom d'utilisateur</dt><dd class="col-sm-8">{{ user.username }}</dd>
      <dt class="col-sm-4 text-muted">Prénom</dt><dd class="col-sm-8">{{ user.first_name|default:"—" }}</dd>
      <dt class="col-sm-4 text-muted">Nom</dt><dd class="col-sm-8">{{ user.last_name|default:"—" }}</dd>
      <dt class="col-sm-4 text-muted">E-mail</dt><dd class="col-sm-8">{{ user.email|default:"—" }}</dd>
      <dt class="col-sm-4 text-muted">Membre depuis</dt><dd class="col-sm-8">{{ user.date_joined|date:"d F Y" }}</dd>
      <dt class="col-sm-4 text-muted">Statut</dt><dd class="col-sm-8"><span class="badge badge-soft badge-teal">Actif</span>
</dd>
    </dl>
  </div></div>
</div>
{% endblock %}
```

templates/accounts/profile_form.html

```
{% extends "base.html" %}
{% block title %}Modifier mon profil{% endblock %}
{% block page_title %}Modifier mon profil{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Compte</li><li class="breadcrumb-item"><a href="{% url 'accounts:profile' %}">Mon
profil</a></li><li class="breadcrumb-item active">Modifier</li>{% endblock %}
{% block content %}
<form method="post" class="card-x card-pad" style="max-width:680px" novalidate>{% csrf_token %}
  <div class="row"><div class="col-md-6">{% include "components/field.html" with field=form.first_name %}</div><div class="col-
md-6">{% include "components/field.html" with field=form.last_name %}</div></div>
  {% include "components/field.html" with field=form.username %}
  {% include "components/field.html" with field=form.email %}
  <div class="d-flex gap-2"><button class="btn btn-primary">Enregistrer</button><a href="{% url 'accounts:profile' %}"
class="btn btn-soft">Annuler</a></div>
```

```
</form>
{% endblock %}
```

templates/accounts/password_form.html

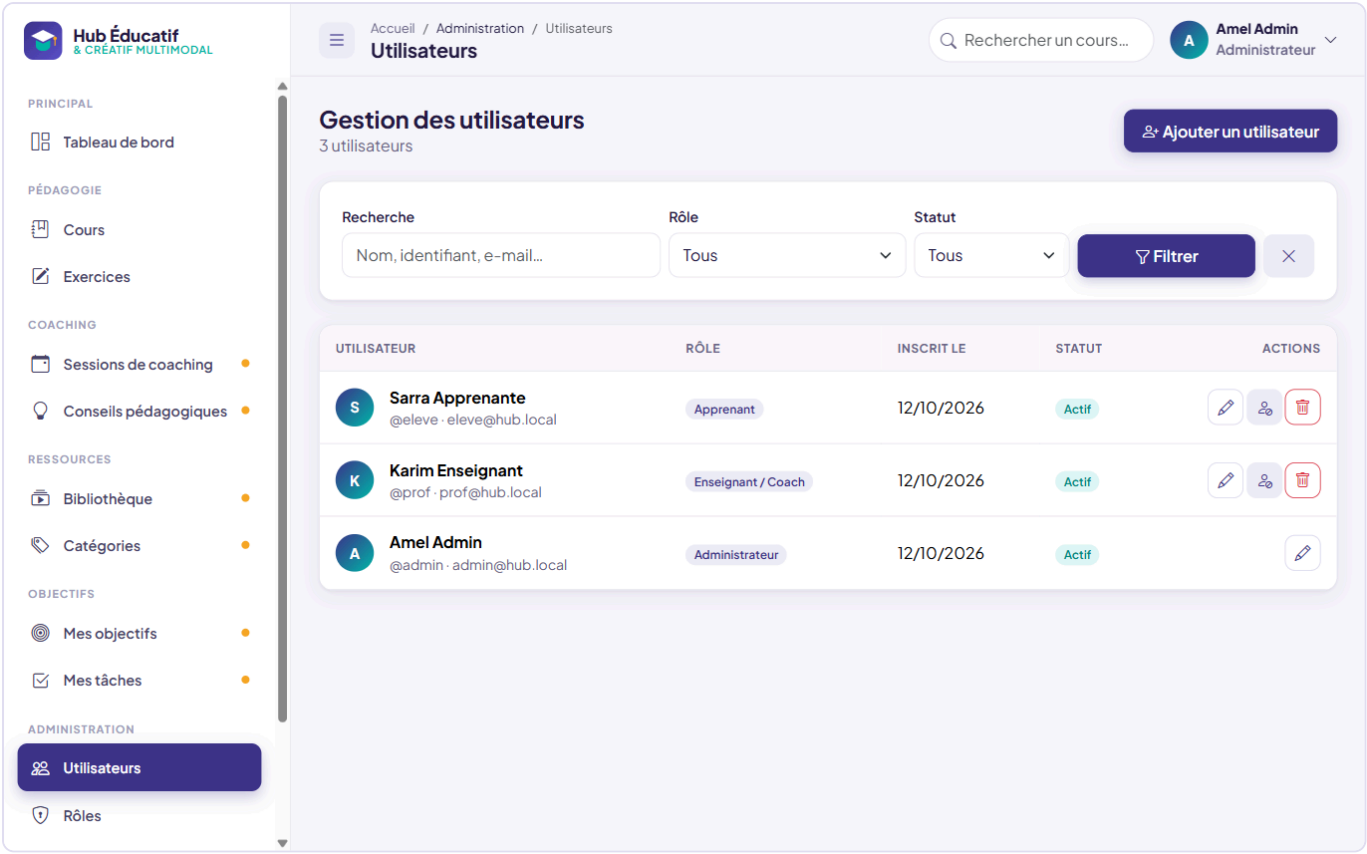
```
{% extends "base.html" %}
{% block title %}Mot de passe{% endblock %}
{% block page_title %}Changer le mot de passe{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Compte</li><li class="breadcrumb-item"><a href="{% url 'accounts:profile' %}">Mon
profil</a></li><li class="breadcrumb-item active">Mot de passe</li>{% endblock %}
{% block content %}
<form method="post" class="card-x card-pad" style="max-width:520px" novalidate>{% csrf_token %}
  {% include "components/field.html" with field=form.old_password %}
  {% include "components/field.html" with field=form.new_password1 %}
  {% include "components/field.html" with field=form.new_password2 %}
  <div class="d-flex gap-2"><button class="btn btn-primary">Enregistrer</button><a href="{% url 'accounts:profile' %}"
class="btn btn-soft">Annuler</a></div>
</form>
{% endblock %}
```

1. `ProfileEditView.get_object()` renvoie `request.user` : on ne peut modifier que **son propre** profil.
2. `PasswordView` utilise `PasswordChangeView` de Django : demande l'ancien mot de passe, valide le nouveau, garde la session ouverte.
3. `form_valid` ajoute un message de succès avec `messages.success` ; il s'affiche en haut de la page (`base.html`).

8.4 Utilisateurs (administrateur) : liste, ajout, modification, désactivation, suppression

Commande / adresse :

```
http://127.0.0.1:8000/comptes/utilisateurs/            (liste, recherche, filtres)
http://127.0.0.1:8000/comptes/utilisateurs/ajouter/    (ajout)
http://127.0.0.1:8000/comptes/utilisateurs/<id>/modifier/
POST /comptes/utilisateurs/<id>/statut/      (activer/désactiver)
POST /comptes/utilisateurs/<id>/supprimer/   (suppression)
```



Liste des utilisateurs (connecté en administrateur)

accounts/views.py — UserListView, UserCreateView, UserUpdateView, UserToggleView, UserDeleteView

```
class UserListView(PermissionRequiredMixin, QueryStringMixin, ListView):
    permission_required = 'auth.view_user'
    template_name = 'accounts/user_list.html'
    paginate_by = 10

    def get_queryset(self):
        qs = User.objects.prefetch_related('groups').order_by('-date_joined')
        g = self.request.GET
        if q := g.get('q', '').strip():
            qs = qs.filter(Q(username__icontains=q) | Q(first_name__icontains=q)
                           | Q(last_name__icontains=q) | Q(email__icontains=q))
        if g.get('role', '').isdigit():
            qs = qs.filter(groups__pk=g['role'])
        if g.get('actif') in ('1', '0'):
            qs = qs.filter(is_active=g['actif'] == '1')
        return qs

    def get_context_data(self, **kw):
        ctx = super().get_context_data(**kw)
        ctx['roles'] = Group.objects.order_by('name')
        return ctx
class UserCreateView(PermissionRequiredMixin, CreateView):
    permission_required = 'auth.add_user'
    form_class = UserCreateForm
    template_name = 'accounts/user_form.html'
    success_url = reverse_lazy('accounts:user_list')

    def form_valid(self, form):
        messages.success(self.request, 'Utilisateur créé.')
        return super().form_valid(form)
class UserUpdateView(PermissionRequiredMixin, UpdateView):
    permission_required = 'auth.change_user'
    model = User
    form_class = UserUpdateForm
    template_name = 'accounts/user_form.html'
    success_url = reverse_lazy('accounts:user_list')

    def form_valid(self, form):
        if self.object == self.request.user and not form.cleaned_data['is_active']:
            form.add_error('is_active', 'Vous ne pouvez pas désactiver votre propre compte.')
            return self.form_invalid(form)
```

```
        messages.success(self.request, 'Utilisateur modifié.')
        return super().form_valid(form)
class UserToggleView(PermissionRequiredMixin, View):
    permission_required = 'auth.change_user'
    http_method_names = ['post']

    def post(self, request, pk):
        user = get_object_or_404(User, pk=pk)
        if user == request.user:
            messages.error(request, 'Vous ne pouvez pas désactiver votre propre compte.')
        else:
            user.is_active = not user.is_active
            user.save(update_fields=['is_active'])
            messages.success(request, f'Compte {"activé" if user.is_active else "désactivé"}.')
        return redirect('accounts:user_list')
class UserDeleteView(PermissionRequiredMixin, DeleteView):
    permission_required = 'auth.delete_user'
    model = User
    http_method_names = ['post']
    success_url = reverse_lazy('accounts:user_list')

    def form_valid(self, form):
        if self.object == self.request.user or self.object.is_superuser:
            messages.error(self.request, 'Ce compte ne peut pas être supprimé.')
        else:
            try:
                self.object.delete()
                messages.success(self.request, 'Utilisateur supprimé.')
            except ProtectedError:
                messages.error(self.request, 'Cet utilisateur possède des cours : désactivez son compte à la place.')
        return redirect(self.success_url)
```

accounts/forms.py — UserCreateForm, UserUpdateForm

```
class UserCreateForm(BootstrapFormMixin, UserCreationForm):
    role = forms.ModelChoiceField(queryset=Group.objects.order_by('name'), label='Rôle')
    is_active = forms.BooleanField(required=False, initial=True, label='Compte actif')

    class Meta:
        model = User
        fields = ('first_name', 'last_name', 'username', 'email')

    def save(self, commit=True):
        user = super().save(commit=False)
        user.is_active = self.cleaned_data['is_active']
        user.save()
        user.groups.set([self.cleaned_data['role']])
        return user
class UserUpdateForm(BootstrapFormMixin, forms.ModelForm):
    role = forms.ModelChoiceField(queryset=Group.objects.order_by('name'), label='Rôle')

    class Meta:
        model = User
        fields = ('first_name', 'last_name', 'username', 'email', 'is_active')
        labels = {'is_active': 'Compte actif'}

    def __init__(self, *a, **kw):
        super().__init__(*a, **kw)
        self.fields['role'].initial = self.instance.groups.first()

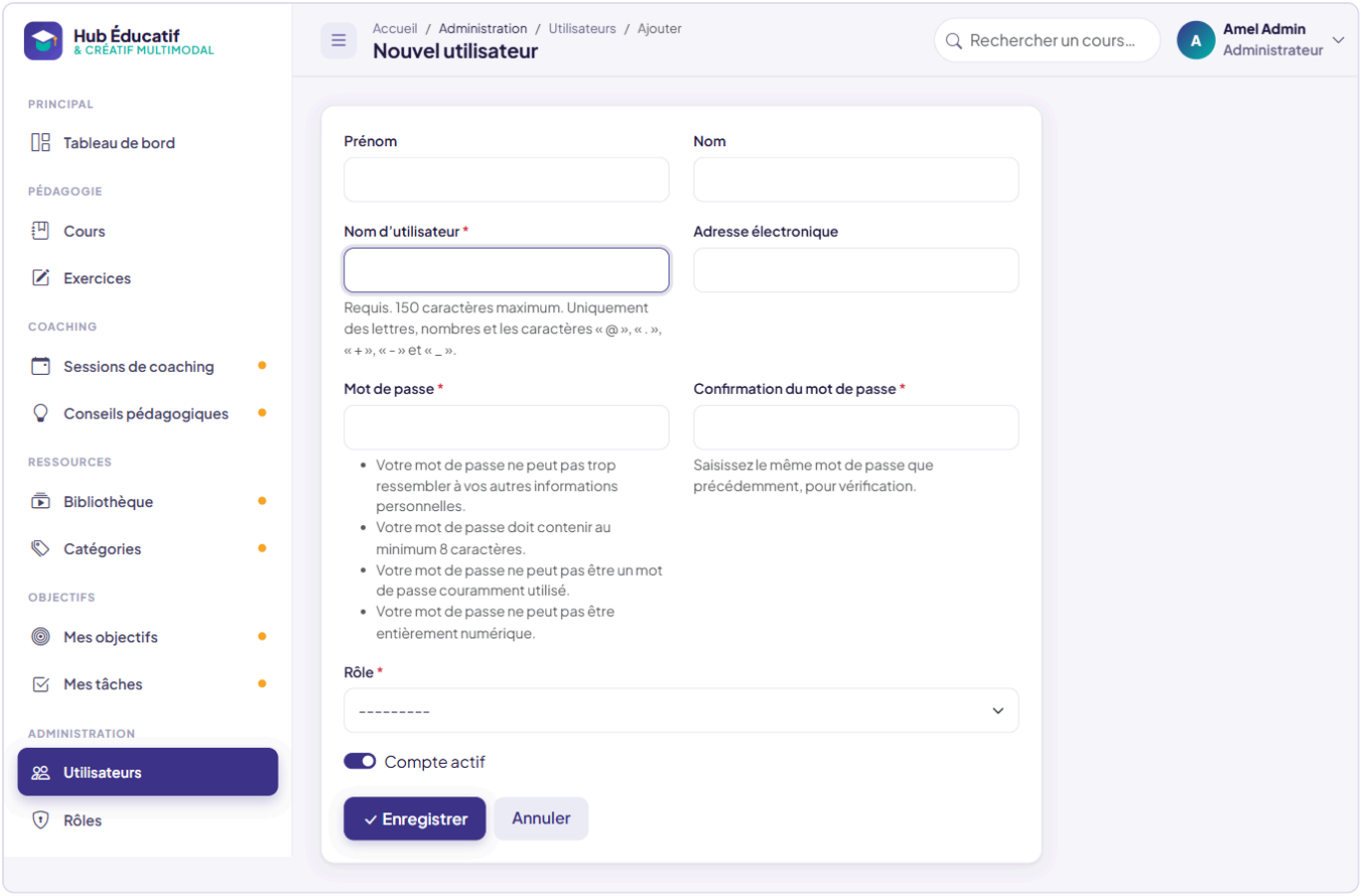
    def save(self, commit=True):
        user = super().save(commit)
        user.groups.set([self.cleaned_data['role']])
        return user
```

templates/accounts/user_list.html

```
{% extends "base.html" %}
{% block title %}Utilisateurs{% endblock %}
{% block page_title %}Utilisateurs{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Administration</li><li class="breadcrumb-item active">Utilisateurs</li>{% endblock
%}
{% block content %}
<div class="page-head">
  <div><h2>Gestion des utilisateurs</h2><p class="text-muted mb-0">{{ paginator.count }} utilisateur{{ paginator.count|pluralize
}}</p></div>
  {% if perms.auth.add_user %}<a href="{% url 'accounts:user_create' %}" class="btn btn-primary"><i class="bi bi-person-plus me-
1"></i>Ajouter un utilisateur</a>{% endif %}
```

```
</div>
<form class="card-x card-pad mb-4" method="get"><div class="row g-2 align-items-end">
  <div class="col-md-4"><label class="form-label">Recherche</label><input class="form-control" name="q" value="{{ request.GET.q
}}" placeholder="Nom, identifiant, e-mail…"></div>
  <div class="col-md-3 col-6"><label class="form-label">Rôle</label><select class="form-select" name="role"><option
value="">Tous</option>{% for r in roles %}<option value="{{ r.pk }}" {% if request.GET.role == r.pk|stringformat:"s"
%}selected{% endif %}>{{ r.name }}</option>{% endfor %}</select></div>
  <div class="col-md-2 col-6"><label class="form-label">Statut</label><select class="form-select" name="actif"><option
value="">Tous</option><option value="1" {% if request.GET.actif == "1" %}selected{% endif %}>Actifs</option><option value="0" {%
if request.GET.actif == "0" %}selected{% endif %}>Désactivés</option></select></div>
  <div class="col-md-3 d-flex gap-2"><button class="btn btn-primary flex-fill"><i class="bi bi-funnel"></i> Filtrer</button><a
href="{% url 'accounts:user_list' %}" class="btn btn-soft"><i class="bi bi-x-lg"></i></a></div>
</div></form>

{% if page_obj %}
<div class="card-x overflow-hidden"><div class="table-responsive"><table class="table table-x">
  <thead><tr><th>Utilisateur</th><th>Rôle</th><th>Inscrit le</th><th>Statut</th><th class="text-end">Actions</th></tr></thead>
  <tbody>
  {% for u in page_obj %}
  <tr>
    <td><div class="d-flex align-items-center gap-3"><span class="avatar">{{ u.first_name|default:u.username|first|upper }}
</span><div><strong>{{ u.get_full_name|default:u.username }}</strong><div class="small text-muted">@{{ u.username }} · {{
u.email|default:"—" }}</div></div></div></td>
    <td><span class="badge badge-soft badge-primary">{{ u.groups.all.0.name|default:"Sans rôle" }}</span></td>
    <td>{{ u.date_joined|date:"d/m/Y" }}</td>
    <td>{% if u.is_active %}<span class="badge badge-soft badge-teal">Actif</span>{% else %}<span class="badge badge-soft badge-
gray">Désactivé</span>{% endif %}</td>
    <td class="text-end text-nowrap">
      {% if perms.auth.change_user %}
      <a href="{% url 'accounts:user_update' u.pk %}" class="btn btn-outline-primary btn-icon" title="Modifier"><i class="bi bi-
pencil"></i></a>
      {% if u != user %}<form method="post" action="{% url 'accounts:user_toggle' u.pk %}" class="d-inline">{% csrf_token %}
<button class="btn btn-soft btn-icon" title="{% if u.is_active %}Désactiver{% else %}Activer{% endif %}"><i class="bi {% if
u.is_active %}bi-person-slash{% else %}bi-person-check{% endif %}"></i></button></form>{% endif %}
      {% endif %}
      {% if perms.auth.delete_user and u != user and not u.is_superuser %}<button class="btn btn-outline-danger btn-icon"
title="Supprimer" data-bs-toggle="modal" data-bs-target="#deleteModal" data-url="{% url 'accounts:user_delete' u.pk %}" data-
name="{{ u.username }}"><i class="bi bi-trash"></i></button>{% endif %}
    </td>
  </tr>
  {% endfor %}
  </tbody></table></div></div>
{% include "components/pagination.html" %}
{% else %}
  {% include "components/empty.html" with icon="bi-people" title="Aucun utilisateur trouvé" text="Aucun résultat pour ces
critères." %}
{% endif %}
{% endblock %}
```



Formulaire d'ajout d'un utilisateur

templates/accounts/user_form.html

```
{% extends "base.html" %}
{% block title %}{% if object %}Modifier l'utilisateur{% else %}Nouvel utilisateur{% endif %}{% endblock %}
{% block page_title %}{% if object %}Modifier l'utilisateur{% else %}Nouvel utilisateur{% endif %}{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Administration</li><li class="breadcrumb-item"><a href="{% url
'accounts:user_list' %}">Utilisateurs</a></li><li class="breadcrumb-item active">{% if object %}Modifier{% else %}Ajouter{%
endif %}</li>{% endblock %}
{% block content %}
<form method="post" class="card-x card-pad" style="max-width:720px" novalidate>{% csrf_token %}
  <div class="row"><div class="col-md-6">{% include "components/field.html" with field=form.first_name %}</div><div class="col-
md-6">{% include "components/field.html" with field=form.last_name %}</div></div>
  <div class="row"><div class="col-md-6">{% include "components/field.html" with field=form.username %}</div><div class="col-md-
6">{% include "components/field.html" with field=form.email %}</div></div>
  {% if form.password1 %}<div class="row"><div class="col-md-6">{% include "components/field.html" with field=form.password1 %}
</div><div class="col-md-6">{% include "components/field.html" with field=form.password2 %}</div></div>{% endif %}
  {% include "components/field.html" with field=form.role %}
  {% include "components/field.html" with field=form.is_active %}
  <div class="d-flex gap-2 pt-2"><button class="btn btn-primary"><i class="bi bi-check-lg me-1"></i>Enregistrer</button><a
href="{% url 'accounts:user_list' %}" class="btn btn-soft">Annuler</a></div>
</form>
{% endblock %}
```

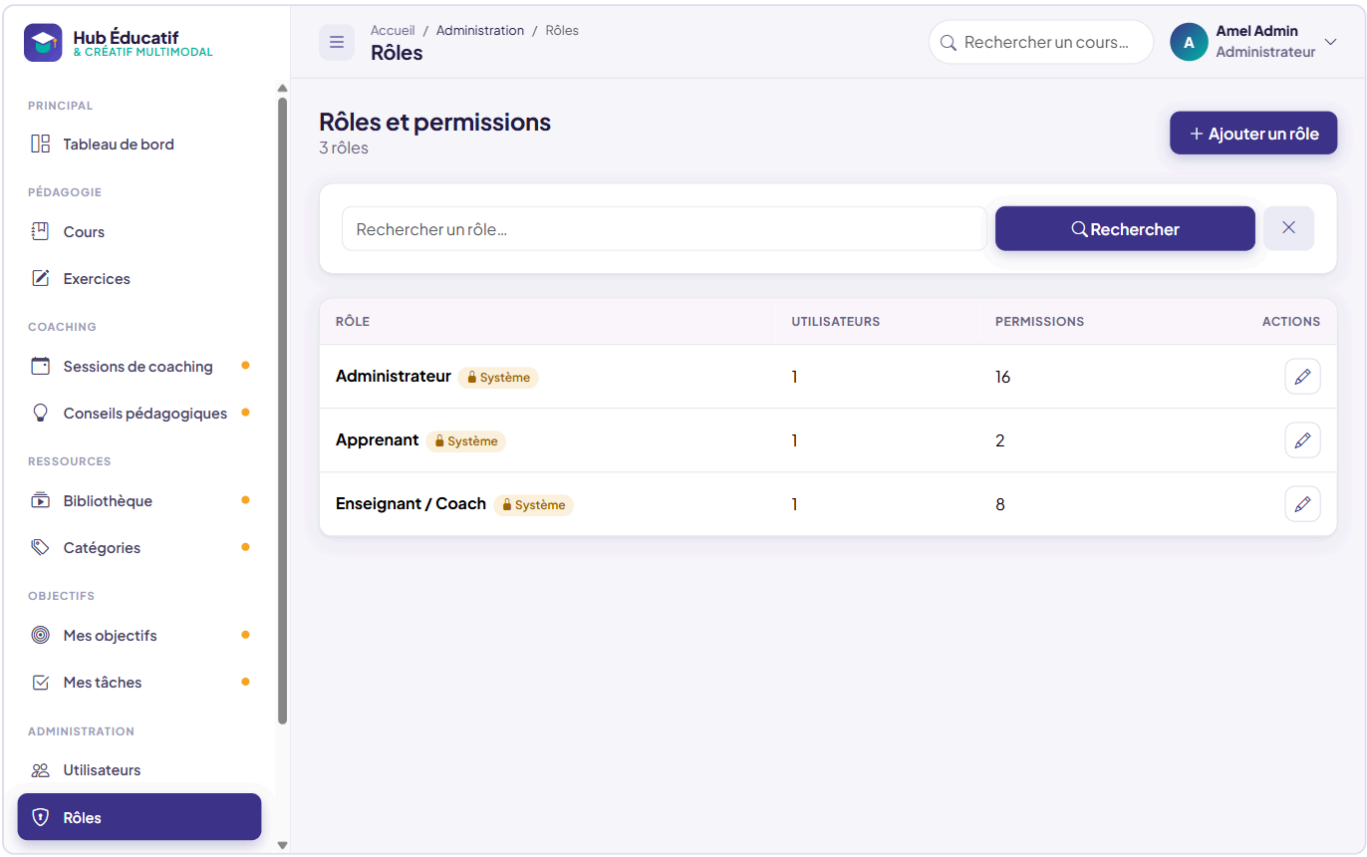
1. `PermissionRequiredMixin` + `permission_required = 'auth.view_user'` : seul un rôle possédant cette permission voit la page (sinon 403).
2. **Recherche et filtres** : `get_queryset()` lit `self.request.GET` (`q`, `role`, `actif`) et construit la requête. `Q(...)|Q(...)` = recherche « OU » sur plusieurs champs ; `icontains` = contient, sans tenir compte de la casse.
3. **Pagination** : `paginate_by = 10` ; le gabarit reçoit `page_obj` et `paginator` ; `components/pagination.html` affiche les numéros.
4. **Création** : `UserCreateForm` ajoute au formulaire standard un champ `role` (liste déroulante des groupes) et `is_active` ; `save()` enregistre l'utilisateur puis `groups.set([role])`.
5. **Modification** : `UserUpdateForm` pré-sélectionne le rôle actuel. On interdit de se désactiver soi-même.

6. **Désactivation** : `UserToggleView` inverse `is_active` (un compte désactivé ne peut plus se connecter mais ses données restent). Réservée au POST.
7. **Suppression sécurisée** : impossible de se supprimer ou de supprimer un superutilisateur ; si l'utilisateur possède des cours, Django lève `ProtectedError` (à cause de `PROTECT`) : la vue l'intercepte et conseille la désactivation.
8. **État vide** : `{% if page_obj %} … {% else %}` inclut `components/empty.html`.

8.5 Rôles : liste, ajout, modification, suppression

Commande / adresse :

```
http://127.0.0.1:8000/comptes/roles/
http://127.0.0.1:8000/comptes/roles/ajouter/
http://127.0.0.1:8000/comptes/roles/<id>/modifier/
POST /comptes/roles/<id>/supprimer/
```



Liste des rôles (rôles système marqués « Système »)

accounts/views.py — RoleListView, RoleCreateView, RoleUpdateView, RoleDeleteView

```
class RoleListView(PermissionRequiredMixin, QueryStringMixin, ListView):
    permission_required = 'auth.view_group'
    template_name = 'accounts/role_list.html'
    paginate_by = 10

    def get_queryset(self):
        qs = Group.objects.annotate(nb_users=Count('user', distinct=True), nb_perms=Count('permissions',
distinct=True)).order_by('name')
        if q := self.request.GET.get('q', '').strip():
            qs = qs.filter(name__icontains=q)
        return qs

    def get_context_data(self, **kw):
        ctx = super().get_context_data(**kw)
        ctx['system_roles'] = SYSTEM_ROLES
        return ctx
class RoleCreateView(PermissionRequiredMixin, CreateView):
```

```
    permission_required = 'auth.add_group'
    form_class = RoleForm
    template_name = 'accounts/role_form.html'
    success_url = reverse_lazy('accounts:role_list')

    def form_valid(self, form):
        messages.success(self.request, 'Rôle créé.')
        return super().form_valid(form)
class RoleUpdateView(PermissionRequiredMixin, UpdateView):
    permission_required = 'auth.change_group'
    model = Group
    form_class = RoleForm
    template_name = 'accounts/role_form.html'
    success_url = reverse_lazy('accounts:role_list')

    def form_valid(self, form):
        messages.success(self.request, 'Rôle modifié.')
        return super().form_valid(form)
class RoleDeleteView(PermissionRequiredMixin, DeleteView):
    permission_required = 'auth.delete_group'
    model = Group
    http_method_names = ['post']
    success_url = reverse_lazy('accounts:role_list')

    def form_valid(self, form):
        if self.object.name in SYSTEM_ROLES:
            messages.error(self.request, 'Les rôles système sont protégés.')
        elif self.object.user_set.exists():
            messages.error(self.request, 'Ce rôle est attribué à des utilisateurs.')
        else:
            self.object.delete()
            messages.success(self.request, 'Rôle supprimé.')
        return redirect(self.success_url)
```

accounts/forms.py — RoleForm

```
class RoleForm(BootstrapFormMixin, forms.ModelForm):
    permissions = forms.ModelMultipleChoiceField(
        queryset=Permission.objects.none(), required=False, widget=forms.CheckboxSelectMultiple
    )

    class Meta:
        model = Group
        fields = ('name', 'permissions')
        labels = {'name': 'Nom du rôle'}

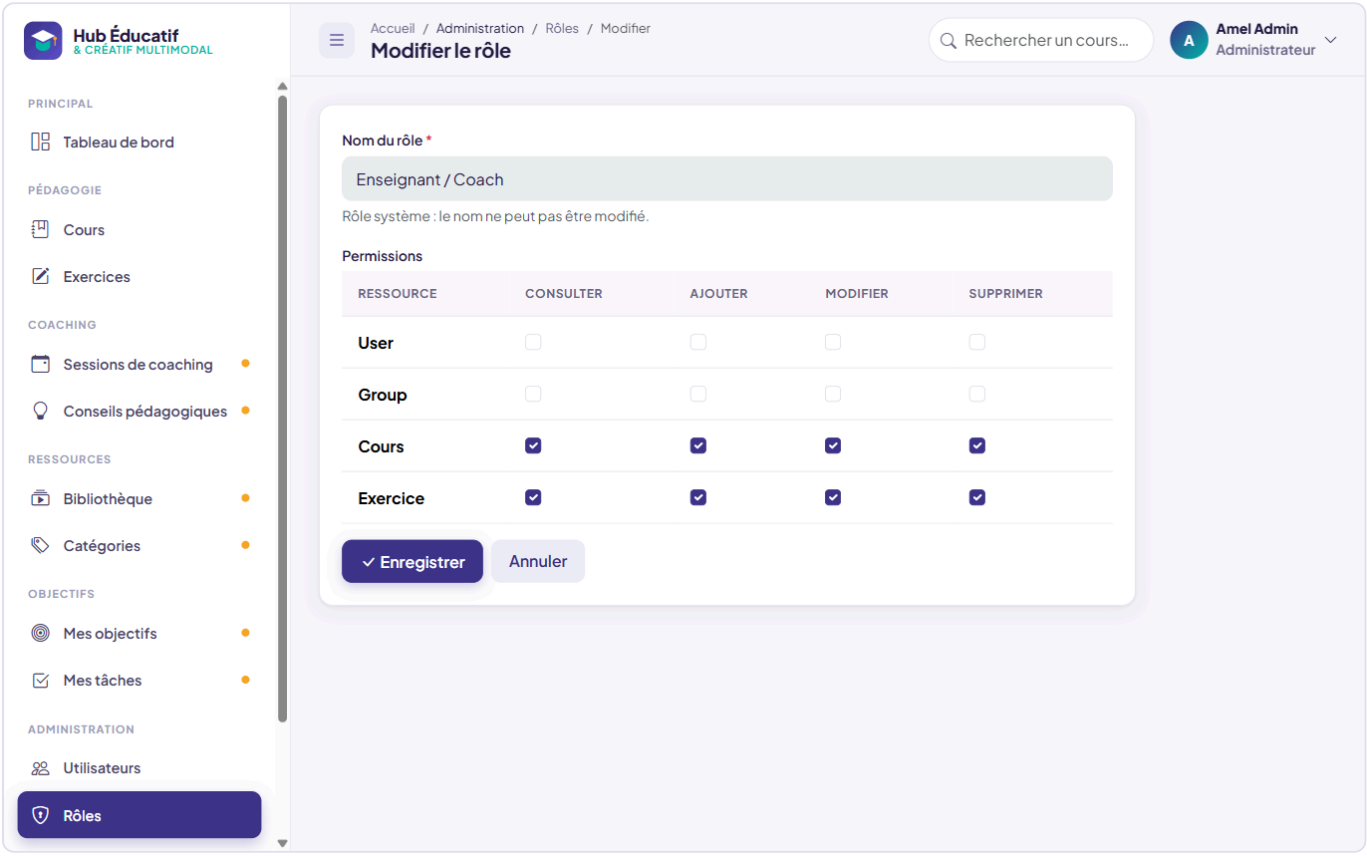
    def __init__(self, *a, **kw):
        super().__init__(*a, **kw)
        perms = Permission.objects.filter(
            content_type__app_label__in=['auth', 'pedagogie'],
            content_type__model__in=[m for _, m in PERM_MODELS],
        ).select_related('content_type')
        self.fields['permissions'].queryset = perms
        if self.instance.pk and self.instance.name in SYSTEM_ROLES:
            self.fields['name'].disabled = True  # rôle système protégé
            self.fields['name'].help_text = 'Rôle système : le nom ne peut pas être modifié.'
        selected = {str(v) for v in (self['permissions'].value() or [])}
        by_code = {(p.content_type.model, p.codename.split('_')[0]): p for p in perms}
        self.perm_groups = [
            (model.capitalize(), [(label, by_code[(model, act)], str(by_code[(model, act)].pk) in selected)
                                  for act, label in ACTIONS.items() if (model, act) in by_code])
            for _, model in PERM_MODELS
        ]
```

templates/accounts/role_list.html

```
{% extends "base.html" %}
{% block title %}Rôles{% endblock %}
{% block page_title %}Rôles{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Administration</li><li class="breadcrumb-item active">Rôles</li>{% endblock %}
{% block content %}
<div class="page-head">
  <div><h2>Rôles et permissions</h2><p class="text-muted mb-0">{{ paginator.count }} rôle{{ paginator.count|pluralize }}</p>
</div>
  {% if perms.auth.add_group %}<a href="{% url 'accounts:role_create' %}" class="btn btn-primary"><i class="bi bi-plus-lg me-1">
</i>Ajouter un rôle</a>{% endif %}
</div>
<form class="card-x card-pad mb-4" method="get"><div class="row g-2"><div class="col-md-8"><input class="form-control" name="q"
```

```
value="{{ request.GET.q }}" placeholder="Rechercher un rôle…"></div><div class="col-md-4 d-flex gap-2"><button class="btn btn-
primary flex-fill"><i class="bi bi-search"></i> Rechercher</button><a href="{% url 'accounts:role_list' %}" class="btn btn-
soft"><i class="bi bi-x-lg"></i></a></div></div></form>

{% if page_obj %}
<div class="card-x overflow-hidden"><div class="table-responsive"><table class="table table-x">
  <thead><tr><th>Rôle</th><th>Utilisateurs</th><th>Permissions</th><th class="text-end">Actions</th></tr></thead>
  <tbody>
  {% for r in page_obj %}
  <tr>
    <td><strong>{{ r.name }}</strong> {% if r.name in system_roles %}<span class="badge badge-soft badge-amber ms-1"><i
class="bi bi-lock-fill"></i> Système</span>{% endif %}</td>
    <td>{{ r.nb_users }}</td><td>{{ r.nb_perms }}</td>
    <td class="text-end text-nowrap">
      {% if perms.auth.change_group %}<a href="{% url 'accounts:role_update' r.pk %}" class="btn btn-outline-primary btn-icon"
title="Modifier"><i class="bi bi-pencil"></i></a>{% endif %}
      {% if perms.auth.delete_group and r.name not in system_roles and r.nb_users == 0 %}<button class="btn btn-outline-danger
btn-icon" title="Supprimer" data-bs-toggle="modal" data-bs-target="#deleteModal" data-url="{% url 'accounts:role_delete' r.pk
%}" data-name="{{ r.name }}"><i class="bi bi-trash"></i></button>{% endif %}
    </td>
  </tr>
  {% endfor %}
  </tbody></table></div></div>
{% include "components/pagination.html" %}
{% else %}
  {% include "components/empty.html" with icon="bi-shield" title="Aucun rôle trouvé" text="Créez un rôle pour regrouper des
permissions." %}
{% endif %}
{% endblock %}
```



Modification d'un rôle : tableau des permissions

templates/accounts/role_form.html

```
{% extends "base.html" %}
{% block title %}{% if object %}Modifier le rôle{% else %}Nouveau rôle{% endif %}{% endblock %}
{% block page_title %}{% if object %}Modifier le rôle{% else %}Nouveau rôle{% endif %}{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Administration</li><li class="breadcrumb-item"><a href="{% url
'accounts:role_list' %}">Rôles</a></li><li class="breadcrumb-item active">{% if object %}Modifier{% else %}Ajouter{% endif %}
</li>{% endblock %}
{% block content %}
<form method="post" class="card-x card-pad" style="max-width:820px" novalidate>{% csrf_token %}
  {% include "components/field.html" with field=form.name %}
  <label class="form-label">Permissions</label>
```

```
  {% for e in form.permissions.errors %}<div class="invalid-feedback">{{ e }}</div>{% endfor %}
  <div class="table-responsive mb-3"><table class="table table-x">
    <thead><tr><th>Ressource</th><th>Consulter</th><th>Ajouter</th><th>Modifier</th><th>Supprimer</th></tr></thead>
    <tbody>
    {% for label, items in form.perm_groups %}
      <tr><td><strong>{{ label }}</strong></td>
      {% for act, perm, checked in items %}{% if perm.codename|slice:":4" == "view" %}<td><input class="form-check-input"
type="checkbox" name="permissions" value="{{ perm.pk }}" {% if checked %}checked{% endif %}></td>{% endif %}{% endfor %}
      {% for act, perm, checked in items %}{% if perm.codename|slice:":3" == "add" %}<td><input class="form-check-input"
type="checkbox" name="permissions" value="{{ perm.pk }}" {% if checked %}checked{% endif %}></td>{% endif %}{% endfor %}
      {% for act, perm, checked in items %}{% if perm.codename|slice:":6" == "change" %}<td><input class="form-check-input"
type="checkbox" name="permissions" value="{{ perm.pk }}" {% if checked %}checked{% endif %}></td>{% endif %}{% endfor %}
      {% for act, perm, checked in items %}{% if perm.codename|slice:":6" == "delete" %}<td><input class="form-check-input"
type="checkbox" name="permissions" value="{{ perm.pk }}" {% if checked %}checked{% endif %}></td>{% endif %}{% endfor %}
      </tr>
    {% endfor %}
    </tbody>
  </table></div>
  <div class="d-flex gap-2"><button class="btn btn-primary"><i class="bi bi-check-lg me-1"></i>Enregistrer</button><a href="{%
url 'accounts:role_list' %}" class="btn btn-soft">Annuler</a></div>
</form>
{% endblock %}
```

1. Un **rôle = un Group** Django. La liste annote chaque rôle avec le nombre d'utilisateurs et de permissions : `annotate(nb_users=Count('user'), nb_perms=Count('permissions'))`.
2. `RoleForm` prépare `perm_groups` : les permissions classées par ressource (Utilisateur, Groupe/Rôle, Cours, Exercice) et par action. Le gabarit les affiche en **tableau de cases à cocher** : cocher/décocher = donner/retirer la permission au rôle (**« contrôler les permissions »**).
3. **Protection des rôles système** : dans `__init__`, si le nom est dans `SYSTEM_ROLES`, le champ nom est `disabled` (non modifiable).
4. `RoleDeleteView` refuse de supprimer un rôle système ou un rôle encore attribué (« supprimer un rôle non utilisé »). Dans la liste, le bouton Supprimer n'est même pas affiché dans ces cas.
5. **Attribuer un rôle à un utilisateur** : se fait dans le formulaire utilisateur (champ « Rôle »).

9. Pages du module pédagogique : cours et exercices

9.1 Le formulaire : pedagogie/forms.py

pedagogie/forms.py

```
from django import forms

from accounts.roles import is_admin
from config.forms import BootstrapFormMixin

from .models import Cours, Exercice


class CoursForm(BootstrapFormMixin, forms.ModelForm):
    class Meta:
        model = Cours
        fields = ('titre', 'description', 'matiere', 'niveau', 'contenu')
        widgets = {'description': forms.Textarea(attrs={'rows': 3}), 'contenu': forms.Textarea(attrs={'rows': 10})}


class ExerciceForm(BootstrapFormMixin, forms.ModelForm):
    class Meta:
        model = Exercice
        fields = ('titre', 'cours', 'type_exercice', 'difficulte', 'question', 'propositions', 'bonne_reponse')
        widgets = {
            'question': forms.Textarea(attrs={'rows': 3}),
            'propositions': forms.Textarea(attrs={'rows': 4, 'placeholder': 'Choix A\nChoix B\nChoix C'}),
            'bonne_reponse': forms.Textarea(attrs={'rows': 2}),
        }

    def __init__(self, *args, user, **kwargs):
        super().__init__(*args, **kwargs)
        qs = Cours.objects.all() if is_admin(user) else Cours.objects.filter(enseignant=user)
        self.fields['cours'].queryset = qs
```

1. `ModelForm` : génère automatiquement le formulaire à partir du modèle (`Meta.model`, `Meta.fields`). La validation du modèle (`clean()`) est réutilisée.
2. `widgets` : personnalise l'affichage (nombre de lignes des zones de texte, texte d'exemple).
3. `ExerciceForm.__init__(user=...)` : limite la liste « Cours » à ceux de l'enseignant connecté (tous pour l'administrateur).

9.2 Les URL : pedagogie/urls.py

pedagogie/urls.py

```
from django.urls import path

from . import views as v

app_name = 'pedagogie'
urlpatterns = [
    path('cours/', v.CoursListView.as_view(), name='cours_list'),
    path('cours/ajouter/', v.CoursCreateView.as_view(), name='cours_create'),
    path('cours/<int:pk>/', v.CoursDetailView.as_view(), name='cours_detail'),
    path('cours/<int:pk>/modifier/', v.CoursUpdateView.as_view(), name='cours_update'),
    path('cours/<int:pk>/supprimer/', v.CoursDeleteView.as_view(), name='cours_delete'),
    path('exercices/', v.ExerciceListView.as_view(), name='exercice_list'),
    path('exercices/ajouter/', v.ExerciceCreateView.as_view(), name='exercice_create'),
    path('exercices/<int:pk>/', v.ExerciceDetailView.as_view(), name='exercice_detail'),
    path('exercices/<int:pk>/modifier/', v.ExerciceUpdateView.as_view(), name='exercice_update'),
    path('exercices/<int:pk>/supprimer/', v.ExerciceDeleteView.as_view(), name='exercice_delete'),
]
```

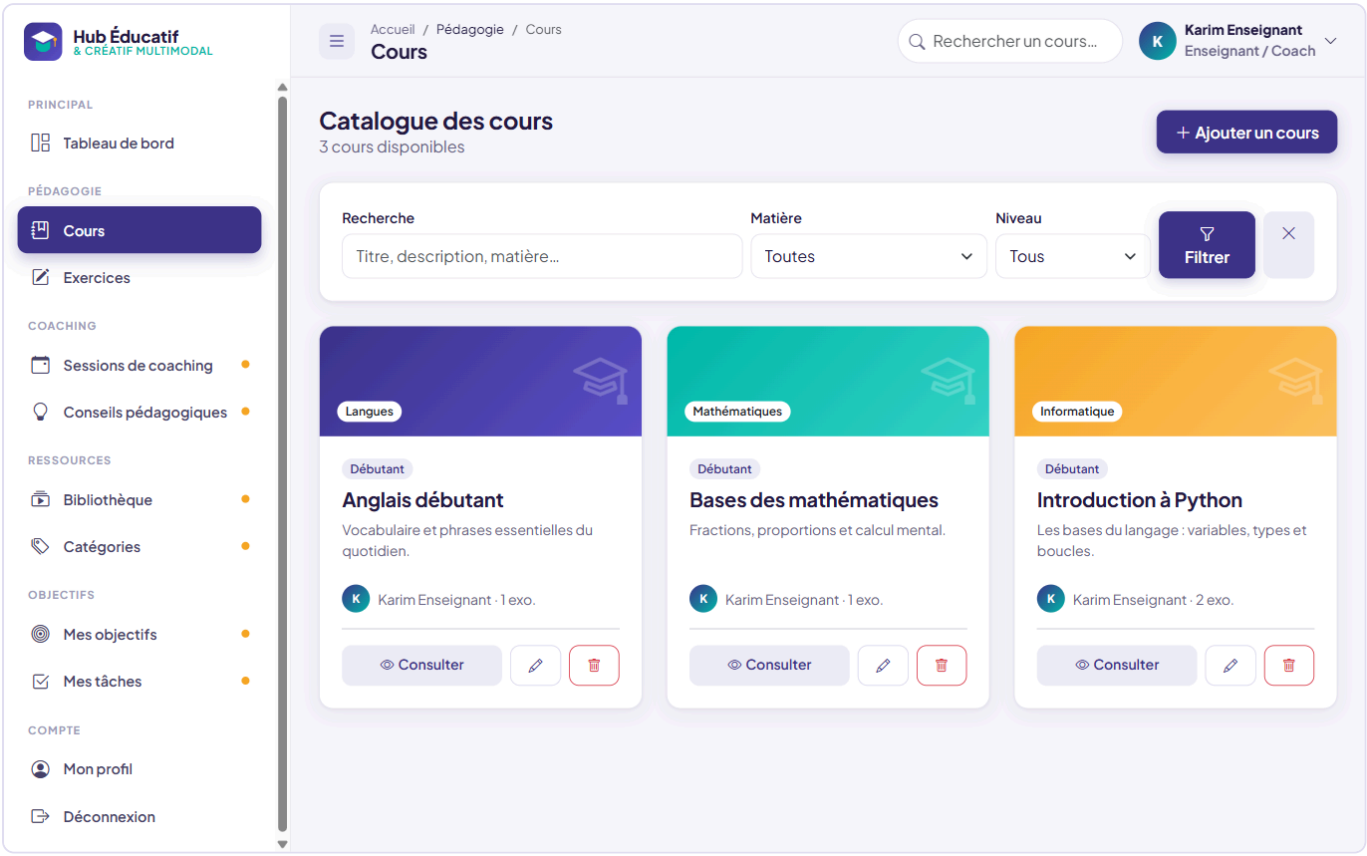
Le préfixe `pedagogie/` vient de `config/urls.py`. `app_name = 'pedagogie'` crée l'espace de noms : on écrit `{% url 'pedagogie:cours_list' %}` dans les gabarits au lieu de l'adresse en dur.

9.3 Liste des cours (cartes) avec recherche, filtres, pagination

Commande / adresse :

`http://127.0.0.1:8000/pedagogie/cours/  (pedagogie:cours_list)`

`Exemple filtré : /pedagogie/cours/?q=python&niveau=debutant`



Cours : cartes pédagogiques, recherche, filtres matière et niveau

pedagogie/views.py — CoursListView

```
class CoursListView(PermissionRequiredMixin, QueryStringMixin, ListView):
    permission_required = 'pedagogie.view_cours'
    template_name = 'pedagogie/cours_list.html'
    paginate_by = 9

    def get_queryset(self):
        qs = Cours.objects.select_related('enseignant')
        g = self.request.GET
        if q := g.get('q', '').strip():
            qs = qs.filter(Q(titre__icontains=q) | Q(description__icontains=q) | Q(matiere__icontains=q))
        if g.get('matiere'):
            qs = qs.filter(matiere=g['matiere'])
        if g.get('niveau'):
            qs = qs.filter(niveau=g['niveau'])
        return qs

    def get_context_data(self, **kw):
        ctx = super().get_context_data(**kw)
        ctx['matieres'] = Cours.objects.order_by('matiere').values_list('matiere', flat=True).distinct()
        ctx['niveaux'] = Cours.NIVEAUX
        return ctx
```

templates/pedagogie/cours_list.html

```
{% extends "base.html" %}
{% load pedagogie_tags %}
{% block title %}Cours{% endblock %}
{% block page_title %}Cours{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Pédagogie</li><li class="breadcrumb-item active">Cours</li>{% endblock %}

{% block content %}
```

```
<div class="page-head">
  <div><h2>Catalogue des cours</h2><p class="text-muted mb-0">{{ paginator.count }} cours disponible{{ paginator.count|pluralize
}}</p></div>
  {% if perms.pedagogie.add_cours %}<a href="{% url 'pedagogie:cours_create' %}" class="btn btn-primary"><i class="bi bi-plus-lg
me-1"></i>Ajouter un cours</a>{% endif %}
</div>

<form class="card-x card-pad mb-4" method="get">
  <div class="row g-2 align-items-end">
    <div class="col-md-5"><label class="form-label">Recherche</label><input class="form-control" name="q" value="{{
request.GET.q }}" placeholder="Titre, description, matière…"></div>
    <div class="col-md-3 col-6"><label class="form-label">Matière</label>
      <select class="form-select" name="matiere"><option value="">Toutes</option>{% for m in matieres %}<option value="{{ m }}"
{% if request.GET.matiere == m %}selected{% endif %}>{{ m }}</option>{% endfor %}</select></div>
    <div class="col-md-2 col-6"><label class="form-label">Niveau</label>
      <select class="form-select" name="niveau"><option value="">Tous</option>{% for k, l in niveaux %}<option value="{{ k }}"
{% if request.GET.niveau == k %}selected{% endif %}>{{ l }}</option>{% endfor %}</select></div>
    <div class="col-md-2 d-flex gap-2"><button class="btn btn-primary flex-fill"><i class="bi bi-funnel"></i> Filtrer</button><a
href="{% url 'pedagogie:cours_list' %}" class="btn btn-soft" title="Réinitialiser"><i class="bi bi-x-lg"></i></a></div>
  </div>
</form>

{% if page_obj %}
<div class="row g-4">
  {% for c in page_obj %}
  <div class="col-md-6 col-xl-4">
    <article class="card-x hover h-100 d-flex flex-column">
      <div class="course-cover {% cycle '' 'c-1' 'c-2' %}">
        <i class="bi bi-mortarboard"></i><span class="badge badge-soft bg-white text-dark">{{ c.matiere }}</span>
      </div>
      <div class="card-pad d-flex flex-column flex-fill">
        <span class="badge badge-soft badge-primary align-self-start mb-2">{{ c.get_niveau_display }}</span>
        <h5><a href="{{ c.get_absolute_url }}" class="text-reset stretched-link-off">{{ c.titre }}</a></h5>
        <p class="text-muted small clamp-2">{{ c.description }}</p>
        <div class="d-flex align-items-center gap-2 small text-muted mt-auto pt-2">
          <span class="avatar" style="width:28px;height:28px;font-size:.7rem">{{
c.enseignant.first_name|default:c.enseignant.username|first|upper }}</span>
          {{ c.enseignant.get_full_name|default:c.enseignant.username }} · {{ c.exercices.count }} exo.
        </div>
        <hr class="my-3">
        <div class="d-flex gap-2">
          <a href="{{ c.get_absolute_url }}" class="btn btn-soft btn-sm flex-fill"><i class="bi bi-eye me-1"></i>Consulter</a>
          {% if c|can_edit:user %}<a href="{% url 'pedagogie:cours_update' c.pk %}" class="btn btn-outline-primary btn-sm"
title="Modifier"><i class="bi bi-pencil"></i></a>{% endif %}
          {% if c|can_delete:user %}<button class="btn btn-outline-danger btn-sm" title="Supprimer" data-bs-toggle="modal" data-
bs-target="#deleteModal" data-url="{% url 'pedagogie:cours_delete' c.pk %}" data-name="{{ c.titre }}"><i class="bi bi-trash">
</i></button>{% endif %}
        </div>
      </div>
    </article>
  </div>
  {% endfor %}
</div>
{% include "components/pagination.html" %}
{% else %}
  {% include "components/empty.html" with icon="bi-journal-x" title="Aucun cours trouvé" text="Modifiez vos filtres ou ajoutez
un premier cours." %}
{% endif %}
{% endblock %}
```

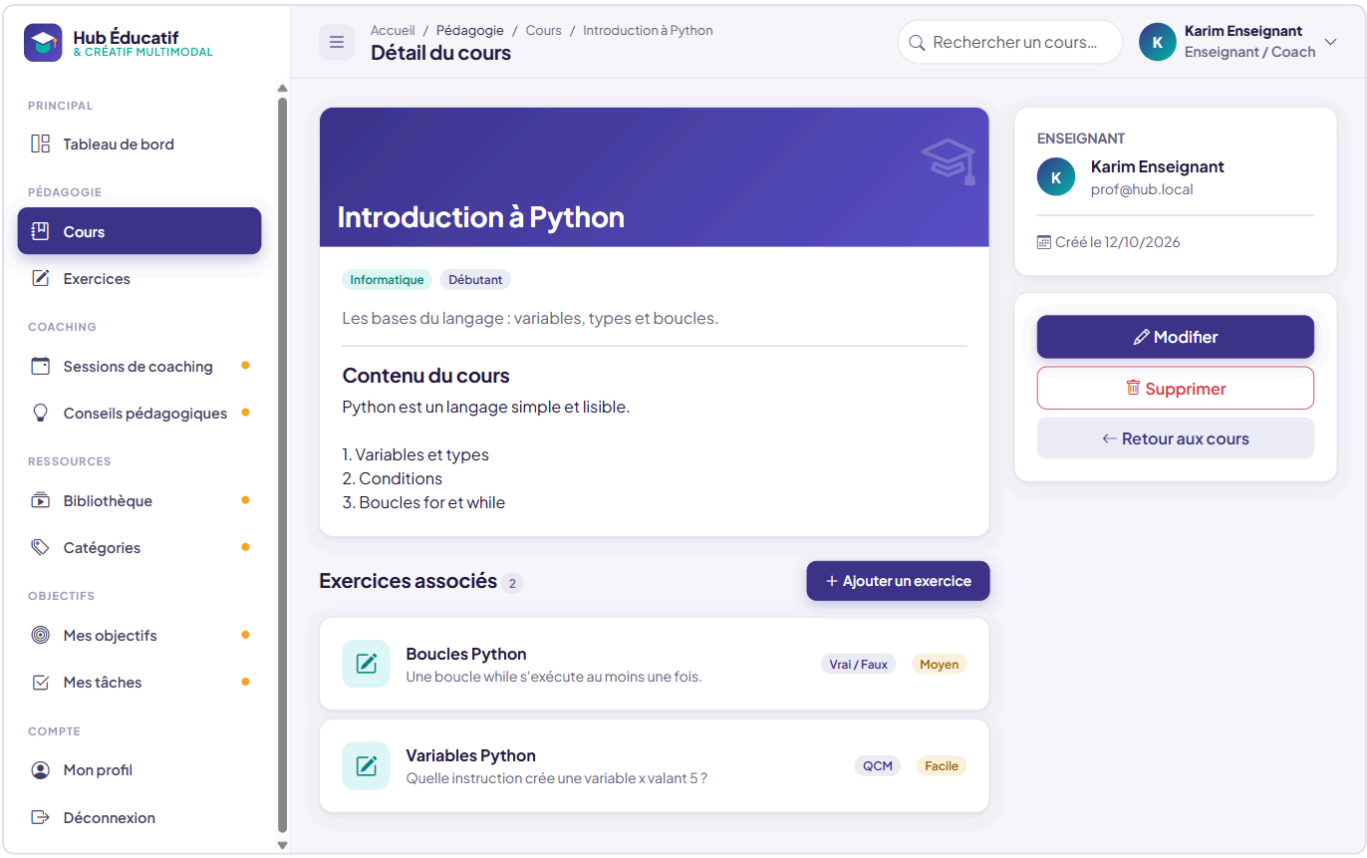
1. `get_queryset()` part de `Cours.objects.select_related('enseignant')` (charge l'enseignant en une seule requête SQL) puis applique les filtres reçus dans l'URL.
2. Le formulaire de recherche est un `<form method="get">` : les critères apparaissent dans l'adresse (`?q=…&matiere=…`), donc la page peut être partagée et la pagination les conserve (`querystring`).
3. La liste des matières du menu vient de `Cours.objects.values_list('matiere').distinct()` : elle se met à jour toute seule.
4. Chaque carte : couverture colorée (`{% cycle '' 'c-1' 'c-2' %}` alterne 3 couleurs), badge de niveau, titre, description limitée à 2 lignes, enseignant et nombre d'exercices.
5. Boutons : **Consulter** toujours ; **Modifier** et **Supprimer** seulement si `c|can_edit:user` / `c|can_delete:user`.
6. Le bouton Supprimer ouvre la **modale de confirmation** de `base.html` ; `data-url` contient l'adresse de suppression et `data-name` le titre affiché.

7. S'il n'y a aucun résultat : composant `empty.html` (état vide agréable).

9.4 Détail d'un cours (avec ses exercices)

Commande / adresse :

`http://127.0.0.1:8000/pedagogie/cours/1/` (pedagogie:cours_detail)



Détail d'un cours et exercices associés

pedagogie/views.py — CoursDetailView

```
class CoursDetailView(PermissionRequiredMixin, DetailView):
    permission_required = 'pedagogie.view_cours'
    model = Cours
    template_name = 'pedagogie/cours_detail.html'
```

templates/pedagogie/cours_detail.html

```
{% extends "base.html" %}
{% load pedagogie_tags %}
{% block title %}{{ object.titre }}{% endblock %}
{% block page_title %}Détail du cours{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Pédagogie</li><li class="breadcrumb-item"><a href="{% url 'pedagogie:cours_list'
%}">Cours</a></li><li class="breadcrumb-item active">{{ object.titre|truncatechars:30 }}</li>{% endblock %}

{% block content %}
<div class="row g-4">
  <div class="col-lg-8">
    <div class="card-x overflow-hidden">
      <div class="course-cover" style="height:140px"><i class="bi bi-mortarboard"></i><h3 class="mb-0">{{ object.titre }}</h3>
</div>
      <div class="card-pad">
        <div class="d-flex flex-wrap gap-2 mb-3">
          <span class="badge badge-soft badge-teal">{{ object.matiere }}</span>
          <span class="badge badge-soft badge-primary">{{ object.get_niveau_display }}</span>
        </div>
        <p class="lead fs-6 text-muted">{{ object.description }}</p>
        <hr>
        <h5>Contenu du cours</h5>
        <div style="white-space: pre-line">{{ object.contenu }}</div>
```

```
      </div>
    </div>

    <div class="d-flex justify-content-between align-items-center mt-4 mb-3">
      <h5 class="mb-0">Exercices associés <span class="badge badge-soft badge-gray">{{ object.exercices.count }}</span></h5>
      {% if object|can_edit:user and perms.pedagogie.add_exercice %}<a href="{% url 'pedagogie:exercice_create' %}?cours={{
object.pk }}" class="btn btn-primary btn-sm"><i class="bi bi-plus-lg me-1"></i>Ajouter un exercice</a>{% endif %}
    </div>
    {% for e in object.exercices.all %}
      <a href="{{ e.get_absolute_url }}" class="card-x hover card-pad d-flex align-items-center gap-3 mb-2 text-reset">
        <div class="icon-box tone-teal"><i class="bi bi-pencil-square"></i></div>
        <div class="flex-fill"><strong>{{ e.titre }}</strong><div class="small text-muted clamp-2">{{ e.question }}</div></div>
        <span class="badge badge-soft badge-primary">{{ e.get_type_exercice_display }}</span>
        <span class="badge badge-soft badge-amber">{{ e.get_difficulte_display }}</span>
      </a>
    {% empty %}
      {% include "components/empty.html" with icon="bi-pencil" title="Aucun exercice" text="Ce cours ne contient pas encore
d'exercice." %}
    {% endfor %}
  </div>

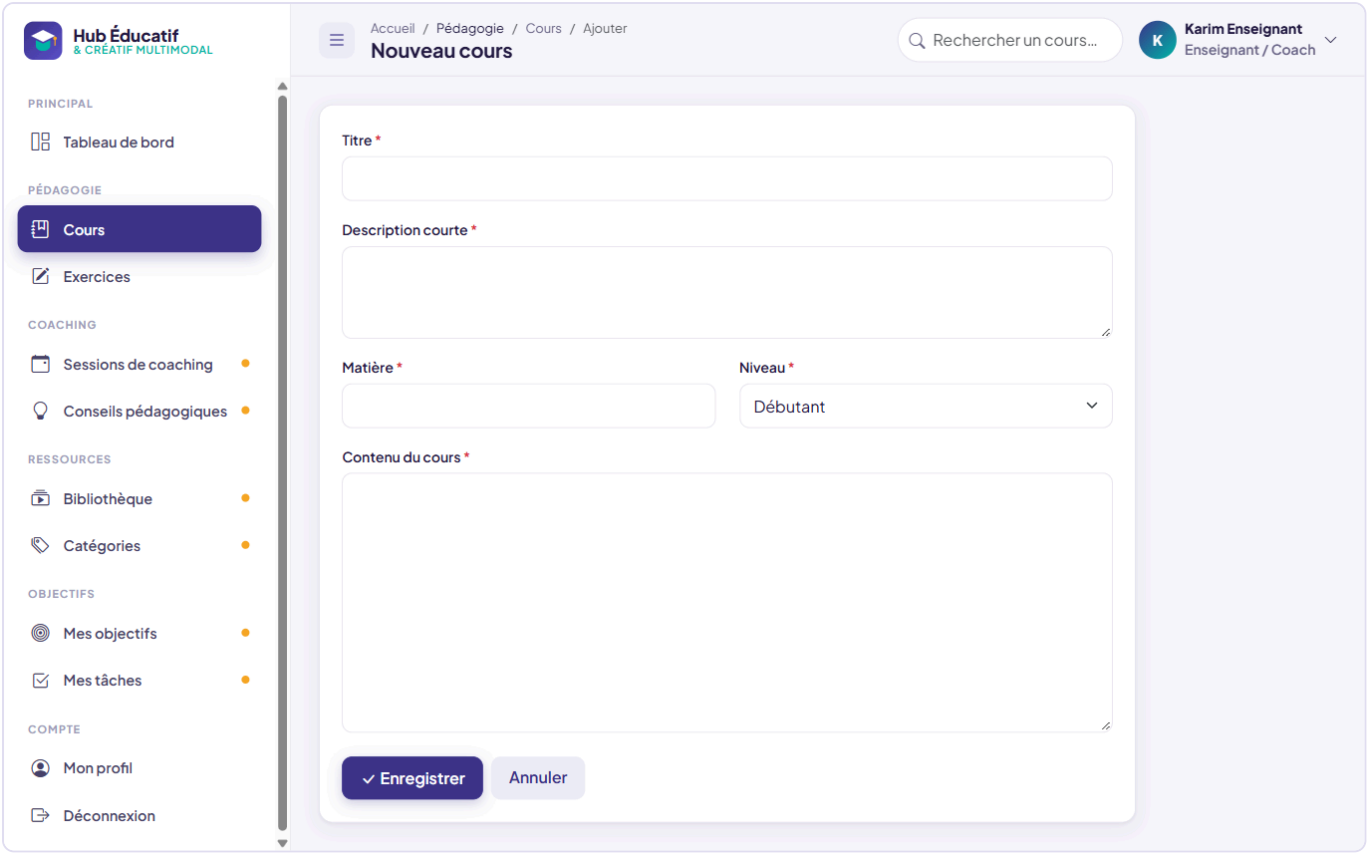
  <div class="col-lg-4">
    <div class="card-x card-pad mb-3">
      <h6 class="text-muted text-uppercase small">Enseignant</h6>
      <div class="d-flex align-items-center gap-3">
        <span class="avatar">{{ object.enseignant.first_name|default:object.enseignant.username|first|upper }}</span>
        <div><strong>{{ object.enseignant.get_full_name|default:object.enseignant.username }}</strong><div class="small text-
muted">{{ object.enseignant.email }}</div></div>
      </div>
      <hr>
      <div class="small text-muted"><i class="bi bi-calendar3 me-1"></i>Créé le {{ object.date_creation|date:"d/m/Y" }}</div>
    </div>
    <div class="card-x card-pad d-grid gap-2">
      {% if object|can_edit:user %}<a href="{% url 'pedagogie:cours_update' object.pk %}" class="btn btn-primary"><i class="bi
bi-pencil me-1"></i>Modifier</a>{% endif %}
      {% if object|can_delete:user %}<button class="btn btn-outline-danger" data-bs-toggle="modal" data-bs-target="#deleteModal"
data-url="{% url 'pedagogie:cours_delete' object.pk %}" data-name="{{ object.titre }}"><i class="bi bi-trash me-1">
</i>Supprimer</button>{% endif %}
      <a href="{% url 'pedagogie:cours_list' %}" class="btn btn-soft"><i class="bi bi-arrow-left me-1"></i>Retour aux cours</a>
    </div>
  </div>
</div>
{% endblock %}
```

1. `DetailView` récupère le cours par son `pk` (numéro dans l'URL) et le place dans le gabarit sous le nom `object`.
2. `object.exercices.all` : grâce à `related_name='exercices'`, on obtient les exercices du cours (relation 1-N).
3. Le bouton « Ajouter un exercice » envoie `?cours={{ object.pk }}` : le formulaire d'exercice s'ouvre avec ce cours déjà choisi (`get_initial`).

9.5 Ajouter / modifier un cours

Commande / adresse :

```
http://127.0.0.1:8000/pedagogie/cours/ajouter/              (pedagogie:cours_create)
http://127.0.0.1:8000/pedagogie/cours/<id>/modifier/     (pedagogie:cours_update)
```



Formulaire de cours

pedagogie/views.py — CoursCreateView, CoursUpdateView, CoursDeleteView, DeleteMessageMixin

```
class CoursCreateView(PermissionRequiredMixin, CreateView):
    permission_required = 'pedagogie.add_cours'
    form_class = CoursForm
    template_name = 'pedagogie/cours_form.html'

    def form_valid(self, form):
        form.instance.enseignant = self.request.user
        messages.success(self.request, 'Cours créé.')
        return super().form_valid(form)
class CoursUpdateView(PermissionRequiredMixin, OwnerOrAdminMixin, UpdateView):
    permission_required = 'pedagogie.change_cours'
    model = Cours
    form_class = CoursForm
    template_name = 'pedagogie/cours_form.html'

    def form_valid(self, form):
        messages.success(self.request, 'Cours modifié.')
        return super().form_valid(form)
class CoursDeleteView(PermissionRequiredMixin, OwnerOrAdminMixin, DeleteMessageMixin, DeleteView):
    permission_required = 'pedagogie.delete_cours'
    model = Cours
    success_url = reverse_lazy('pedagogie:cours_list')
class DeleteMessageMixin:
    http_method_names = ['post']

    def form_valid(self, form):
        messages.success(self.request, 'Suppression effectuée.')
        return super().form_valid(form)
```

templates/pedagogie/cours_form.html

```
{% extends "base.html" %}
{% block title %}{% if object %}Modifier le cours{% else %}Nouveau cours{% endif %}{% endblock %}
{% block page_title %}{% if object %}Modifier le cours{% else %}Nouveau cours{% endif %}{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Pédagogie</li><li class="breadcrumb-item"><a href="{% url 'pedagogie:cours_list'
%}">Cours</a></li><li class="breadcrumb-item active">{% if object %}Modifier{% else %}Ajouter{% endif %}</li>{% endblock %}

{% block content %}
<form method="post" class="card-x card-pad" style="max-width: 820px" novalidate>{% csrf_token %}
  {% if form.non_field_errors %}<div class="alert alert-danger">{{ form.non_field_errors }}</div>{% endif %}
```

```
  {% include "components/field.html" with field=form.titre %}
  {% include "components/field.html" with field=form.description %}
  <div class="row"><div class="col-md-6">{% include "components/field.html" with field=form.matiere %}</div><div class="col-md-
6">{% include "components/field.html" with field=form.niveau %}</div></div>
  {% include "components/field.html" with field=form.contenu %}
  <div class="d-flex gap-2 pt-2"><button class="btn btn-primary"><i class="bi bi-check-lg me-1"></i>Enregistrer</button><a
href="{% if object %}{{ object.get_absolute_url }}{% else %}{% url 'pedagogie:cours_list' %}{% endif %}" class="btn btn-
soft">Annuler</a></div>
</form>
{% endblock %}
```

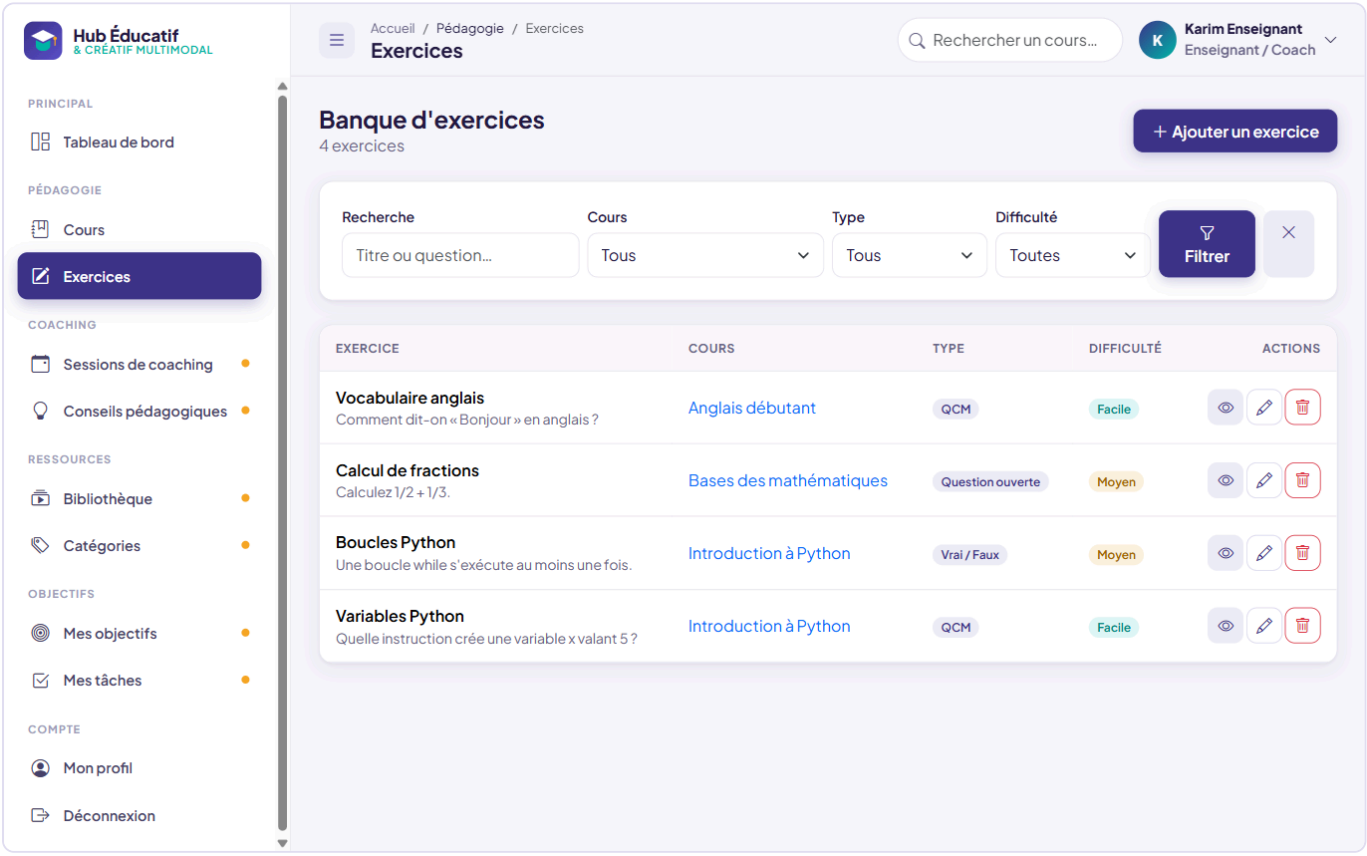
1. **Même gabarit** pour l'ajout et la modification : `{% if object %}` distingue les deux cas (en modification, `object` existe).
2. `CoursCreateView.form_valid` met `form.instance.enseignant = request.user` : l'enseignant n'est **pas** un champ du formulaire, il est défini automatiquement (impossible de créer un cours au nom d'un autre).
3. **Validation** : si un champ requis est vide, le formulaire se réaffiche avec les messages d'erreur (`field.errors` dans `field.html`). Les champs requis portent une étoile rouge.
4. **Modification/Suppression** : `PermissionRequiredMixin` (permission du rôle) + `OwnerOrAdminMixin` (propriétaire ou admin) → sinon 403.
5. `CoursDeleteView` accepte seulement le POST (`http_method_names = ['post']`) : on ne peut pas supprimer en ouvrant simplement une adresse ; il faut passer par la modale.
6. Boutons **Enregistrer** (soumet) et **Annuler** (lien de retour).

9.6 Liste des exercices

Commande / adresse :

`http://127.0.0.1:8000/pedagogie/exercices/` (pedagogie:exercice_list)

`Exemple : /pedagogie/exercices/?type_exercice=qcm&difficulte=facile&cours=1`



Exercices : tableau avec recherche et filtres

`pedagogie/views.py — ExerciceListView`

```
class ExerciceListView(PermissionRequiredMixin, QueryStringMixin, ListView):
    permission_required = 'pedagogie.view_exercice'
    template_name = 'pedagogie/exercice_list.html'
    paginate_by = 10

    def get_queryset(self):
        qs = Exercice.objects.select_related('cours')
        g = self.request.GET
        if q := g.get('q', '').strip():
            qs = qs.filter(Q(titre__icontains=q) | Q(question__icontains=q))
        for field in ('type_exercice', 'difficulte'):
            if g.get(field):
                qs = qs.filter(**{field: g[field]})
        if g.get('cours', '').isdigit():
            qs = qs.filter(cours_id=g['cours'])
        return qs

    def get_context_data(self, **kw):
        ctx = super().get_context_data(**kw)
        ctx.update(types=Exercice.TYPES, difficultes=Exercice.DIFFICULTES, cours_all=Cours.objects.order_by('titre'))
        return ctx
```

templates/pedagogie/exercice_list.html

```
{% extends "base.html" %}
{% load pedagogie_tags %}
{% block title %}Exercices{% endblock %}
{% block page_title %}Exercices{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Pédagogie</li><li class="breadcrumb-item active">Exercices</li>{% endblock %}

{% block content %}
<div class="page-head">
  <div><h2>Banque d'exercices</h2><p class="text-muted mb-0">{{ paginator.count }} exercice{{ paginator.count|pluralize }}</p>
</div>
  {% if perms.pedagogie.add_exercice %}<a href="{% url 'pedagogie:exercice_create' %}" class="btn btn-primary"><i class="bi bi-
plus-lg me-1"></i>Ajouter un exercice</a>{% endif %}
</div>

<form class="card-x card-pad mb-4" method="get">
  <div class="row g-2 align-items-end">
    <div class="col-lg-3"><label class="form-label">Recherche</label><input class="form-control" name="q" value="{{
request.GET.q }}" placeholder="Titre ou question…"></div>
    <div class="col-lg-3 col-md-6"><label class="form-label">Cours</label>
      <select class="form-select" name="cours"><option value="">Tous</option>{% for c in cours_all %}<option value="{{ c.pk }}"
{% if request.GET.cours == c.pk|stringformat:"s" %}selected{% endif %}>{{ c.titre }}</option>{% endfor %}</select></div>
    <div class="col-lg-2 col-md-3 col-6"><label class="form-label">Type</label>
      <select class="form-select" name="type_exercice"><option value="">Tous</option>{% for k, l in types %}<option value="{{ k
}}" {% if request.GET.type_exercice == k %}selected{% endif %}>{{ l }}</option>{% endfor %}</select></div>
    <div class="col-lg-2 col-md-3 col-6"><label class="form-label">Difficulté</label>
      <select class="form-select" name="difficulte"><option value="">Toutes</option>{% for k, l in difficultes %}<option value="
{{ k }}" {% if request.GET.difficulte == k %}selected{% endif %}>{{ l }}</option>{% endfor %}</select></div>
    <div class="col-lg-2 d-flex gap-2"><button class="btn btn-primary flex-fill"><i class="bi bi-funnel"></i> Filtrer</button><a
href="{% url 'pedagogie:exercice_list' %}" class="btn btn-soft" title="Réinitialiser"><i class="bi bi-x-lg"></i></a></div>
  </div>
</form>

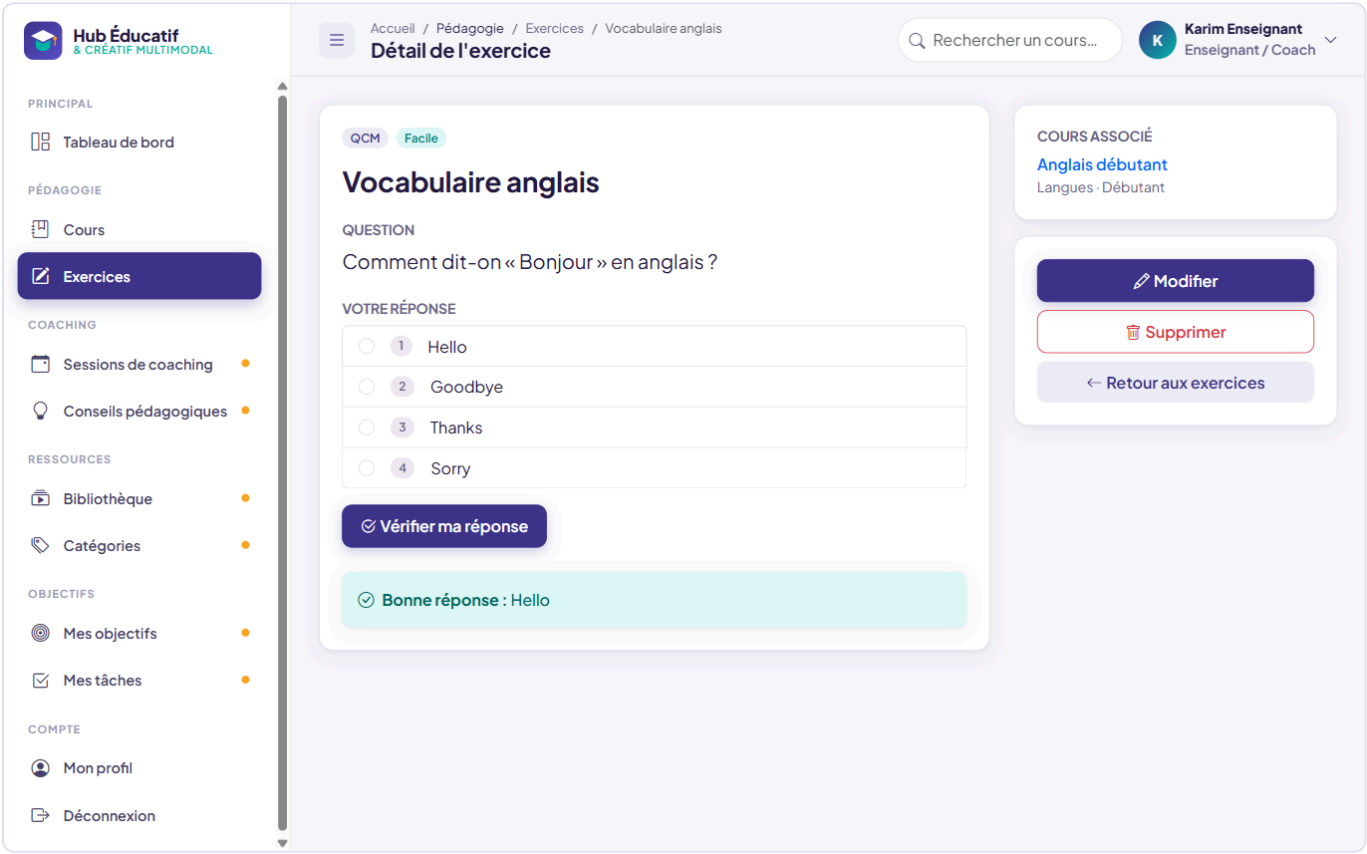
{% if page_obj %}
<div class="card-x overflow-hidden"><div class="table-responsive">
  <table class="table table-x align-middle">
    <thead><tr><th>Exercice</th><th>Cours</th><th>Type</th><th>Difficulté</th><th class="text-end">Actions</th></tr></thead>
    <tbody>
    {% for e in page_obj %}
      <tr>
        <td><a href="{{ e.get_absolute_url }}" class="fw-semibold text-reset">{{ e.titre }}</a><div class="small text-muted
clamp-2" style="max-width:380px">{{ e.question }}</div></td>
        <td><a href="{{ e.cours.get_absolute_url }}">{{ e.cours.titre }}</a></td>
        <td><span class="badge badge-soft badge-primary">{{ e.get_type_exercice_display }}</span></td>
        <td><span class="badge badge-soft {% if e.difficulte == 'facile' %}badge-teal{% elif e.difficulte == 'moyen' %}badge-
amber{% else %}badge-danger{% endif %}">{{ e.get_difficulte_display }}</span></td>
        <td class="text-end text-nowrap">
          <a href="{{ e.get_absolute_url }}" class="btn btn-soft btn-icon" title="Consulter"><i class="bi bi-eye"></i></a>
          {% if e|can_edit:user %}<a href="{% url 'pedagogie:exercice_update' e.pk %}" class="btn btn-outline-primary btn-icon"
title="Modifier"><i class="bi bi-pencil"></i></a>{% endif %}
          {% if e|can_delete:user %}<button class="btn btn-outline-danger btn-icon" title="Supprimer" data-bs-toggle="modal"
data-bs-target="#deleteModal" data-url="{% url 'pedagogie:exercice_delete' e.pk %}" data-name="{{ e.titre }}"><i class="bi bi-
trash"></i></button>{% endif %}
        </td>
      </tr>
```

```
    {% endfor %}
    </tbody>
  </table>
</div></div>
{% include "components/pagination.html" %}
{% else %}
  {% include "components/empty.html" with icon="bi-pencil-square" title="Aucun exercice trouvé" text="Modifiez vos filtres ou
créez un nouvel exercice." %}
{% endif %}
{% endblock %}
```

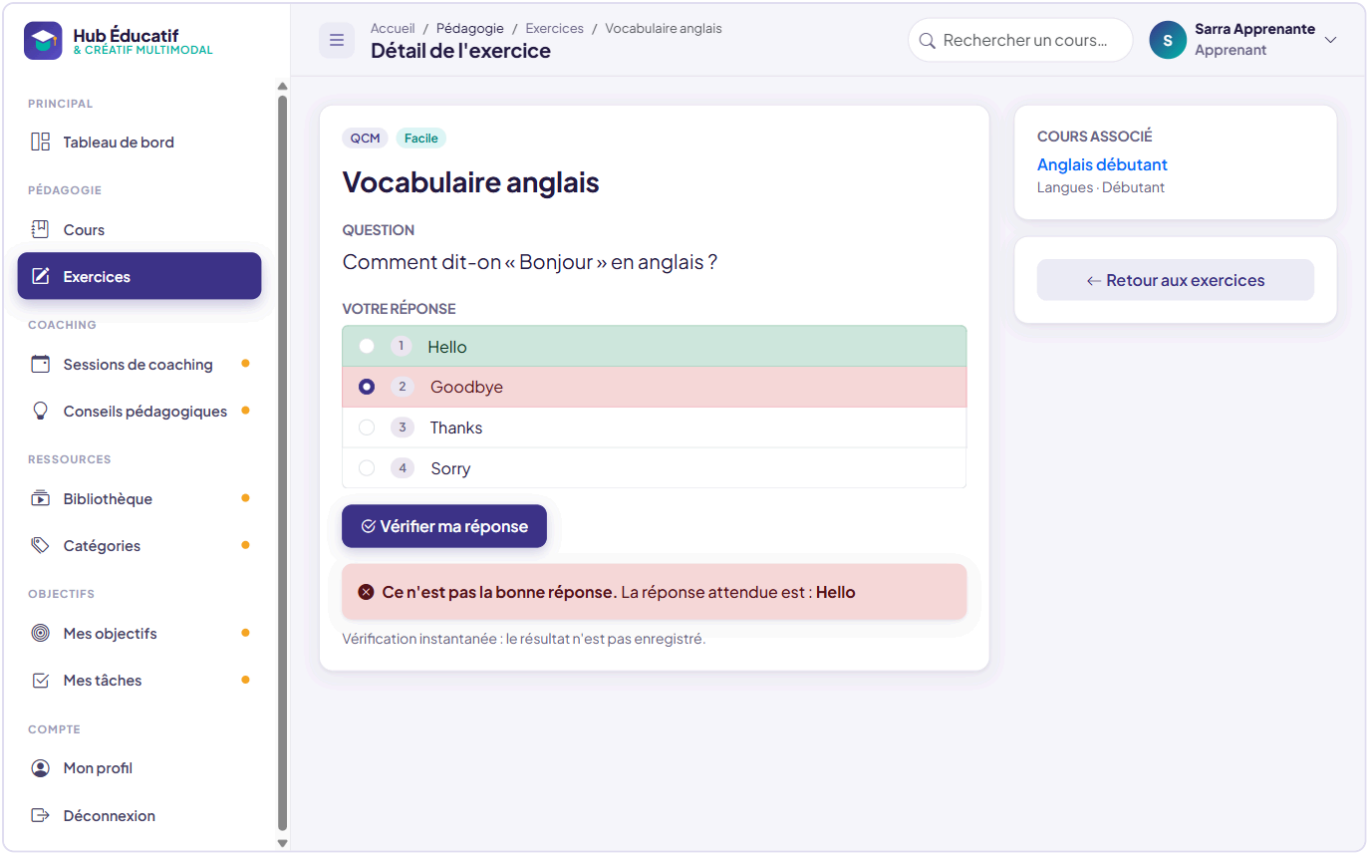
1. Filtres : cours, type (QCM/Vrai-Faux/Question ouverte), difficulté et recherche dans le titre ou la question.
2. Le tableau est dans un `.table-responsive` : il défile horizontalement sur mobile.
3. Les badges de difficulté changent de couleur selon la valeur (`{% if e.difficulte == 'facile' %}`…).
4. **Afficher les exercices associés à un cours** : soit le filtre « Cours » de cette liste, soit la liste en bas de la page détail du cours.

9.7 Détail d'un exercice et vérification de la réponse

Commande / adresse :
`http://127.0.0.1:8000/pedagogie/exercices/4/` (`pedagogie:exercice_detail`)



Détail d'un exercice vu par l'enseignant (bonne réponse visible)



Même exercice vu par l'apprenant après avoir répondu « Goodbye »

pedagogie/views.py — ExerciceDetailView

```
class ExerciceDetailView(PermissionRequiredMixin, DetailView):
    permission_required = 'pedagogie.view_exercice'
    model = Exercice
    queryset = Exercice.objects.select_related('cours__enseignant')
    template_name = 'pedagogie/exercice_detail.html'

    def post(self, request, *args, **kwargs):
        """Vérification simple (rien n'est enregistré) : compare la réponse à la bonne réponse."""
        self.object = self.get_object()
        donnee = request.POST.get('reponse', '').strip()
        ctx = self.get_context_data(object=self.object)
        if donnee:
            ctx.update(reponse_donnee=donnee, resultat_juste=donnee.lower() == self.object.bonne_reponse.strip().lower())
        else:
            ctx['erreur_reponse'] = 'Choisissez ou saisissez une réponse avant de vérifier.'
        return self.render_to_response(ctx)
```

templates/pedagogie/exercice_detail.html

```
{% extends "base.html" %}
{% load pedagogie_tags %}
{% block title %}{{ object.titre }}{% endblock %}
{% block page_title %}Détail de l'exercice{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Pédagogie</li><li class="breadcrumb-item"><a href="{% url
'pedagogie:exercice_list' %}">Exercices</a></li><li class="breadcrumb-item active">{{ object.titre|truncatechars:30 }}</li>{%
endblock %}

{% block content %}
<div class="row g-4">
  <div class="col-lg-8">
    <div class="card-x card-pad">
      <div class="d-flex flex-wrap gap-2 mb-3">
        <span class="badge badge-soft badge-primary">{{ object.get_type_exercice_display }}</span>
        <span class="badge badge-soft {% if object.difficulte == 'facile' %}badge-teal{% elif object.difficulte == 'moyen'
%}badge-amber{% else %}badge-danger{% endif %}">{{ object.get_difficulte_display }}</span>
      </div>
      <h3>{{ object.titre }}</h3>
      <h6 class="text-muted text-uppercase small mt-4">Question</h6>
      <p class="fs-5" style="white-space: pre-line">{{ object.question }}</p>
```

```
      <form method="post" class="mt-4">{% csrf_token %}
        <h6 class="text-muted text-uppercase small">Votre réponse</h6>
        {% if object.type_exercice == 'qcm' %}
          <div class="list-group">
          {% for p in object.liste_propositions %}
            <label class="list-group-item d-flex align-items-center gap-3 {% if reponse_donnee and p|lower ==
object.bonne_reponse|lower %}list-group-item-success{% elif reponse_donnee and p == reponse_donnee %}list-group-item-danger{%
endif %}" style="cursor:pointer">
              <input class="form-check-input mt-0" type="radio" name="reponse" value="{{ p }}" {% if p == reponse_donnee
%}checked{% endif %} required>
              <span class="badge badge-soft badge-gray">{{ forloop.counter }}</span>{{ p }}
            </label>
          {% endfor %}
          </div>
        {% elif object.type_exercice == 'vf' %}
          <div class="d-flex gap-2">
            <label class="chip"><input class="form-check-input me-2" type="radio" name="reponse" value="Vrai" {% if
reponse_donnee == 'Vrai' %}checked{% endif %} required>Vrai</label>
            <label class="chip"><input class="form-check-input me-2" type="radio" name="reponse" value="Faux" {% if
reponse_donnee == 'Faux' %}checked{% endif %} required>Faux</label>
          </div>
        {% else %}
          <textarea class="form-control" name="reponse" rows="3" placeholder="Saisissez votre réponse…" required>{{
reponse_donnee }}</textarea>
        {% endif %}
        <button class="btn btn-primary mt-3"><i class="bi bi-check2-circle me-1"></i>Vérifier ma réponse</button>
      </form>

      {% if erreur_reponse %}<div class="alert alert-warning mt-3 mb-0">{{ erreur_reponse }}</div>{% endif %}
      {% if reponse_donnee %}
        {% if resultat_juste %}
          <div class="alert alert-success mt-3 mb-0"><i class="bi bi-check-circle-fill me-2"></i><strong>Bonne réponse !
</strong> Bravo.</div>
        {% else %}
          <div class="alert alert-danger mt-3 mb-0"><i class="bi bi-x-circle-fill me-2"></i><strong>Ce n'est pas la bonne
réponse.</strong> La réponse attendue est : <strong>{{ object.bonne_reponse }}</strong></div>
        {% endif %}
        <p class="small text-muted mt-2 mb-0">Vérification instantanée : le résultat n'est pas enregistré.</p>
      {% endif %}

      {% if object|can_edit:user %}
      <div class="alert mt-4 mb-0" style="background:var(--teal-50);color:#006b62"><i class="bi bi-check-circle me-2"></i>
<strong>Bonne réponse :</strong> {{ object.bonne_reponse }}</div>
      {% endif %}
    </div>
  </div>
  <div class="col-lg-4">
    <div class="card-x card-pad mb-3">
      <h6 class="text-muted text-uppercase small">Cours associé</h6>
      <a href="{{ object.cours.get_absolute_url }}" class="fw-semibold">{{ object.cours.titre }}</a>
      <div class="small text-muted">{{ object.cours.matiere }} · {{ object.cours.get_niveau_display }}</div>
    </div>
    <div class="card-x card-pad d-grid gap-2">
      {% if object|can_edit:user %}<a href="{% url 'pedagogie:exercice_update' object.pk %}" class="btn btn-primary"><i
class="bi bi-pencil me-1"></i>Modifier</a>{% endif %}
      {% if object|can_delete:user %}<button class="btn btn-outline-danger" data-bs-toggle="modal" data-bs-target="#deleteModal"
data-url="{% url 'pedagogie:exercice_delete' object.pk %}" data-name="{{ object.titre }}"><i class="bi bi-trash me-1">
</i>Supprimer</button>{% endif %}
      <a href="{% url 'pedagogie:exercice_list' %}" class="btn btn-soft"><i class="bi bi-arrow-left me-1"></i>Retour aux
exercices</a>
    </div>
  </div>
</div>
{% endblock %}
```

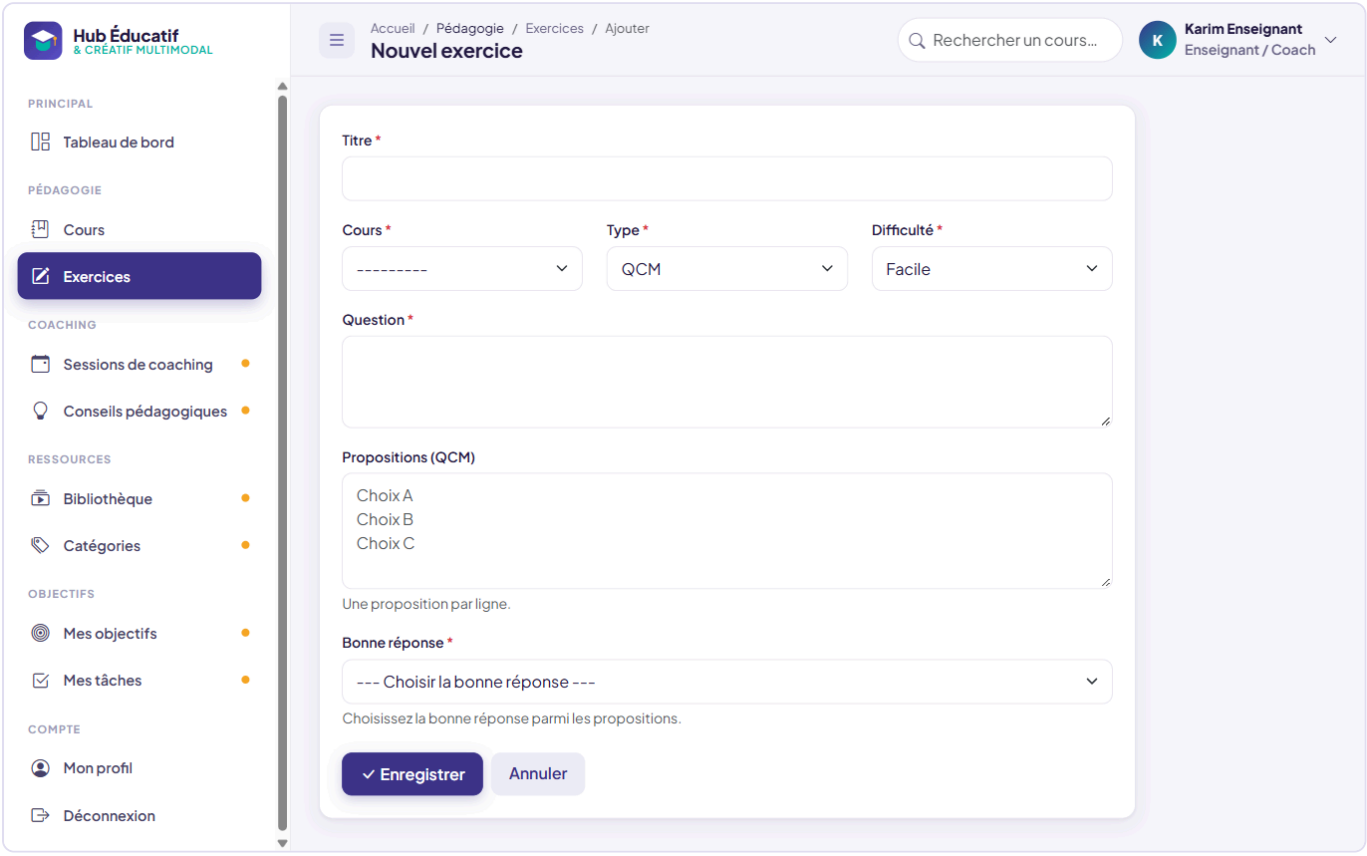
1. **GET** : affiche la question et les propositions sous forme de boutons radio (QCM), Vrai/Faux ou zone de texte (question ouverte).
2. **POST** (clic sur « Vérifier ma réponse ») : la méthode `post()` compare la réponse donnée à `bonne_reponse` (sans tenir compte des majuscules ni des espaces autour) et renvoie la même page avec le résultat : vert « Bonne réponse ! » ou rouge avec la réponse attendue.
3. La comparaison est faite **côté serveur** : la bonne réponse n'est jamais envoyée au navigateur d'un apprenant avant son essai. Seuls ceux qui peuvent modifier l'exercice voient l'encadré « Bonne réponse ».

4. **Rien n'est enregistré** : pas de note ni d'historique (la correction automatique avancée n'est pas dans cette version, comme demandé).

9.8 Ajouter / modifier un exercice (QCM avec liste déroulante)

Commande / adresse :

```
http://127.0.0.1:8000/pedagogie/exercices/ajouter/                  (pedagogie:exercice_create)
http://127.0.0.1:8000/pedagogie/exercices/ajouter/?cours=1        (cours pré-sélectionné)
http://127.0.0.1:8000/pedagogie/exercices/<id>/modifier/          (pedagogie:exercice_update)
```



Formulaire d'exercice

pedagogie/views.py — ExerciceFormMixin, ExerciceCreateView, ExerciceUpdateView, ExerciceDeleteView

```
class ExerciceFormMixin:
    form_class = ExerciceForm
    template_name = 'pedagogie/exercice_form.html'

    def get_form_kwargs(self):
        kw = super().get_form_kwargs()
        kw['user'] = self.request.user
        return kw
class ExerciceCreateView(PermissionRequiredMixin, ExerciceFormMixin, CreateView):
    permission_required = 'pedagogie.add_exercice'

    def get_initial(self):
        cid = self.request.GET.get('cours', '')
        return {'cours': cid} if cid.isdigit() else {}

    def form_valid(self, form):
        messages.success(self.request, 'Exercice créé.')
        return super().form_valid(form)
class ExerciceUpdateView(PermissionRequiredMixin, OwnerOrAdminMixin, ExerciceFormMixin, UpdateView):
    permission_required = 'pedagogie.change_exercice'
    model = Exercice

    def form_valid(self, form):
        messages.success(self.request, 'Exercice modifié.')
        return super().form_valid(form)
class ExerciceDeleteView(PermissionRequiredMixin, OwnerOrAdminMixin, DeleteMessageMixin, DeleteView):
```

```
    permission_required = 'pedagogie.delete_exercice'
    model = Exercice
    success_url = reverse_lazy('pedagogie:exercice_list')
```

templates/pedagogie/exercice_form.html

```
{% extends "base.html" %}
{% block title %}{% if object %}Modifier l'exercice{% else %}Nouvel exercice{% endif %}{% endblock %}
{% block page_title %}{% if object %}Modifier l'exercice{% else %}Nouvel exercice{% endif %}{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Pédagogie</li><li class="breadcrumb-item"><a href="{% url
'pedagogie:exercice_list' %}">Exercices</a></li><li class="breadcrumb-item active">{% if object %}Modifier{% else %}Ajouter{%
endif %}</li>{% endblock %}

{% block content %}
{% if not form.fields.cours.queryset.exists %}
  {% include "components/empty.html" with icon="bi-journal-plus" title="Aucun cours disponible" text="Créez d'abord un cours
pour pouvoir lui associer des exercices." %}
{% else %}
<form method="post" class="card-x card-pad" style="max-width: 820px" novalidate>{% csrf_token %}
  {% if form.non_field_errors %}<div class="alert alert-danger">{{ form.non_field_errors }}</div>{% endif %}
  {% include "components/field.html" with field=form.titre %}
  <div class="row">
    <div class="col-md-4">{% include "components/field.html" with field=form.cours %}</div>
    <div class="col-md-4">{% include "components/field.html" with field=form.type_exercice %}</div>
    <div class="col-md-4">{% include "components/field.html" with field=form.difficulte %}</div>
  </div>
  {% include "components/field.html" with field=form.question %}
  {% include "components/field.html" with field=form.propositions %}
  {% include "components/field.html" with field=form.bonne_reponse hint="Recopiez exactement la bonne proposition." %}
  <div class="d-flex gap-2 pt-2"><button class="btn btn-primary"><i class="bi bi-check-lg me-1"></i>Enregistrer</button><a
href="{% if object %}{{ object.get_absolute_url }}{% else %}{% url 'pedagogie:exercice_list' %}{% endif %}" class="btn btn-
soft">Annuler</a></div>
</form>
{% endif %}
{% endblock %}
```

JavaScript du formulaire (version complète en section 12)

```
// static/js/hub.js — extrait : liste déroulante de la bonne réponse
const type = document.getElementById('id_type_exercice');
if (type) {
  const props = document.getElementById('id_propositions').closest('.field');
  const hint = document.getElementById('answer-hint');
  const area = document.getElementById('id_bonne_reponse');
  const propsArea = document.getElementById('id_propositions');
  const sel = document.createElement('select');      // crée la liste déroulante
  sel.className = 'form-select';
  area.after(sel);
  sel.addEventListener('change', () => { area.value = sel.value; });   // recopie le choix dans le vrai champ
  const fill = (choices) => { /* remplit la liste avec les choix */ };
  const sync = () => {
    props.style.display = type.value === 'qcm' ? '' : 'none';   // propositions visibles seulement en QCM
    area.style.display = type.value === 'ouverte' ? '' : 'none';
    sel.style.display  = type.value === 'ouverte' ? 'none' : '';
    if (type.value === 'vf')  fill(['Vrai', 'Faux']);
    if (type.value === 'qcm') fill(propsArea.value.split('\n').map(p => p.trim()).filter(Boolean));
  };
  type.addEventListener('change', sync);
  propsArea.addEventListener('input', sync);   // la liste se met à jour quand on tape les propositions
  sync();
}
```

1. **Saisie des propositions d'un QCM** : zone de texte, **une proposition par ligne**. Le champ « Bonne réponse » devient une **liste déroulante** qui reprend automatiquement ces propositions ; pour Vrai/Faux elle propose « Vrai » et « Faux » ; pour une question ouverte, un champ texte est conservé.
2. Le JavaScript **recopie** le choix de la liste dans le vrai champ `bonne_reponse` (envoyé au serveur) : le serveur valide ensuite avec `Exercice.clean()` (réponse présente dans les propositions…).
3. `ExerciceFormMixin.get_form_kwargs()` passe l'utilisateur au formulaire pour limiter les cours.
4. `ExerciceCreateView.get_initial()` lit `?cours=…` pour pré-sélectionner le cours.

5. Si aucun cours n'existe, la page affiche « Aucun cours disponible » (`{% if not form.fields.cours.queryset.exists %}`).

10. Pages statiques : coaching, ressources, objectifs

Principe : ces modules ne contiennent **aucun modèle, aucune migration, aucun formulaire, aucun CRUD**. Chaque app a : `demo.py` (données fictives sous forme de listes Python), `views.py` (une vue qui envoie ces données au gabarit), `urls.py`. Quand le collègue développera son module, il remplacera `demo.py` par des requêtes sur ses modèles.

Composants communs aux pages de démonstration

`templates/components/demo_banner.html`

```
<div class="demo-banner">
  <span class="badge-demo">Démonstration</span>
  <span>Page statique : les données affichées sont des <strong>exemples fictifs</strong>. Ce module sera développé par un autre
membre de l'équipe.</span>
</div>
```

`templates/components/soon_button.html`

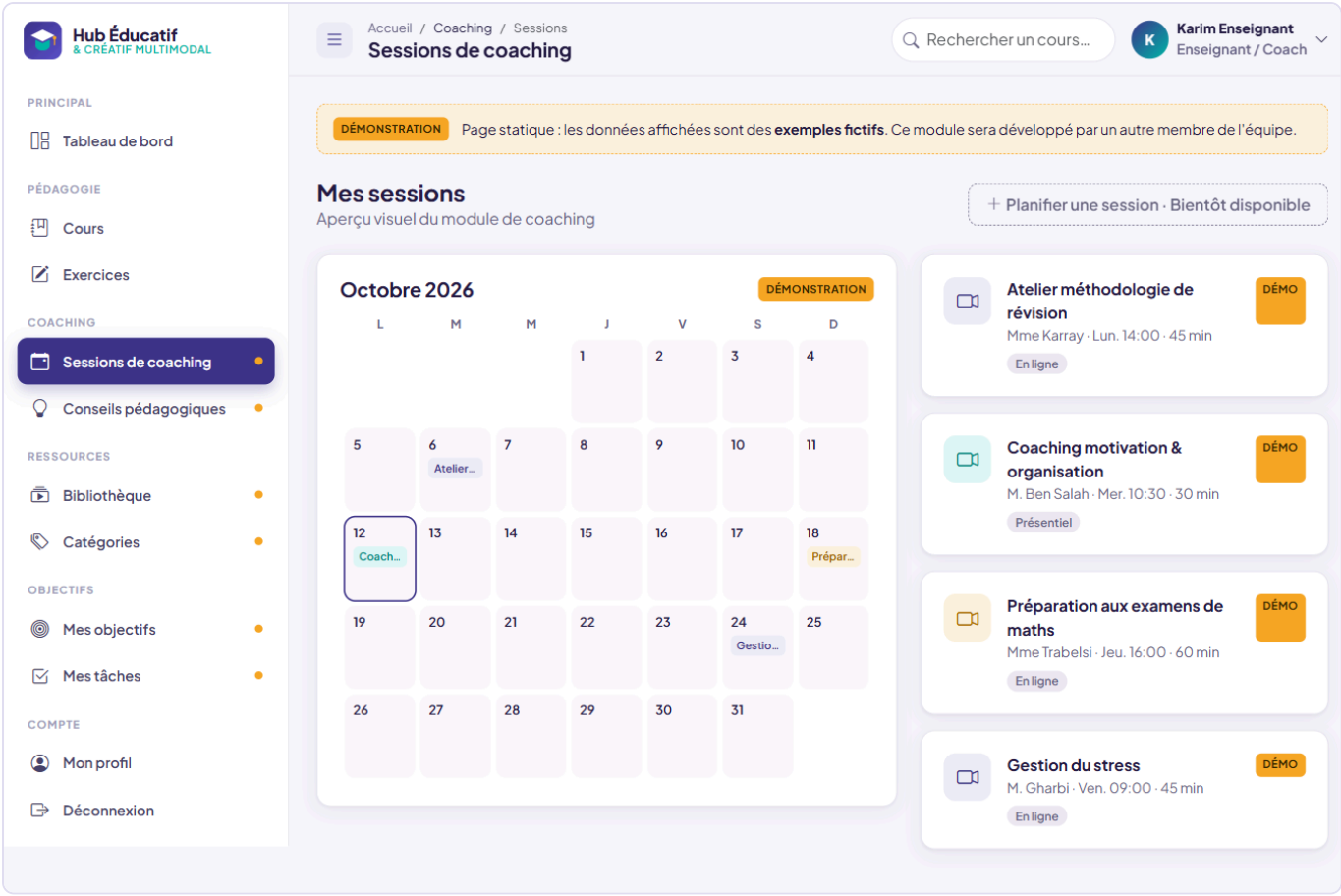
```
{# Bouton désactivé pour les fonctionnalités des autres modules. Usage : {% include "components/soon_button.html" with
label="Ajouter" icon="bi-plus-lg" %} #}
<span class="d-inline-block" title="Bientôt disponible"><button type="button" class="btn btn-soon" disabled><i class="bi {{
icon|default:'bi-plus-lg' }} me-1"></i>{{ label }} · Bientôt disponible</button></span>
```

`demo_banner.html` : bandeau jaune « Démonstration » sur chaque page statique. `soon_button.html` : bouton **désactivé** « … · Bientôt disponible » à la place des boutons Ajouter/Modifier (aucune fausse sauvegarde).

10.1 Sessions de coaching (calendrier visuel)

Commande / adresse :

`http://127.0.0.1:8000/coaching/sessions/  (coaching:sessions)`



Sessions de coaching : calendrier et cartes de séances (démonstration)

coaching/demo.py

```
"""Données de DÉMONSTRATION (statiques). Aucune base de données : à remplacer par les modèles du module."""

SESSIONS = [
    {'titre': 'Atelier méthodologie de révision', 'coach': 'Mme Karray', 'date': 'Lun. 14:00', 'duree': '45 min', 'mode': 'En ligne', 'jour': 6, 'couleur': 'primary'},
    {'titre': 'Coaching motivation & organisation', 'coach': 'M. Ben Salah', 'date': 'Mer. 10:30', 'duree': '30 min', 'mode': 'Présentiel', 'jour': 12, 'couleur': 'teal'},
    {'titre': 'Préparation aux examens de maths', 'coach': 'Mme Trabelsi', 'date': 'Jeu. 16:00', 'duree': '60 min', 'mode': 'En ligne', 'jour': 18, 'couleur': 'amber'},
    {'titre': 'Gestion du stress', 'coach': 'M. Gharbi', 'date': 'Ven. 09:00', 'duree': '45 min', 'mode': 'En ligne', 'jour': 24, 'couleur': 'primary'},
]

CONSEILS = [
    {'icone': 'bi-clock-history', 'titre': 'Étudier par blocs de 25 minutes', 'texte': 'La technique Pomodoro limite la fatigue et améliore la concentration.', 'categorie': 'Organisation'},
    {'icone': 'bi-arrow-repeat', 'titre': 'Réviser à intervalles espacés', 'texte': 'Revoir une notion à J+1, J+3 et J+7 ancre durablement les connaissances.', 'categorie': 'Mémorisation'},
    {'icone': 'bi-pencil-square', 'titre': 'Pratiquer avant de relire', 'texte': 'Résoudre des exercices est plus efficace que relire passivement un cours.', 'categorie': 'Méthode'},
    {'icone': 'bi-moon-stars', 'titre': 'Soigner son sommeil', 'texte': 'Dormir 7 à 8 heures consolide ce qui a été appris dans la journée.', 'categorie': 'Bien-être'},
    {'icone': 'bi-people', 'titre': 'Expliquer pour mieux comprendre', 'texte': 'Reformuler une notion à un camarade révèle les zones d\'ombre.', 'categorie': 'Méthode'},
    {'icone': 'bi-bullseye', 'titre': 'Se fixer un objectif par séance', 'texte': 'Un objectif précis et mesurable donne un cap à chaque session de travail.', 'categorie': 'Organisation'},
]
```

coaching/views.py

```
import calendar
from datetime import date

from django.views.generic import TemplateView

from . import demo
```

```
MOIS = ['', 'Janvier', 'Février', 'Mars', 'Avril', 'Mai', 'Juin', 'Juillet', 'Août', 'Septembre', 'Octobre', 'Novembre',
'Décembre']


class SessionsView(TemplateView):
    template_name = 'coaching/sessions.html'

    def get_context_data(self, **kw):
        ctx = super().get_context_data(**kw)
        today = date.today()
        events = {s['jour']: s for s in demo.SESSIONS}
        ctx['sessions'] = demo.SESSIONS
        ctx['mois'] = f'{MOIS[today.month]} {today.year}'
        ctx['today'] = today.day
        # semaines -> jours {n, event} ; n=0 = case vide
        ctx['weeks'] = [[{'n': d, 'event': events.get(d)} for d in week]
                        for week in calendar.Calendar().monthdayscalendar(today.year, today.month)]
        return ctx


class ConseilsView(TemplateView):
    template_name = 'coaching/conseils.html'
    extra_context = {'conseils': demo.CONSEILS}
```

coaching/urls.py

```
from django.urls import path

from . import views

app_name = 'coaching'
urlpatterns = [
    path('sessions/', views.SessionsView.as_view(), name='sessions'),
    path('conseils/', views.ConseilsView.as_view(), name='conseils'),
]
```

templates/coaching/sessions.html

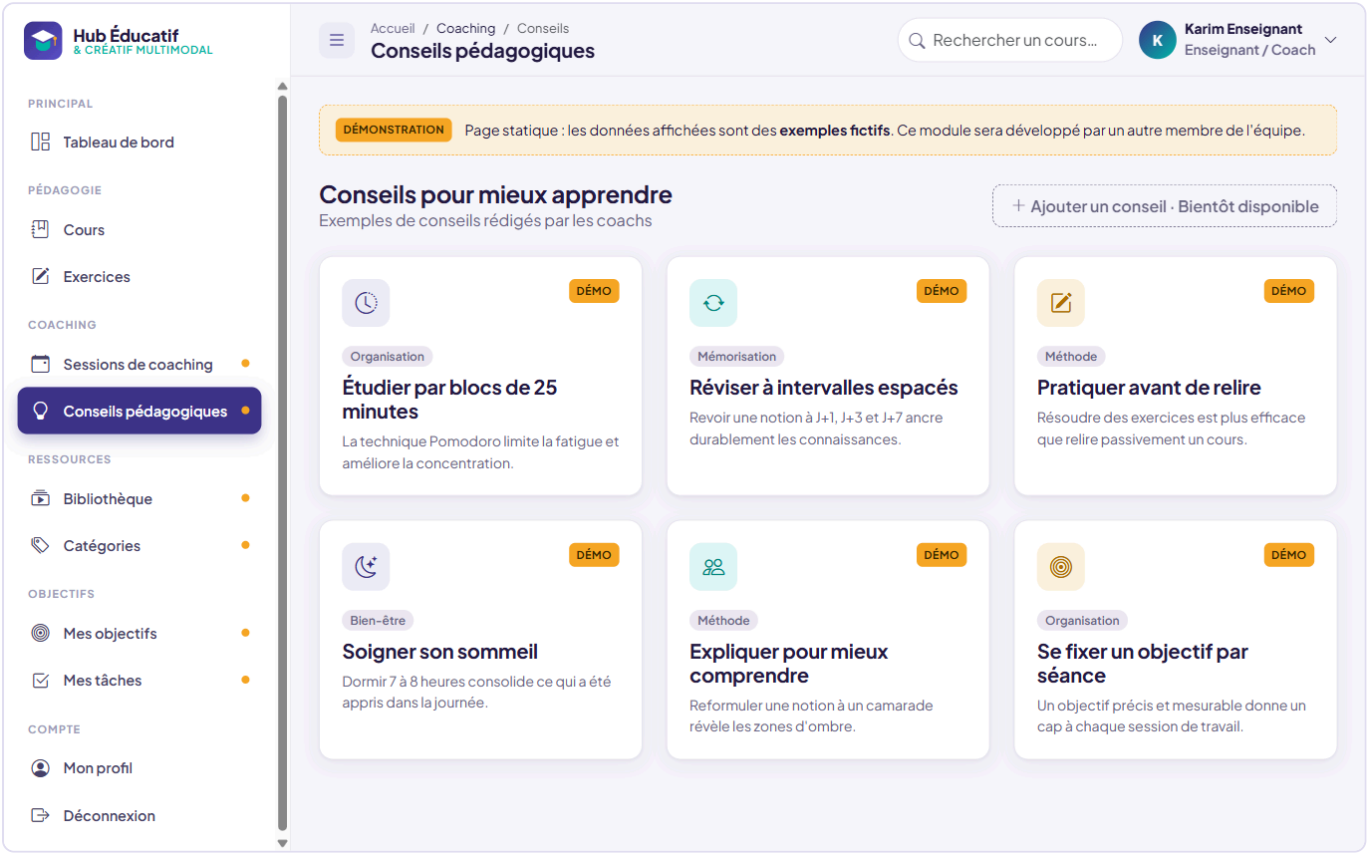
```
{% extends "base.html" %}
{% block title %}Sessions de coaching{% endblock %}
{% block page_title %}Sessions de coaching{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Coaching</li><li class="breadcrumb-item active">Sessions</li>{% endblock %}
{% block content %}
{% include "components/demo_banner.html" %}
<div class="page-head"><div><h2>Mes sessions</h2><p class="text-muted mb-0">Aperçu visuel du module de coaching</p></div>{%
include "components/soon_button.html" with label="Planifier une session" %}</div>
<div class="row g-4">
  <div class="col-xl-7"><div class="card-x card-pad">
    <div class="d-flex justify-content-between mb-2"><h5 class="mb-0">{{ mois }}</h5><span class="badge-
demo">Démonstration</span></div>
    <table class="cal"><thead><tr>{% for j in "LMMJVSD" %}<th>{{ j }}</th>{% endfor %}</tr></thead><tbody>
    {% for week in weeks %}<tr>{% for d in week %}
      <td class="{% if not d.n %}empty-day{% elif d.n == today %}today{% endif %}">{% if d.n %}{{ d.n }}{% if d.event %}<span
class="ev {{ d.event.couleur }}">{{ d.event.titre }}</span>{% endif %}{% endif %}</td>
    {% endfor %}</tr>{% endfor %}
    </tbody></table>
  </div></div>
  <div class="col-xl-5">
    {% for s in sessions %}
    <div class="card-x hover card-pad mb-3 d-flex gap-3">
      <div class="icon-box tone-{{ s.couleur }}"><i class="bi bi-camera-video"></i></div>
      <div class="flex-fill"><div class="d-flex justify-content-between"><strong>{{ s.titre }}</strong><span class="badge-
demo">Démo</span></div>
        <div class="small text-muted">{{ s.coach }} · {{ s.date }} · {{ s.duree }}</div>
        <span class="badge badge-soft badge-gray mt-2">{{ s.mode }}</span></div>
    </div>
    {% endfor %}
  </div>
</div>
{% endblock %}
```

1. `demo.SESSIONS` : liste de dictionnaires (titre, coach, date, durée, mode, jour du mois, couleur). Aucune base de données.

2. `SessionsView.get_context_data` construit le calendrier du **mois courant** avec le module Python `calendar` : `monthdayscalendar()` renvoie les semaines (0 = case vide). On associe à chaque jour la séance éventuelle grâce au dictionnaire `events`.
3. Le gabarit parcourt `weeks` puis les jours : `{% for week in weeks %}{% for d in week %}`. Le jour d'aujourd'hui reçoit la classe `today` (contour indigo).
4. À droite : une carte par séance, avec badge « Démo ».

10.2 Conseils pédagogiques

Commande / adresse :
`http://127.0.0.1:8000/coaching/conseils/` (`coaching:conseils`)



Conseils pédagogiques (démonstration)

templates/coaching/conseils.html

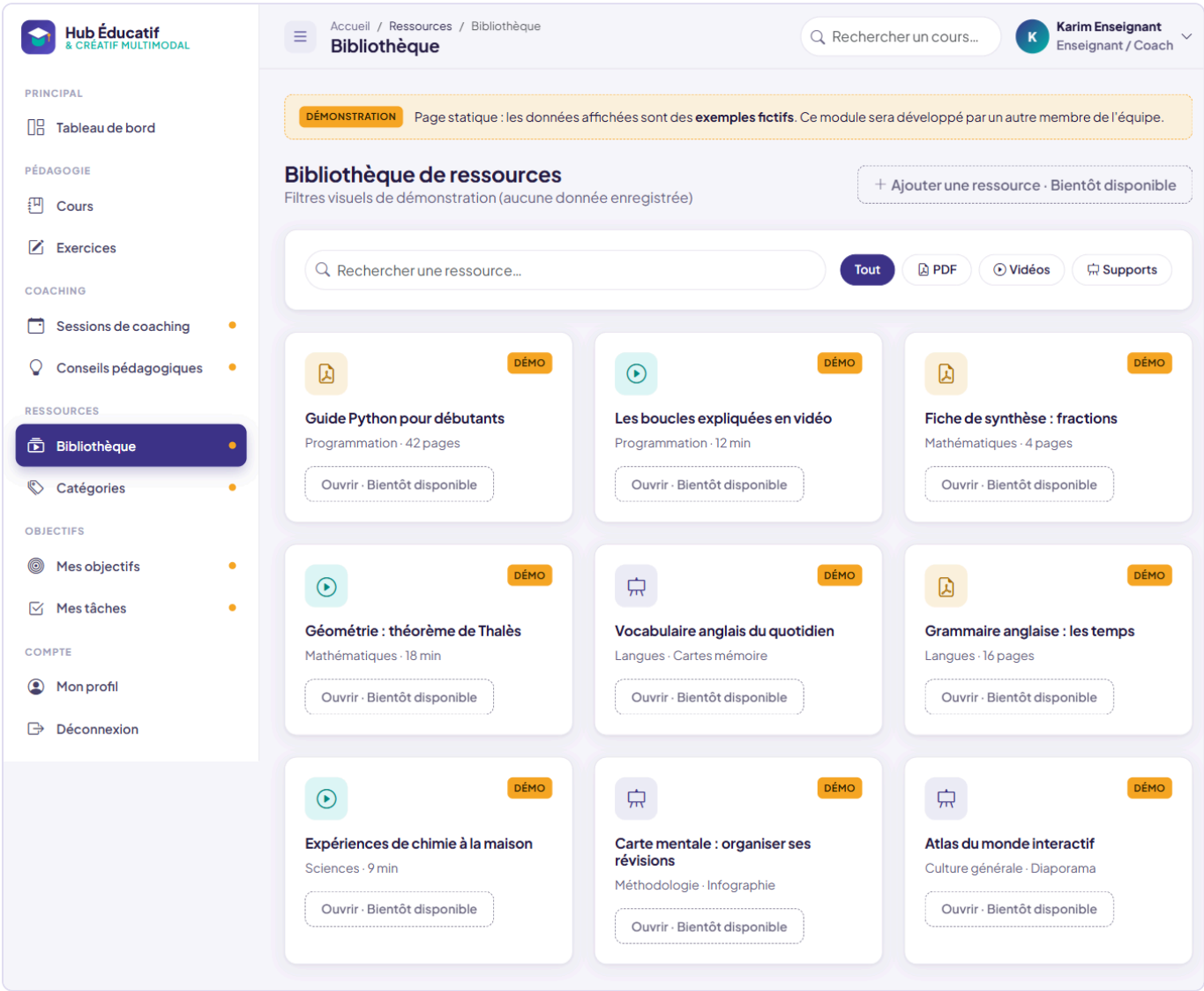
```
{% extends "base.html" %}
{% block title %}Conseils pédagogiques{% endblock %}
{% block page_title %}Conseils pédagogiques{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Coaching</li><li class="breadcrumb-item active">Conseils</li>{% endblock %}
{% block content %}
{% include "components/demo_banner.html" %}
<div class="page-head"><div><h2>Conseils pour mieux apprendre</h2><p class="text-muted mb-0">Exemples de conseils rédigés par
les coachs</p></div>{% include "components/soon_button.html" with label="Ajouter un conseil" %}</div>
<div class="row g-4">
  {% for c in conseils %}
  <div class="col-md-6 col-xl-4"><div class="card-x hover card-pad h-100">
    <div class="d-flex justify-content-between mb-3"><div class="icon-box tone-{% cycle 'primary' 'teal' 'amber' %}"><i
class="bi {{ c.icone }}"></i></div><span class="badge-demo align-self-start">Démo</span></div>
    <span class="badge badge-soft badge-gray mb-2">{{ c.categorie }}</span>
    <h5>{{ c.titre }}</h5><p class="text-muted small mb-0">{{ c.texte }}</p>
  </div></div>
  {% endfor %}
</div>
{% endblock %}
```

La vue `ConseilsView` (voir `coaching/views.py` ci-dessus) est une `TemplateView` avec `extra_context = {'conseils': demo.CONSEILS}`. Le gabarit crée une carte par conseil ; `{% cycle 'primary' 'teal' 'amber' %}` alterne les couleurs d'icône.

10.3 Bibliothèque de ressources (recherche visuelle)

Commande / adresse :

`http://127.0.0.1:8000/ressources/bibliotheque/` `(ressources:bibliotheque)`



Bibliothèque : cartes PDF, vidéos et supports (démonstration)

ressources/demo.py

```
"""Données de DÉMONSTRATION (statiques). Aucune base de données : à remplacer par les modèles du module."""

CATEGORIES = [
    {'nom': 'Programmation', 'icone': 'bi-code-slash', 'nb': 24, 'couleur': 'primary'},
    {'nom': 'Mathématiques', 'icone': 'bi-calculator', 'nb': 18, 'couleur': 'teal'},
    {'nom': 'Langues', 'icone': 'bi-translate', 'nb': 15, 'couleur': 'amber'},
    {'nom': 'Sciences', 'icone': 'bi-flask', 'nb': 11, 'couleur': 'primary'},
    {'nom': 'Méthodologie', 'icone': 'bi-journal-check', 'nb': 9, 'couleur': 'teal'},
    {'nom': 'Culture générale', 'icone': 'bi-globe2', 'nb': 7, 'couleur': 'amber'},
]

RESSOURCES = [
    {'titre': 'Guide Python pour débutants', 'type': 'pdf', 'categorie': 'Programmation', 'info': '42 pages'},
    {'titre': 'Les boucles expliquées en vidéo', 'type': 'video', 'categorie': 'Programmation', 'info': '12 min'},
    {'titre': 'Fiche de synthèse : fractions', 'type': 'pdf', 'categorie': 'Mathématiques', 'info': '4 pages'},
    {'titre': 'Géométrie : théorème de Thalès', 'type': 'video', 'categorie': 'Mathématiques', 'info': '18 min'},
```

```
    {'titre': 'Vocabulaire anglais du quotidien', 'type': 'support', 'categorie': 'Langues', 'info': 'Cartes mémoire'},
    {'titre': 'Grammaire anglaise : les temps', 'type': 'pdf', 'categorie': 'Langues', 'info': '16 pages'},
    {'titre': 'Expériences de chimie à la maison', 'type': 'video', 'categorie': 'Sciences', 'info': '9 min'},
    {'titre': 'Carte mentale : organiser ses révisions', 'type': 'support', 'categorie': 'Méthodologie', 'info': 'Infographie'},
    {'titre': 'Atlas du monde interactif', 'type': 'support', 'categorie': 'Culture générale', 'info': 'Diaporama'},
]
```

ressources/views.py

```
from django.views.generic import TemplateView

from . import demo


class BibliothequeView(TemplateView):
    template_name = 'ressources/bibliotheque.html'
    extra_context = {'ressources': demo.RESSOURCES}


class CategoriesView(TemplateView):
    template_name = 'ressources/categories.html'
    extra_context = {'categories': demo.CATEGORIES}
```

ressources/urls.py

```
from django.urls import path

from . import views

app_name = 'ressources'
urlpatterns = [
    path('bibliotheque/', views.BibliothequeView.as_view(), name='bibliotheque'),
    path('categories/', views.CategoriesView.as_view(), name='categories'),
]
```

templates/ressources/bibliotheque.html

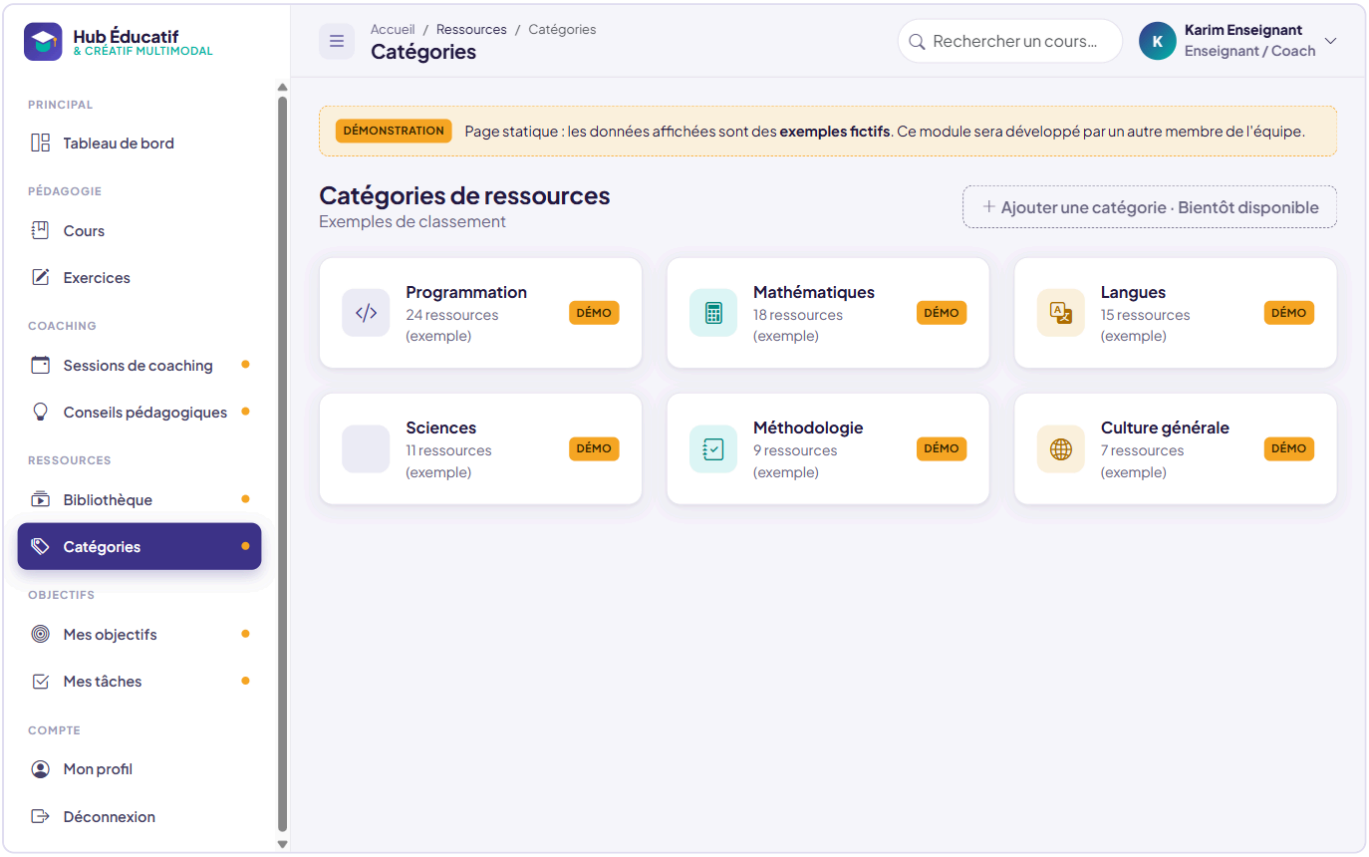
```
{% extends "base.html" %}
{% block title %}Bibliothèque{% endblock %}
{% block page_title %}Bibliothèque{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Ressources</li><li class="breadcrumb-item active">Bibliothèque</li>{% endblock %}
{% block content %}
{% include "components/demo_banner.html" %}
<div class="page-head"><div><h2>Bibliothèque de ressources</h2><p class="text-muted mb-0">Filtres visuels de démonstration
(aucune donnée enregistrée)</p></div>{% include "components/soon_button.html" with label="Ajouter une ressource" %}</div>
<div class="card-x card-pad mb-4 d-flex flex-wrap gap-3 align-items-center">
  <div class="topbar-search flex-fill" style="max-width:none"><i class="bi bi-search"></i><input class="form-control" data-demo-
search placeholder="Rechercher une ressource…"></div>
  <div class="d-flex gap-2 flex-wrap">
    <button class="chip active" data-filter="all">Tout</button><button class="chip" data-filter="pdf"><i class="bi bi-file-
earmark-pdf me-1"></i>PDF</button>
    <button class="chip" data-filter="video"><i class="bi bi-play-circle me-1"></i>Vidéos</button><button class="chip" data-
filter="support"><i class="bi bi-easel me-1"></i>Supports</button>
  </div>
</div>
<div class="row g-4">
  {% for r in ressources %}
  <div class="col-md-6 col-xl-4" data-kind="{{ r.type }}"><div class="card-x hover card-pad h-100">
    <div class="d-flex justify-content-between mb-3">
      <div class="icon-box {% if r.type == 'pdf' %}tone-amber{% elif r.type == 'video' %}tone-teal{% else %}tone-primary{% endif
%}"><i class="bi {% if r.type == 'pdf' %}bi-file-earmark-pdf{% elif r.type == 'video' %}bi-play-circle{% else %}bi-easel{% endif
%}"></i></div>
      <span class="badge-demo align-self-start">Démo</span>
    </div>
    <h6>{{ r.titre }}</h6><div class="small text-muted mb-3">{{ r.categorie }} · {{ r.info }}</div>
    <button class="btn btn-soon btn-sm" disabled>Ouvrir · Bientôt disponible</button>
  </div></div>
  {% endfor %}
</div>
{% endblock %}
```

1. Chaque carte porte `data-kind="pdf|video|support"` ; les boutons de filtre ont `data-filter` et le champ de recherche `data-demo-search`.

2. Le JavaScript de `hub.js` (section « Filtres de démonstration ») ajoute ou retire la classe Bootstrap `d-none` pour cacher les cartes qui ne correspondent pas : c'est **purement visuel**, rien n'est enregistré.
3. Les boutons « Ouvrir » sont désactivés (« Bientôt disponible »).

10.4 Catégories

Commande / adresse :
`http://127.0.0.1:8000/ressources/categories/` `(ressources:categories)`



Catégories de ressources (démonstration)

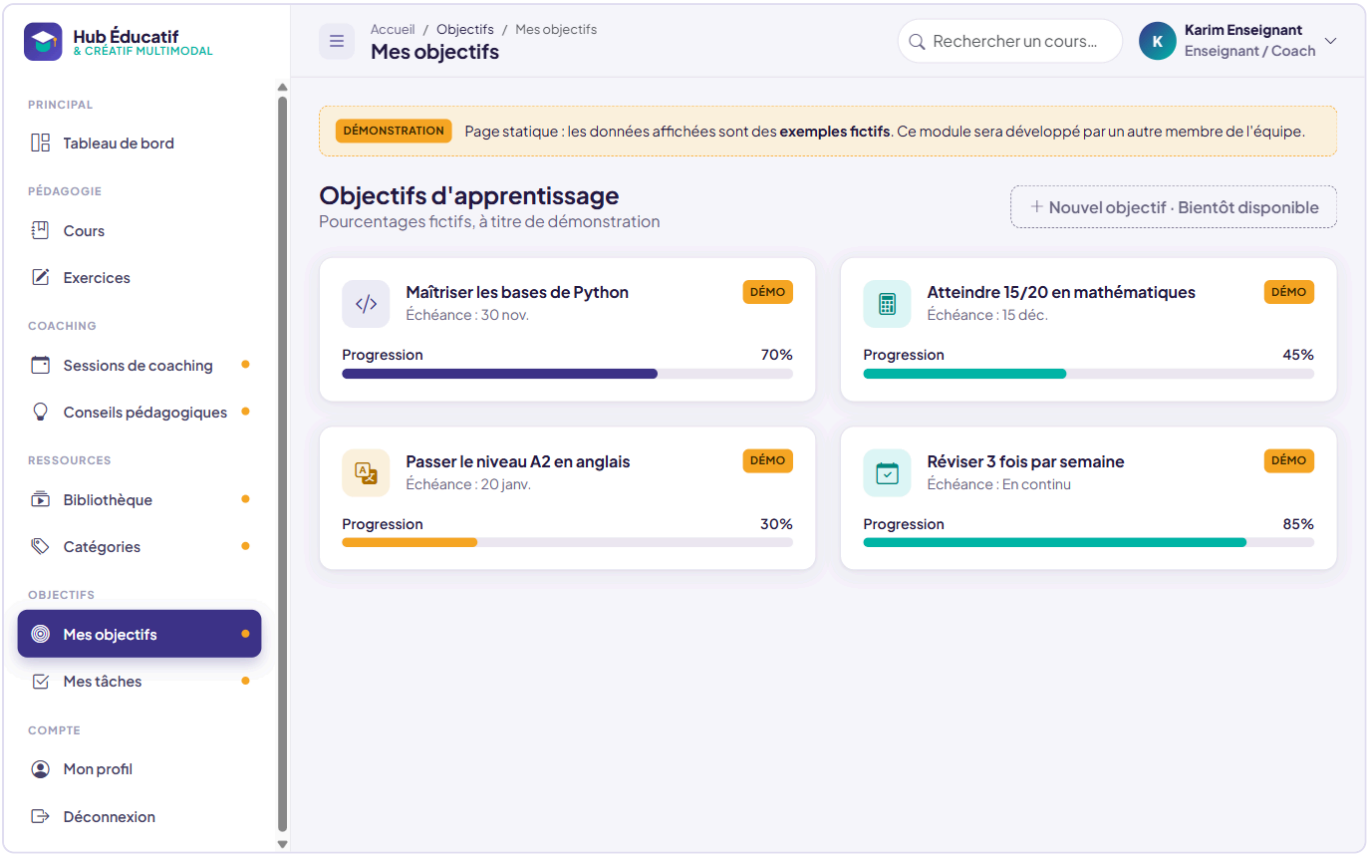
templates/ressources/categories.html

```
{% extends "base.html" %}
{% block title %}Catégories{% endblock %}
{% block page_title %}Catégories{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Ressources</li><li class="breadcrumb-item active">Catégories</li>{% endblock %}
{% block content %}
{% include "components/demo_banner.html" %}
<div class="page-head"><div><h2>Catégories de ressources</h2><p class="text-muted mb-0">Exemples de classement</p></div>{%
include "components/soon_button.html" with label="Ajouter une catégorie" %}</div>
<div class="row g-4">
  {% for c in categories %}
  <div class="col-md-6 col-xl-4"><div class="card-x hover card-pad h-100 d-flex align-items-center gap-3">
    <div class="icon-box tone-{{ c.couleur }}"><i class="bi {{ c.icone }}"></i></div>
    <div class="flex-fill"><strong>{{ c.nom }}</strong><div class="small text-muted">{{ c.nb }} ressources (exemple)</div></div>
    <span class="badge-demo">Démo</span>
  </div></div>
  {% endfor %}
</div>
{% endblock %}
```

10.5 Mes objectifs (barres de progression)

Commande / adresse :

`http://127.0.0.1:8000/objectifs/mes-objectifs/` `(objectifs:objectifs)`



Objectifs avec barres de progression (pourcentages fictifs)

objectifs/demo.py

```
"""Données de DÉMONSTRATION (statiques). Aucune base de données : à remplacer par les modèles du module."""

OBJECTIFS = [
    {'titre': 'Maîtriser les bases de Python', 'echeance': '30 nov.', 'progression': 70, 'couleur': 'primary', 'icone': 'bi-
code-slash'},
    {'titre': 'Atteindre 15/20 en mathématiques', 'echeance': '15 déc.', 'progression': 45, 'couleur': 'teal', 'icone': 'bi-
calculator'},
    {'titre': 'Passer le niveau A2 en anglais', 'echeance': '20 janv.', 'progression': 30, 'couleur': 'amber', 'icone': 'bi-
translate'},
    {'titre': 'Réviser 3 fois par semaine', 'echeance': 'En continu', 'progression': 85, 'couleur': 'teal', 'icone': 'bi-
calendar-check'},
]

TACHES = [
    {'objectif': 'Maîtriser les bases de Python', 'taches': [
        {'titre': 'Terminer le chapitre sur les variables', 'fait': True},
        {'titre': 'Résoudre 10 exercices de boucles', 'fait': True},
        {'titre': 'Écrire un mini-programme de calcul', 'fait': False},
    ]},
    {'objectif': 'Atteindre 15/20 en mathématiques', 'taches': [
        {'titre': 'Revoir les fractions', 'fait': True},
        {'titre': 'Faire le devoir blanc', 'fait': False},
        {'titre': 'Corriger les erreurs fréquentes', 'fait': False},
    ]},
    {'objectif': 'Passer le niveau A2 en anglais', 'taches': [
        {'titre': 'Apprendre 20 mots de vocabulaire', 'fait': False},
        {'titre': 'Écouter un podcast de 10 minutes', 'fait': False},
    ]},
]
```

objectifs/views.py

```
from django.views.generic import TemplateView

from . import demo


class ObjectifsView(TemplateView):
    template_name = 'objectifs/objectifs.html'
    extra_context = {'objectifs': demo.OBJECTIFS}


class TachesView(TemplateView):
    template_name = 'objectifs/taches.html'
    extra_context = {'groupes': demo.TACHES}
```

objectifs/urls.py

```
from django.urls import path

from . import views

app_name = 'objectifs'
urlpatterns = [
    path('mes-objectifs/', views.ObjectifsView.as_view(), name='objectifs'),
    path('mes-taches/', views.TachesView.as_view(), name='taches'),
]
```

templates/objectifs/objectifs.html

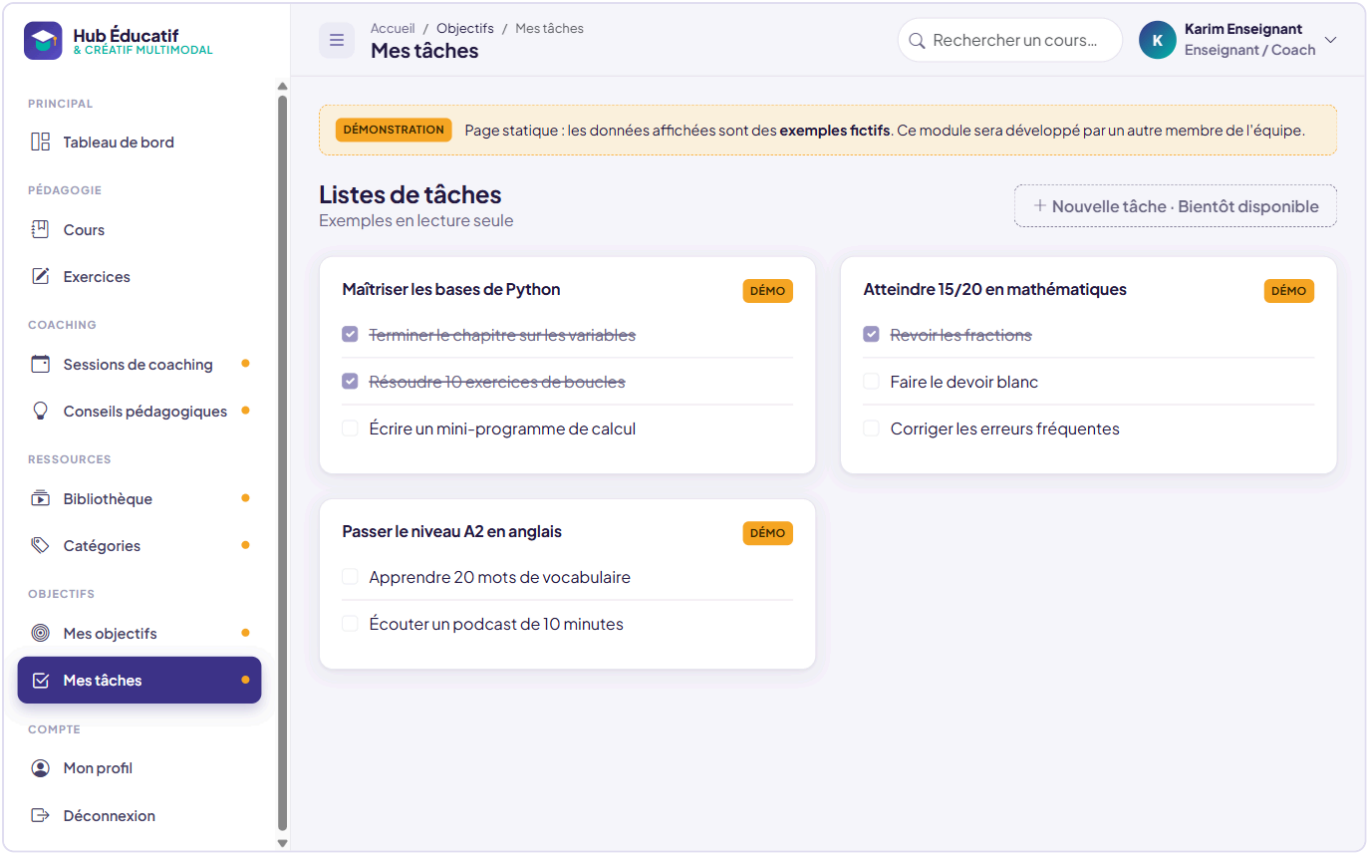
```
{% extends "base.html" %}
{% block title %}Mes objectifs{% endblock %}
{% block page_title %}Mes objectifs{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Objectifs</li><li class="breadcrumb-item active">Mes objectifs</li>{% endblock %}
{% block content %}
{% include "components/demo_banner.html" %}
<div class="page-head"><div><h2>Objectifs d'apprentissage</h2><p class="text-muted mb-0">Pourcentages fictifs, à titre de
démonstration</p></div>{% include "components/soon_button.html" with label="Nouvel objectif" %}</div>
<div class="row g-4">
  {% for o in objectifs %}
  <div class="col-md-6"><div class="card-x hover card-pad h-100">
    <div class="d-flex gap-3 mb-3">
      <div class="icon-box tone-{{ o.couleur }}"><i class="bi {{ o.icone }}"></i></div>
      <div class="flex-fill"><strong>{{ o.titre }}</strong><div class="small text-muted">Échéance : {{ o.echeance }}</div></div>
      <span class="badge-demo align-self-start">Démo</span>
    </div>
    <div class="d-flex justify-content-between small fw-semibold mb-1"><span>Progression</span><span>{{ o.progression }}%</span>
</div>
    <div class="progress"><div class="progress-bar {% if o.couleur != 'primary' %}{{ o.couleur }}{% endif %}" style="width:{{
o.progression }}%"></div></div>
  </div></div>
  {% endfor %}
</div>
{% endblock %}
```

La barre est un `<div class="progress-bar" style="width:{{ o.progression }}%">` : la largeur dépend du pourcentage fictif du dictionnaire.

10.6 Mes tâches

Commande / adresse :

`http://127.0.0.1:8000/objectifs/mes-taches/` (objectifs:taches)



Listes de tâches en lecture seule (démonstration)

templates/objectifs/taches.html

```
{% extends "base.html" %}
{% block title %}Mes tâches{% endblock %}
{% block page_title %}Mes tâches{% endblock %}
{% block crumbs %}<li class="breadcrumb-item">Objectifs</li><li class="breadcrumb-item active">Mes tâches</li>{% endblock %}
{% block content %}
{% include "components/demo_banner.html" %}
<div class="page-head"><div><h2>Listes de tâches</h2><p class="text-muted mb-0">Exemples en lecture seule</p></div>{% include
"components/soon_button.html" with label="Nouvelle tâche" %}</div>
<div class="row g-4">
  {% for g in groupes %}
  <div class="col-lg-6"><div class="card-x card-pad h-100">
    <div class="d-flex justify-content-between mb-2"><h6 class="mb-0">{{ g.objectif }}</h6><span class="badge-demo">Démo</span>
</div>
    {% for t in g.taches %}
    <div class="task-item {% if t.fait %}done{% endif %}"><input class="form-check-input mt-0" type="checkbox" disabled {% if
t.fait %}checked{% endif %}><span>{{ t.titre }}</span></div>
    {% endfor %}
  </div></div>
  {% endfor %}
</div>
{% endblock %}
```

Les cases à cocher sont `disabled` : l'utilisateur ne peut rien modifier. Les tâches « faites » sont barrées par la classe CSS `.task-item.done`.

11. Accueil public et tableau de bord

11.1 Vues et URL

dashboard/views.py

```
from django.contrib.auth.decorators import login_not_required
from django.contrib.auth.models import Group, User
from django.db.models import Count
from django.shortcuts import redirect, render
from django.views.generic import TemplateView

from pedagogie.models import Cours, Exercice


@login_not_required
def landing(request):
    if request.user.is_authenticated:
        return redirect('dashboard:home')
    return render(request, 'dashboard/landing.html')


class HomeView(TemplateView):
    template_name = 'dashboard/home.html'

    def get_context_data(self, **kw):
        ctx = super().get_context_data(**kw)
        ctx['nb_cours'] = Cours.objects.count()
        ctx['nb_exercices'] = Exercice.objects.count()
        ctx['nb_users'] = User.objects.count()
        ctx['nb_roles'] = Group.objects.count()
        ctx['recent_cours'] = Cours.objects.select_related('enseignant')[:3]
        # Données réelles pour le graphique
        par_diff = dict(Exercice.objects.values_list('difficulte').annotate(n=Count('id')))
        ctx['chart_labels'] = [lbl for _, lbl in Exercice.DIFFICULTES]
        ctx['chart_data'] = [par_diff.get(k, 0) for k, _ in Exercice.DIFFICULTES]
        return ctx
```

dashboard/urls.py

```
from django.urls import path

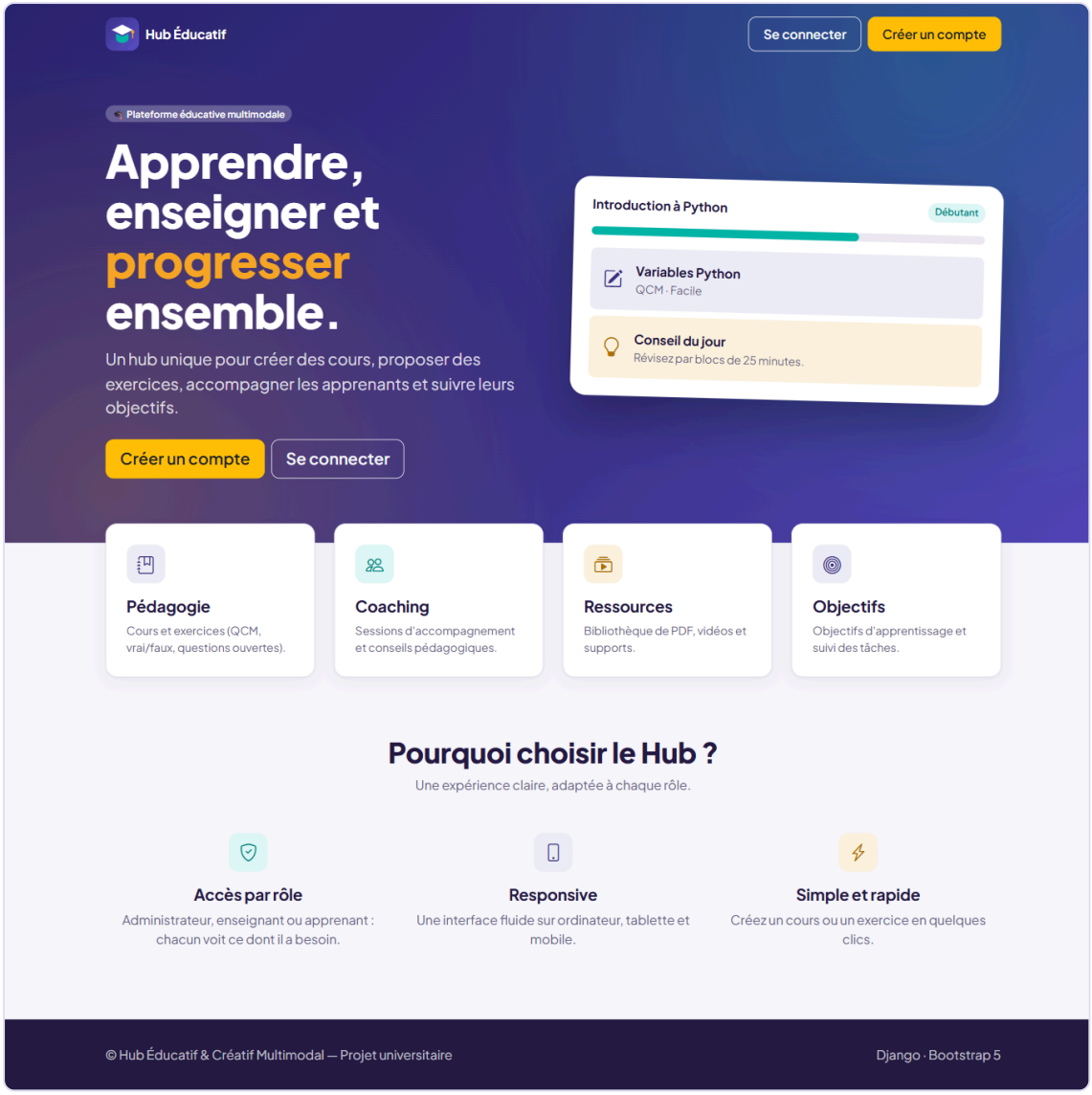
from . import views

app_name = 'dashboard'
urlpatterns = [
    path('', views.landing, name='landing'),
    path('tableau-de-bord/', views.HomeView.as_view(), name='home'),
]
```

11.2 Page d'accueil publique

Commande / adresse :

`http://127.0.0.1:8000/` (dashboard:landing) — visible sans être connecté



Page d'accueil : hero, modules, avantages

templates/dashboard/landing.html

```
{% load static %}<!doctype html>
<html lang="fr">
<head>{% include "components/head.html" %}<title>Hub Éducatif & Créatif Multimodal</title></head>
<body>
<section class="hero">
  <div class="container">
    <nav class="pub-nav d-flex align-items-center justify-content-between">
      <a href="{% url 'dashboard:landing' %}" class="d-flex align-items-center gap-2 text-white fw-bold"><img src="{% static
'images/logo.svg' %}" width="42" alt="">Hub Éducatif</a>
      <div class="d-flex gap-2"><a href="{% url 'accounts:login' %}" class="btn btn-outline-light">Se connecter</a><a href="{%
url 'accounts:register' %}" class="btn btn-warning text-dark d-none d-sm-inline-block">Créer un compte</a></div>
    </nav>
    <div class="row align-items-center g-5 pt-5">
      <div class="col-lg-6">
        <span class="badge badge-soft bg-white bg-opacity-25 mb-3">🎓 Plateforme éducative multimodale</span>
        <h1>Apprendre, enseigner et <em>progresser</em> ensemble.</h1>
        <p class="lead mt-3">Un hub unique pour créer des cours, proposer des exercices, accompagner les apprenants et suivre
leurs objectifs.</p>
        <div class="d-flex flex-wrap gap-2 mt-4">
          <a href="{% url 'accounts:register' %}" class="btn btn-warning btn-lg text-dark">Créer un compte</a>
          <a href="{% url 'accounts:login' %}" class="btn btn-outline-light btn-lg">Se connecter</a>
        </div>
      </div>
      <div class="col-lg-6 d-none d-lg-block">
        <div class="hero-mock">
          <div class="d-flex justify-content-between mb-3"><strong>Introduction à Python</strong><span class="badge badge-soft
badge-teal">Débutant</span></div>
          <div class="progress mb-3"><div class="progress-bar teal" style="width:68%"></div></div>
          <div class="d-flex align-items-center gap-3 p-3 rounded-3 mb-2" style="background:var(--indigo-50)"><i class="bi bi-
```

```
pencil-square fs-4 text-primary"></i><div><strong>Variables Python</strong><div class="small text-muted">QCM · Facile</div>
</div></div>
          <div class="d-flex align-items-center gap-3 p-3 rounded-3" style="background:var(--amber-50)"><i class="bi bi-
lightbulb fs-4" style="color:#b06f00"></i><div><strong>Conseil du jour</strong><div class="small text-muted">Révisez par blocs
de 25 minutes.</div></div></div>
        </div>
      </div>
    </div>
  </div>
</section>

<section class="container" style="margin-top:-3rem">
  <div class="row g-4">
    <div class="col-md-6 col-lg-3"><div class="card-x hover module-card"><div class="icon-box tone-primary mb-3"><i class="bi
bi-journal-bookmark"></i></div><h5>Pédagogie</h5><p class="text-muted small mb-0">Cours et exercices (QCM, vrai/faux, questions
ouvertes).</p></div></div>
    <div class="col-md-6 col-lg-3"><div class="card-x hover module-card"><div class="icon-box tone-teal mb-3"><i class="bi bi-
people"></i></div><h5>Coaching</h5><p class="text-muted small mb-0">Sessions d'accompagnement et conseils pédagogiques.</p>
</div></div>
    <div class="col-md-6 col-lg-3"><div class="card-x hover module-card"><div class="icon-box tone-amber mb-3"><i class="bi bi-
collection-play"></i></div><h5>Ressources</h5><p class="text-muted small mb-0">Bibliothèque de PDF, vidéos et supports.</p>
</div></div>
    <div class="col-md-6 col-lg-3"><div class="card-x hover module-card"><div class="icon-box tone-primary mb-3"><i class="bi
bi-bullseye"></i></div><h5>Objectifs</h5><p class="text-muted small mb-0">Objectifs d'apprentissage et suivi des tâches.</p>
</div></div>
  </div>
</section>

<section class="container py-5 my-4">
  <div class="text-center mb-5"><h2 class="section-title">Pourquoi choisir le Hub ?</h2><p class="text-muted">Une expérience
claire, adaptée à chaque rôle.</p></div>
  <div class="row g-4 text-center">
    <div class="col-md-4"><div class="icon-box tone-teal mx-auto mb-3"><i class="bi bi-shield-check"></i></div><h5>Accès par
rôle</h5><p class="text-muted">Administrateur, enseignant ou apprenant : chacun voit ce dont il a besoin.</p></div>
    <div class="col-md-4"><div class="icon-box tone-primary mx-auto mb-3"><i class="bi bi-phone"></i></div><h5>Responsive</h5><p
class="text-muted">Une interface fluide sur ordinateur, tablette et mobile.</p></div>
    <div class="col-md-4"><div class="icon-box tone-amber mx-auto mb-3"><i class="bi bi-lightning-charge"></i></div><h5>Simple
et rapide</h5><p class="text-muted">Créez un cours ou un exercice en quelques clics.</p></div>
  </div>
</section>

<footer class="footer-pub"><div class="container d-flex flex-wrap justify-content-between gap-2"><span>© Hub Éducatif & Créatif
Multimodal — Projet universitaire</span><span>Django · Bootstrap 5</span></div></footer>
{% include "components/scripts.html" %}
</body>
</html>
```

1. `@login_not_required` rend la page publique ; si l'utilisateur est déjà connecté, il est redirigé vers le tableau de bord.
2. Ce gabarit **n'étend pas** `base.html` (pas de sidebar) : il a sa propre structure (hero avec dégradé, 4 cartes de modules, section avantages, pied de page) et les boutons « Se connecter » / « Créer un compte ».

11.3 Tableau de bord

Commande / adresse :

`http://127.0.0.1:8000/tableau-de-bord/    (dashboard:home)`

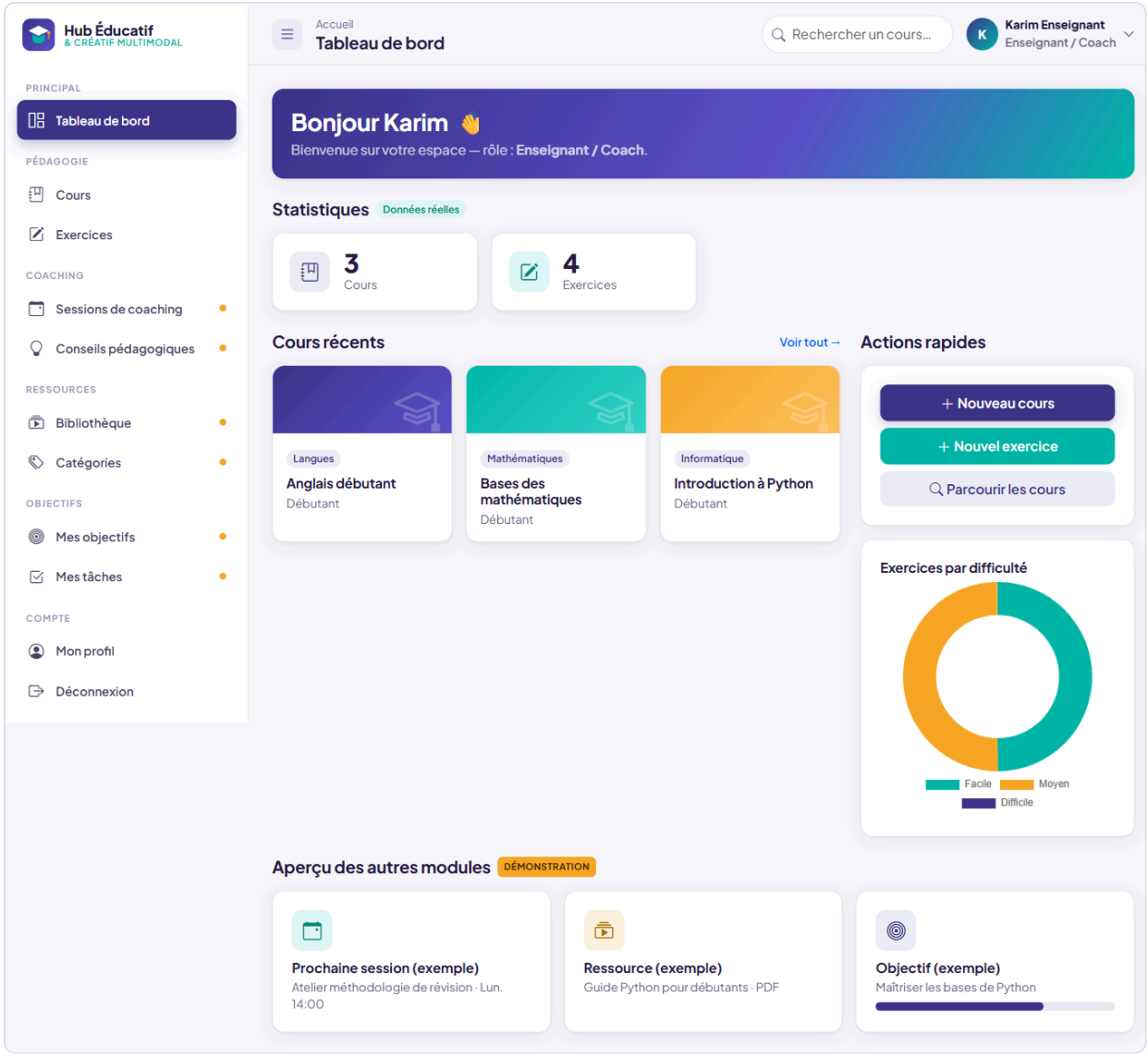


Tableau de bord vu par un enseignant

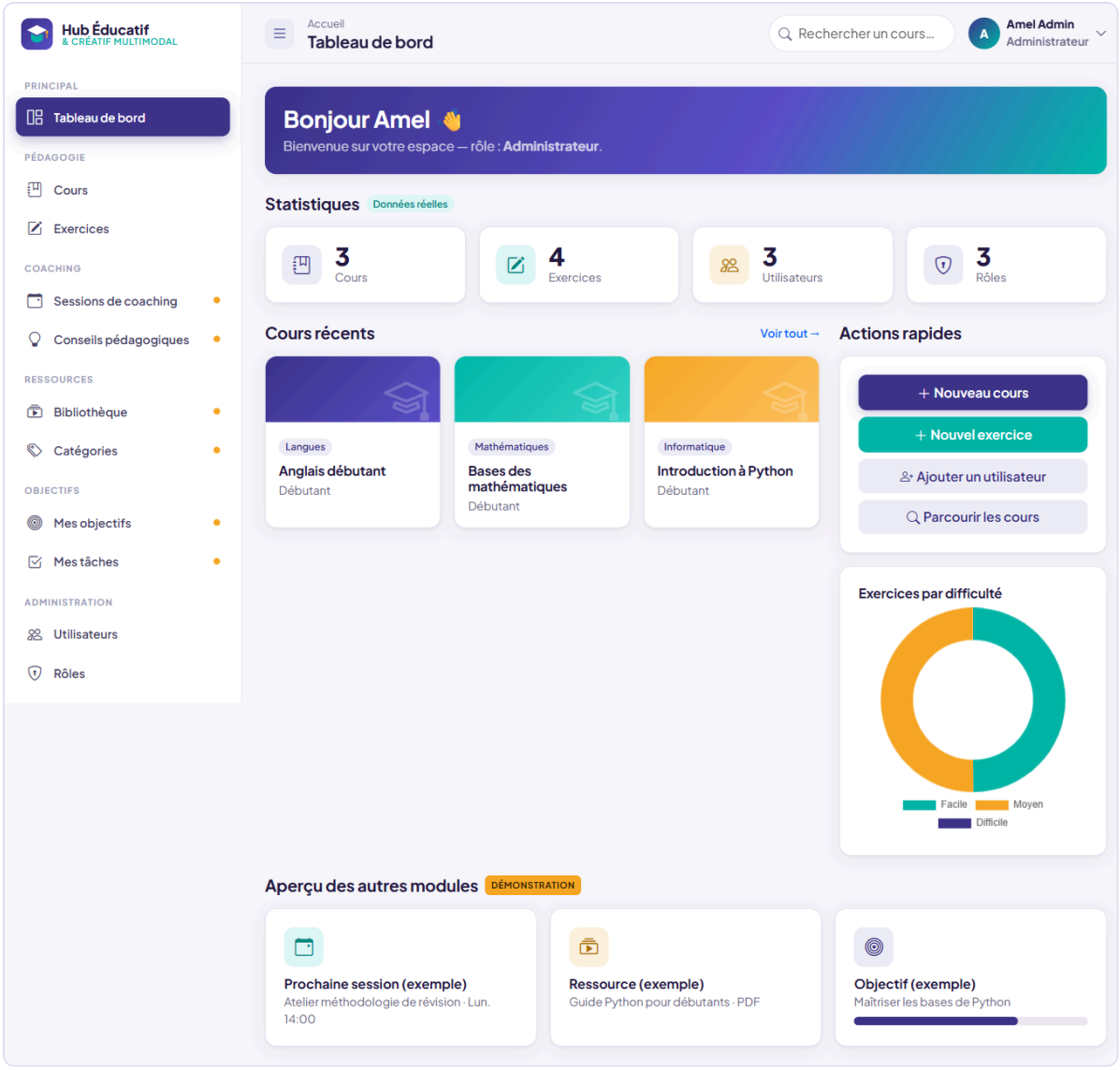


Tableau de bord vu par l'administrateur (cartes Utilisateurs et Rôles en plus)

templates/dashboard/home.html

```
{% extends "base.html" %}
{% block title %}Tableau de bord{% endblock %}
{% block page_title %}Tableau de bord{% endblock %}
{% block content %}
<div class="card-x card-pad mb-4 text-white border-0" style="background:linear-gradient(120deg,var(--indigo),#5b50c9 60%,var(--
teal))">
  <h3 class="mb-1">Bonjour {{ user.first_name|default:user.username }} 👋</h3>
  <p class="mb-0 opacity-75">Bienvenue sur votre espace — rôle : <strong>{{ user.groups.first.name|default:"Utilisateur" }}
</strong>.</p>
</div>

<div class="d-flex align-items-center gap-2 mb-3"><h5 class="mb-0">Statistiques</h5><span class="badge badge-soft badge-
teal">Données réelles</span></div>
<div class="row g-3 mb-4">
  <div class="col-sm-6 col-xl-3"><div class="card-x stat"><div class="icon-box tone-primary"><i class="bi bi-journal-bookmark">
</i></div><div><div class="num">{{ nb_cours }}</div><div class="text-muted small">Cours</div></div></div></div>
  <div class="col-sm-6 col-xl-3"><div class="card-x stat"><div class="icon-box tone-teal"><i class="bi bi-pencil-square"></i>
</div><div><div class="num">{{ nb_exercices }}</div><div class="text-muted small">Exercices</div></div></div></div>
  {% if perms.auth.view_user %}<div class="col-sm-6 col-xl-3"><div class="card-x stat"><div class="icon-box tone-amber"><i
class="bi bi-people"></i></div><div><div class="num">{{ nb_users }}</div><div class="text-muted small">Utilisateurs</div></div>
</div></div>{% endif %}
  {% if perms.auth.view_group %}<div class="col-sm-6 col-xl-3"><div class="card-x stat"><div class="icon-box tone-primary"><i
class="bi bi-shield-lock"></i></div><div><div class="num">{{ nb_roles }}</div><div class="text-muted small">Rôles</div></div>
</div></div>{% endif %}
</div>

<div class="row g-4 mb-4">
  <div class="col-lg-8">
    <div class="d-flex justify-content-between align-items-center mb-3"><h5 class="mb-0">Cours récents</h5><a href="{% url
```

```
'pedagogie:cours_list' %}" class="small fw-semibold">Voir tout →</a></div>
    <div class="row g-3">
      {% for c in recent_cours %}
      <div class="col-md-4"><a href="{{ c.get_absolute_url }}" class="card-x hover d-block text-reset h-100">
        <div class="course-cover {% cycle '' 'c-1' 'c-2' %}" style="height:80px"><i class="bi bi-mortarboard"></i></div>
        <div class="card-pad p-3"><span class="badge badge-soft badge-primary mb-2">{{ c.matiere }}</span><h6 class="mb-1">{{
c.titre }}</h6><div class="small text-muted">{{ c.get_niveau_display }}</div></div>
      </a></div>
      {% empty %}
      <div class="col-12">{% include "components/empty.html" with icon="bi-journal" title="Aucun cours" text="Les cours
apparaîtront ici dès qu'ils seront créés." %}</div>
      {% endfor %}
    </div>
  </div>
  <div class="col-lg-4">
    <h5 class="mb-3">Actions rapides</h5>
    <div class="card-x card-pad d-grid gap-2">
      {% if perms.pedagogie.add_cours %}<a href="{% url 'pedagogie:cours_create' %}" class="btn btn-primary"><i class="bi bi-
plus-lg me-1"></i>Nouveau cours</a>{% endif %}
      {% if perms.pedagogie.add_exercice %}<a href="{% url 'pedagogie:exercice_create' %}" class="btn btn-teal"><i class="bi bi-
plus-lg me-1"></i>Nouvel exercice</a>{% endif %}
      {% if perms.auth.add_user %}<a href="{% url 'accounts:user_create' %}" class="btn btn-soft"><i class="bi bi-person-plus
me-1"></i>Ajouter un utilisateur</a>{% endif %}
      <a href="{% url 'pedagogie:cours_list' %}" class="btn btn-soft"><i class="bi bi-search me-1"></i>Parcourir les cours</a>
    </div>
    <div class="card-x card-pad mt-3">
      <h6>Exercices par difficulté</h6>
      <canvas id="chart" height="170"></canvas>
    </div>
  </div>
</div>

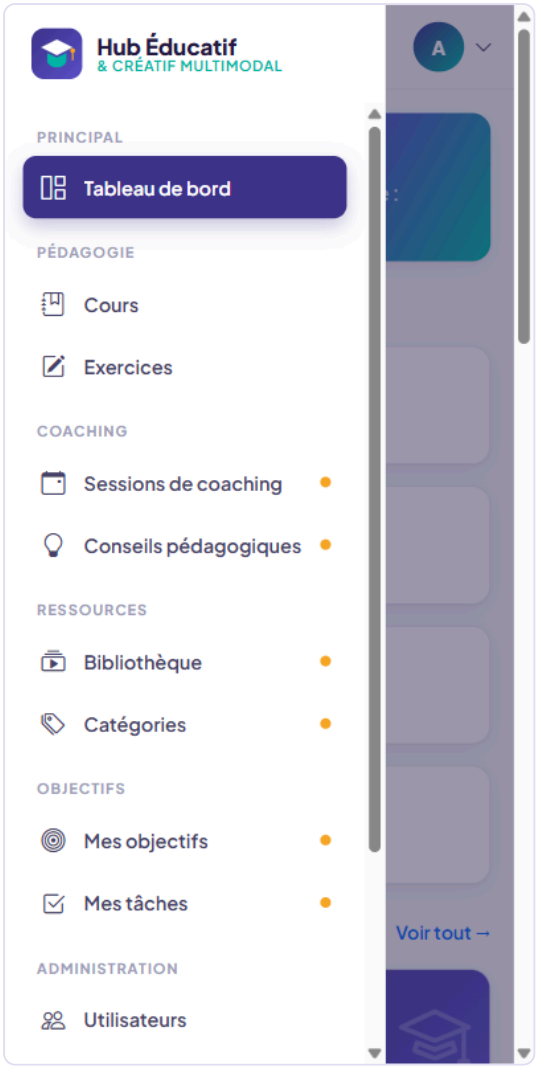
<div class="d-flex align-items-center gap-2 mb-3"><h5 class="mb-0">Aperçu des autres modules</h5><span class="badge-
demo">Démonstration</span></div>
<div class="row g-3">
  <div class="col-md-4"><a href="{% url 'coaching:sessions' %}" class="card-x hover card-pad d-block text-reset h-100"><div
class="icon-box tone-teal mb-2"><i class="bi bi-calendar-event"></i></div><strong>Prochaine session (exemple)</strong><div
class="small text-muted">Atelier méthodologie de révision · Lun. 14:00</div></a></div>
  <div class="col-md-4"><a href="{% url 'ressources:bibliotheque' %}" class="card-x hover card-pad d-block text-reset h-100">
<div class="icon-box tone-amber mb-2"><i class="bi bi-collection-play"></i></div><strong>Ressource (exemple)</strong><div
class="small text-muted">Guide Python pour débutants · PDF</div></a></div>
  <div class="col-md-4"><a href="{% url 'objectifs:objectifs' %}" class="card-x hover card-pad d-block text-reset h-100"><div
class="icon-box tone-primary mb-2"><i class="bi bi-bullseye"></i></div><strong>Objectif (exemple)</strong><div class="small
text-muted mb-2">Maîtriser les bases de Python</div><div class="progress"><div class="progress-bar" style="width:70%"></div>
</div></a></div>
</div>
{% endblock %}

{% block extra_js %}
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.3/dist/chart.umd.min.js"></script>
{{ chart_labels|json_script:"chart-labels" }}{{ chart_data|json_script:"chart-data" }}
<script>
new Chart(document.getElementById('chart'), {
  type: 'doughnut',
  data: { labels: JSON.parse(document.getElementById('chart-labels').textContent),
          datasets: [{ data: JSON.parse(document.getElementById('chart-data').textContent), backgroundColor: ['#00B8A9',
'#F9A826', '#3D348B'], borderWidth: 0 }] },
  options: { cutout: '65%', plugins: { legend: { position: 'bottom' } } }
});
</script>
{% endblock %}
```

1. **Statistiques réelles** : `Cours.objects.count()`, `Exercice.objects.count()`, `User.objects.count()`, `Group.objects.count()` — calculées depuis la base, badge « Données réelles ».
2. Les cartes Utilisateurs et Rôles sont entourées de `{% if perms.auth.view_user %}` : visibles seulement par l'administrateur.
3. **Cours récents** : `Cours.objects.select_related('enseignant')[:3]` (les 3 derniers, car ordre par date décroissante).
4. **Graphique Chart.js** : les données (`chart_labels`, `chart_data`) sont transmises avec `json_script` puis lues en JavaScript.
5. **Actions rapides** : boutons affichés selon les permissions.
6. **Aperçu des autres modules** : cartes marquées « Démonstration » (jamais mélangées avec les chiffres réels).

12. Gabarit de base, composants, CSS et JavaScript

12.1 base.html — sidebar, topbar, messages, modale



Version mobile : la sidebar devient un tiroir (bouton ☰)

templates/base.html

```
<!doctype html>
<html lang="fr">
<head>
  {% include "components/head.html" %}
  <title>{% block title %}Hub Éducatif{% endblock %} · Hub Éducatif</title>
  {% block extra_head %}{% endblock %}
</head>
<body>
{% with v=request.resolver_match.view_name %}
<aside class="sidebar" id="sidebar">
  {% load static %}
  <a href="{% url 'dashboard:home' %}" class="sidebar-brand"><img src="{% static 'images/logo.svg' %}" alt=""><span>Hub
Éducatif<small>& Créatif Multimodal</small></span></a>
  <nav class="sidebar-scroll">
    <div class="nav-section">Principal</div>
    <a class="nav-link-x {% if v == 'dashboard:home' %}active{% endif %}" href="{% url 'dashboard:home' %}"><i class="bi bi-
grid-1x2"></i><span class="label">Tableau de bord</span></a>

    <div class="nav-section">Pédagogie</div>
    <a class="nav-link-x {% if 'cours' in v %}active{% endif %}" href="{% url 'pedagogie:cours_list' %}"><i class="bi bi-
journal-bookmark"></i><span class="label">Cours</span></a>
    <a class="nav-link-x {% if 'exercice' in v %}active{% endif %}" href="{% url 'pedagogie:exercice_list' %}"><i class="bi bi-
pencil-square"></i><span class="label">Exercices</span></a>

    <div class="nav-section">Coaching</div>
```

```
    <a class="nav-link-x {% if v == 'coaching:sessions' %}active{% endif %}" href="{% url 'coaching:sessions' %}"><i class="bi
bi-calendar-event"></i><span class="label">Sessions de coaching</span><span class="tag" title="Démonstration"></span></a>
    <a class="nav-link-x {% if v == 'coaching:conseils' %}active{% endif %}" href="{% url 'coaching:conseils' %}"><i class="bi
bi-lightbulb"></i><span class="label">Conseils pédagogiques</span><span class="tag" title="Démonstration"></span></a>

    <div class="nav-section">Ressources</div>
    <a class="nav-link-x {% if v == 'ressources:bibliotheque' %}active{% endif %}" href="{% url 'ressources:bibliotheque' %}"><i
class="bi bi-collection-play"></i><span class="label">Bibliothèque</span><span class="tag" title="Démonstration"></span></a>
    <a class="nav-link-x {% if v == 'ressources:categories' %}active{% endif %}" href="{% url 'ressources:categories' %}"><i
class="bi bi-tags"></i><span class="label">Catégories</span><span class="tag" title="Démonstration"></span></a>

    <div class="nav-section">Objectifs</div>
    <a class="nav-link-x {% if v == 'objectifs:objectifs' %}active{% endif %}" href="{% url 'objectifs:objectifs' %}"><i
class="bi bi-bullseye"></i><span class="label">Mes objectifs</span><span class="tag" title="Démonstration"></span></a>
    <a class="nav-link-x {% if v == 'objectifs:taches' %}active{% endif %}" href="{% url 'objectifs:taches' %}"><i class="bi bi-
check2-square"></i><span class="label">Mes tâches</span><span class="tag" title="Démonstration"></span></a>

    {% if perms.auth.view_user or perms.auth.view_group %}
    <div class="nav-section">Administration</div>
    {% if perms.auth.view_user %}<a class="nav-link-x {% if 'user_' in v %}active{% endif %}" href="{% url 'accounts:user_list'
%}"><i class="bi bi-people"></i><span class="label">Utilisateurs</span></a>{% endif %}
    {% if perms.auth.view_group %}<a class="nav-link-x {% if 'role_' in v %}active{% endif %}" href="{% url 'accounts:role_list'
%}"><i class="bi bi-shield-lock"></i><span class="label">Rôles</span></a>{% endif %}
    {% endif %}

    <div class="nav-section">Compte</div>
    <a class="nav-link-x {% if 'profile' in v or v == 'accounts:password' %}active{% endif %}" href="{% url 'accounts:profile'
%}"><i class="bi bi-person-circle"></i><span class="label">Mon profil</span></a>
    <form method="post" action="{% url 'accounts:logout' %}">{% csrf_token %}<button class="nav-link-x"><i class="bi bi-box-
arrow-right"></i><span class="label">Déconnexion</span></button></form>
  </nav>
</aside>
<div class="sidebar-backdrop"></div>

<div class="app-main">
  <header class="topbar">
    <button class="btn btn-icon btn-soft" data-sidebar-toggle aria-label="Menu"><i class="bi bi-list fs-5"></i></button>
    <div class="me-auto">
      <nav aria-label="Fil d'Ariane"><ol class="breadcrumb">
        <li class="breadcrumb-item"><a href="{% url 'dashboard:home' %}">Accueil</a></li>
        {% block crumbs %}{% endblock %}
      </ol></nav>
      <h1>{% block page_title %}{% endblock %}</h1>
    </div>
    <form class="topbar-search d-none d-md-block" action="{% url 'pedagogie:cours_list' %}" role="search">
      <i class="bi bi-search"></i><input class="form-control" name="q" placeholder="Rechercher un cours…">
    </form>
    <div class="dropdown">
      <button class="btn p-0 border-0 d-flex align-items-center gap-2" data-bs-toggle="dropdown">
        <span class="avatar">{{ user.first_name|default:user.username|first|upper }}</span>
        <span class="d-none d-sm-block text-start lh-sm"><strong class="d-block small">{{
user.get_full_name|default:user.username }}</strong><small class="text-muted">{{ user.groups.first.name|default:"Utilisateur" }}
</small></span>
        <i class="bi bi-chevron-down small"></i>
      </button>
      <ul class="dropdown-menu dropdown-menu-end">
        <li><a class="dropdown-item" href="{% url 'accounts:profile' %}"><i class="bi bi-person me-2"></i>Mon profil</a></li>
        <li><a class="dropdown-item" href="{% url 'accounts:password' %}"><i class="bi bi-key me-2"></i>Mot de passe</a></li>
        <li><hr class="dropdown-divider"></li>
        <li><form method="post" action="{% url 'accounts:logout' %}">{% csrf_token %}<button class="dropdown-item text-danger">
<i class="bi bi-box-arrow-right me-2"></i>Déconnexion</button></form></li>
      </ul>
    </div>
  </header>

  <main class="content">
    {% for m in messages %}<div class="alert alert-{{ m.tags|default:'info' }} alert-dismissible fade show">{{ m }}<button
type="button" class="btn-close" data-bs-dismiss="alert"></button></div>{% endfor %}
    {% block content %}{% endblock %}
  </main>
</div>
{% endwith %}

<div class="modal fade" id="deleteModal" tabindex="-1" aria-hidden="true">
  <div class="modal-dialog modal-dialog-centered">
    <form method="post" class="modal-content">{% csrf_token %}
      <div class="modal-body text-center p-4">
        <div class="icon-box mx-auto mb-3 tone-amber"><i class="bi bi-exclamation-triangle"></i></div>
        <h5>Confirmer la suppression</h5>
        <p class="text-muted mb-4">Voulez-vous vraiment supprimer « <strong data-name></strong> » ? Cette action est
```

```
irréversible.</p>
        <button type="button" class="btn btn-soft me-2" data-bs-dismiss="modal">Annuler</button>
        <button class="btn btn-danger">Supprimer</button>
      </div>
    </form>
  </div>
</div>
{% include "components/scripts.html" %}
{% block extra_js %}{% endblock %}
</body>
</html>
```

1. **Blocs** : `{% block title %}`, `page_title`, `crumbs` (fil d'Ariane), `content`. Chaque page remplit ces blocs ; tout le reste (sidebar, barre du haut) est commun.
2. **Page active** : `request.resolver_match.view_name` contient le nom d'URL (ex. `pedagogie:cours_list`) ; `{% if 'cours' in v %}active{% endif %}` met en évidence le lien.
3. **Rubriques** : Principal, Pédagogie, Coaching, Ressources, Objectifs, Administration (si permissions), Compte. Un point ambre signale les modules de démonstration.
4. **Topbar** : bouton ☰, fil d'Ariane, titre, recherche de cours, avatar (initiale) + menu du compte (profil, mot de passe, déconnexion).
5. **Messages** : `{% for m in messages %}` affiche les messages de succès/erreur envoyés par les vues.
6. **Modale de suppression unique** : un seul formulaire POST dont l'adresse est remplie par JavaScript selon le bouton cliqué.

12.2 Composants réutilisables (templates/components/)

📄 templates/components/head.html

```
{% load static %}<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="icon" href="{% static 'images/logo.svg' %}">
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&display=swap" rel="stylesheet">
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css" rel="stylesheet">
<link href="{% static 'css/hub.css' %}?v={{ ASSET_V }}" rel="stylesheet">
```

📄 templates/components/scripts.html

```
{% load static %}<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
<script src="{% static 'js/hub.js' %}?v={{ ASSET_V }}"></script>
```

📄 templates/components/pagination.html

```
{% if page_obj.has_other_pages %}
<nav class="mt-4" aria-label="Pagination">
  <ul class="pagination justify-content-center flex-wrap">
    {% if page_obj.has_previous %}<li class="page-item"><a class="page-link" href="?{% if querystring %}{{ querystring }}&{%
endif %}page={{ page_obj.previous_page_number }}"><i class="bi bi-chevron-left"></i></a></li>{% endif %}
    {% for n in paginator.page_range %}
      <li class="page-item {% if n == page_obj.number %}active{% endif %}"><a class="page-link" href="?{% if querystring %}{{
querystring }}&{% endif %}page={{ n }}">{{ n }}</a></li>
    {% endfor %}
    {% if page_obj.has_next %}<li class="page-item"><a class="page-link" href="?{% if querystring %}{{ querystring }}&{% endif
%}page={{ page_obj.next_page_number }}"><i class="bi bi-chevron-right"></i></a></li>{% endif %}
  </ul>
</nav>
{% endif %}
```

📄 templates/components/empty.html

```
{# Usage : {% include "components/empty.html" with icon="bi-journal" title="..." text="..." %} #}
<div class="card-x empty">
  <div class="icon-box tone-primary"><i class="bi {{ icon|default:'bi-inbox' }}"></i></div>
  <h5>{{ title }}</h5>
  <p class="text-muted mb-3">{{ text }}</p>
```

```
  {{ extra|default:"" }}
</div>
```

Composant	Utilisation
`field.html`	`{% include "components/field.html" with field=form.titre %}`
`pagination.html`	`{% include "components/pagination.html" %}` sous une liste paginée
`empty.html`	`{% include "components/empty.html" with icon="bi-inbox" title="…" text="…" %}`
`demo_banner.html` / `soon_button.html`	Pages statiques

12.3 Design system : static/css/hub.css

Le fichier est organisé en sections : variables de couleurs (`:root`), boutons, formulaires, cartes, badges, tableaux, avatar, pagination/alertes/modales, mise en page de l'application (sidebar, topbar), pages de démonstration, pages publiques, responsive.

Responsive : jusqu'à 1199 px, la sidebar est cachée (`transform: translateX(-100%)`) et s'ouvre en tiroir avec un fond assombri ; sous 768 px, les en-têtes passent en colonne, les boutons occupent toute la largeur, le calendrier est compacté. Les grilles Bootstrap (`col-md-6 col-xl-4`) réorganisent les cartes selon la largeur.

static/css/hub.css

```
/* ===== Hub Éducatif — design system ===== */
:root {
  --indigo: #3D348B; --indigo-600: #32297a; --indigo-50: #eeedf8; --indigo-100: #dcd9f0;
  --teal: #00B8A9; --teal-50: #e0f7f5;
  --amber: #F9A826; --amber-50: #fef3de;
  --ink: #231D45; --muted: #6b6788; --line: #e7e5f0;
  --bg: #f6f5fb; --surface: #fff;
  --radius: 14px; --radius-sm: 10px;
  --shadow: 0 1px 2px rgba(35,29,69,.05), 0 4px 16px rgba(35,29,69,.06);
  --shadow-lg: 0 10px 30px rgba(61,52,139,.16);
  --sidebar-w: 288px; --sidebar-w-collapsed: 80px;
  --bs-primary: #3D348B; --bs-primary-rgb: 61,52,139; --bs-body-color: #231D45;
  --bs-body-font-family: 'Plus Jakarta Sans', system-ui, sans-serif; --bs-link-color: #3D348B;
}
body { background: var(--bg); color: var(--ink); font-family: var(--bs-body-font-family); -webkit-font-smoothing: antialiased; }
h1,h2,h3,h4,h5,h6 { font-weight: 700; letter-spacing: -.01em; }
a { text-decoration: none; }
.text-muted, .text-secondary { color: var(--muted) !important; }

/* Boutons */
.btn { border-radius: 10px; font-weight: 600; padding: .55rem 1.1rem; }
.btn-primary { --bs-btn-bg: var(--indigo); --bs-btn-border-color: var(--indigo); --bs-btn-hover-bg: var(--indigo-600); --bs-btn-
hover-border-color: var(--indigo-600); --bs-btn-active-bg: var(--indigo-600); --bs-btn-active-border-color: var(--indigo-600); -
-bs-btn-disabled-bg: var(--indigo); --bs-btn-disabled-border-color: var(--indigo); box-shadow: 0 4px 12px rgba(61,52,139,.25); }
.btn-teal { --bs-btn-color: #fff; --bs-btn-bg: var(--teal); --bs-btn-border-color: var(--teal); --bs-btn-hover-color: #fff; --
bs-btn-hover-bg: #00a094; --bs-btn-hover-border-color: #00a094; }
.btn-outline-primary { --bs-btn-color: var(--indigo); --bs-btn-border-color: var(--indigo-100); --bs-btn-hover-bg: var(--
indigo); --bs-btn-hover-border-color: var(--indigo); --bs-btn-active-bg: var(--indigo); }
.btn-soft { background: var(--indigo-50); color: var(--indigo); border: 0; }
.btn-soft:hover { background: var(--indigo-100); color: var(--indigo); }
.btn-icon { width: 36px; height: 36px; padding: 0; display: inline-grid; place-items: center; border-radius: 10px; }
.btn-soon { background: var(--bg); color: var(--muted); border: 1px dashed var(--indigo-100); cursor: not-allowed; }

/* Formulaires */
.form-control, .form-select { border-radius: 10px; border-color: var(--line); padding: .6rem .85rem; background-color: #fff; }
.form-control:focus, .form-select:focus { border-color: var(--indigo); box-shadow: 0 0 0 .22rem rgba(61,52,139,.14); }
.form-label { font-weight: 600; font-size: .875rem; margin-bottom: .35rem; }
.form-check-input:checked { background-color: var(--indigo); border-color: var(--indigo); }
.invalid-feedback { display: block; }

/* Cartes */
.card-x { background: var(--surface); border: 1px solid var(--line); border-radius: var(--radius); box-shadow: var(--shadow); }
```

```
.card-x.hover { transition: transform .18s, box-shadow .18s; }
.card-x.hover:hover { transform: translateY(-3px); box-shadow: var(--shadow-lg); }
.card-pad { padding: 1.4rem; }
.stat { display: flex; align-items: center; gap: 1rem; padding: 1.25rem; }
.stat .num { font-size: 1.9rem; font-weight: 800; line-height: 1; }
.icon-box { width: 48px; height: 48px; border-radius: 12px; display: grid; place-items: center; font-size: 1.35rem; flex: none;
}
.tone-primary { background: var(--indigo-50); color: var(--indigo); }
.tone-teal { background: var(--teal-50); color: #008a7f; }
.tone-amber { background: var(--amber-50); color: #b06f00; }

/* Cours (cartes pédagogiques) */
.course-cover { height: 110px; border-radius: var(--radius) var(--radius) 0 0; position: relative; color: #fff; display: flex;
align-items: flex-end; padding: .9rem 1.1rem; background: linear-gradient(135deg, var(--indigo), #5b50c9); }
.course-cover.c-1 { background: linear-gradient(135deg, var(--teal), #35d3c6); }
.course-cover.c-2 { background: linear-gradient(135deg, var(--amber), #fbc15e); }
.course-cover i { position: absolute; right: 14px; top: 8px; font-size: 3.4rem; opacity: .22; }
.clamp-2 { display: -webkit-box; -webkit-line-clamp: 2; -webkit-box-orient: vertical; overflow: hidden; }

/* Badges */
.badge-soft { font-weight: 600; padding: .38em .7em; border-radius: 999px; font-size: .74rem; }
.badge-primary { background: var(--indigo-50); color: var(--indigo); }
.badge-teal { background: var(--teal-50); color: #008a7f; }
.badge-amber { background: var(--amber-50); color: #a46500; }
.badge-danger { background: #fde8ea; color: #c4283c; }
.badge-gray { background: #eceaf3; color: var(--muted); }
.badge-demo { background: var(--amber); color: #3b2800; font-weight: 700; letter-spacing: .04em; text-transform: uppercase;
font-size: .68rem; padding: .35em .7em; border-radius: 6px; }

/* Tableaux */
.table-x { margin: 0; }
.table-x th { font-size: .75rem; text-transform: uppercase; letter-spacing: .05em; color: var(--muted); background: #faf9fd;
border-bottom: 1px solid var(--line); padding: .85rem 1rem; white-space: nowrap; }
.table-x td { padding: .85rem 1rem; vertical-align: middle; border-color: var(--line); }
.table-x tbody tr:hover { background: #faf9fd; }

/* Avatar */
.avatar { width: 38px; height: 38px; border-radius: 50%; background: linear-gradient(135deg, var(--indigo), var(--teal)); color:
#fff; display: inline-grid; place-items: center; font-weight: 700; font-size: .9rem; flex: none; }
.avatar.lg { width: 84px; height: 84px; font-size: 2rem; }

/* Pagination / alertes / modales */
.pagination { --bs-pagination-active-bg: var(--indigo); --bs-pagination-active-border-color: var(--indigo); --bs-pagination-
color: var(--indigo); --bs-pagination-border-radius: 10px; gap: .25rem; }
.page-link { border-radius: 10px !important; border-color: var(--line); }
.alert { border: 0; border-radius: var(--radius-sm); font-weight: 500; box-shadow: var(--shadow); }
.modal-content { border: 0; border-radius: var(--radius); box-shadow: var(--shadow-lg); }
.dropdown-menu { border: 1px solid var(--line); border-radius: 12px; box-shadow: var(--shadow-lg); padding: .5rem; }
.dropdown-item { border-radius: 8px; font-weight: 500; }
.breadcrumb { font-size: .8rem; margin: 0; }
.breadcrumb-item a { color: var(--muted); }

/* État vide */
.empty { text-align: center; padding: 3.5rem 1rem; }
.empty .icon-box { width: 76px; height: 76px; font-size: 2rem; margin: 0 auto 1rem; border-radius: 50%; }

/* ===== Layout application ===== */
.sidebar { position: fixed; inset: 0 auto 0 0; width: var(--sidebar-w); background: var(--surface); border-right: 1px solid
var(--line); display: flex; flex-direction: column; z-index: 1040; transition: width .2s, transform .25s; }
.sidebar-brand { display: flex; align-items: center; gap: .7rem; padding: 1.1rem 1.25rem; font-weight: 800; color: var(--ink);
font-size: 1.05rem; line-height: 1.1; }
.sidebar-brand img { width: 38px; height: 38px; flex: none; }
.sidebar-brand small { display: block; font-weight: 600; color: var(--teal); font-size: .68rem; letter-spacing: .06em; text-
transform: uppercase; }
.sidebar-scroll { flex: 1; overflow-y: auto; padding: 0 .85rem 1rem; }
.nav-section { font-size: .68rem; font-weight: 700; letter-spacing: .09em; text-transform: uppercase; color: #a29fbb; padding:
1.1rem .65rem .4rem; white-space: nowrap; }
.nav-link-x { display: flex; align-items: center; gap: .75rem; padding: .62rem .75rem; border-radius: 10px; color: #4a4569;
font-weight: 600; font-size: .9rem; border: 0; background: none; width: 100%; text-align: left; white-space: nowrap; }
.nav-link-x i { font-size: 1.15rem; width: 22px; text-align: center; flex: none; }
.nav-link-x:hover { background: var(--indigo-50); color: var(--indigo); }
.nav-link-x.active { background: var(--indigo); color: #fff; box-shadow: 0 6px 14px rgba(61,52,139,.28); }
.nav-link-x .tag { margin-left: auto; font-size: .56rem; background: var(--amber-50); color: #a46500; padding: .15rem .4rem;
border-radius: 5px; font-weight: 700; }
.nav-link-x.active .tag { background: rgba(255,255,255,.2); color: #fff; }

.app-main { margin-left: var(--sidebar-w); min-height: 100vh; transition: margin .2s; }
.topbar { position: sticky; top: 0; z-index: 1030; background: rgba(246,245,251,.88); backdrop-filter: blur(10px); display:
flex; align-items: center; gap: 1rem; padding: .85rem 1.75rem; border-bottom: 1px solid var(--line); }
.topbar h1 { font-size: 1.25rem; margin: 0; }
```

```
.topbar-search { max-width: 280px; position: relative; }
.topbar-search i { position: absolute; left: 12px; top: 50%; transform: translateY(-50%); color: var(--muted); }
.topbar-search input { padding-left: 2.2rem; border-radius: 999px; }
.content { padding: 1.75rem; max-width: 1360px; }
.page-head { display: flex; flex-wrap: wrap; justify-content: space-between; align-items: center; gap: 1rem; margin-bottom:
1.5rem; }
.page-head h2 { margin: 0; font-size: 1.5rem; }

body.sidebar-collapsed .sidebar { width: var(--sidebar-w-collapsed); }
body.sidebar-collapsed .app-main { margin-left: var(--sidebar-w-collapsed); }
body.sidebar-collapsed .sidebar .label, body.sidebar-collapsed .sidebar .tag, body.sidebar-collapsed .sidebar-brand span {
display: none; }
body.sidebar-collapsed .nav-section { visibility: hidden; padding: .4rem 0; }
body.sidebar-collapsed .nav-link-x { justify-content: center; }
.sidebar-backdrop { display: none; }
@media (max-width: 1199.98px) {
  .sidebar { transform: translateX(-100%); width: var(--sidebar-w) !important; }
  body.sidebar-open .sidebar { transform: none; }
  body.sidebar-open .sidebar-backdrop { display: block; position: fixed; inset: 0; background: rgba(35,29,69,.45); z-index:
1035; }
  .app-main, body.sidebar-collapsed .app-main { margin-left: 0; }
  body.sidebar-collapsed .sidebar .label, body.sidebar-collapsed .sidebar .tag, body.sidebar-collapsed .sidebar-brand span {
display: inline; }
  .topbar { padding: .75rem 1rem; } .content { padding: 1.1rem; }
}

/* ===== Pages statiques / démo ===== */
.demo-banner { display: flex; align-items: center; gap: .8rem; background: var(--amber-50); border: 1px dashed var(--amber);
border-radius: var(--radius-sm); padding: .8rem 1rem; margin-bottom: 1.5rem; font-size: .9rem; }
.progress { height: 10px; border-radius: 999px; background: #eceaf3; }
.progress-bar { border-radius: 999px; background: var(--indigo); }
.progress-bar.teal { background: var(--teal); } .progress-bar.amber { background: var(--amber); }
.cal { width: 100%; table-layout: fixed; border-collapse: separate; border-spacing: 6px; }
.cal th { text-align: center; font-size: .75rem; color: var(--muted); text-transform: uppercase; }
.cal td { height: 84px; vertical-align: top; background: #faf9fd; border-radius: 10px; padding: .4rem .5rem; font-size: .8rem;
font-weight: 600; }
.cal td.empty-day { background: transparent; }
.cal td.today { outline: 2px solid var(--indigo); }
.cal .ev { display: block; margin-top: .25rem; padding: .15rem .35rem; border-radius: 6px; font-size: .68rem; font-weight: 600;
white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
.ev.primary { background: var(--indigo-50); color: var(--indigo); } .ev.teal { background: var(--teal-50); color: #008a7f; }
.ev.amber { background: var(--amber-50); color: #a46500; }
.chip { border: 1px solid var(--line); background: #fff; border-radius: 999px; padding: .4rem 1rem; font-weight: 600; font-size:
.85rem; color: var(--ink); }
.chip.active { background: var(--indigo); color: #fff; border-color: var(--indigo); }
.task-item { display: flex; gap: .7rem; align-items: center; padding: .7rem 0; border-bottom: 1px solid var(--line); }
.task-item:last-child { border: 0; }
.task-item.done span { text-decoration: line-through; color: var(--muted); }

/* ===== Public : accueil + auth ===== */
.pub-nav { padding: 1rem 0; }
.hero { background: radial-gradient(circle at 85% 10%, rgba(0,184,169,.28), transparent 45%), radial-gradient(circle at 10% 90%,
rgba(249,168,38,.22), transparent 40%), linear-gradient(135deg, #2c2470, var(--indigo) 60%, #5246b8); color: #fff; padding-
bottom: 5rem; }
.hero h1 { font-size: clamp(2.2rem, 5vw, 3.6rem); font-weight: 800; line-height: 1.08; }
.hero h1 em { font-style: normal; color: var(--amber); }
.hero .lead { color: rgba(255,255,255,.82); max-width: 560px; }
.hero-mock { background: #fff; color: var(--ink); border-radius: 20px; padding: 1.4rem; box-shadow: 0 30px 60px rgba(0,0,0,.28);
transform: rotate(1.5deg); }
.section-title { font-weight: 800; font-size: clamp(1.6rem, 3vw, 2.2rem); }
.module-card { padding: 1.6rem; height: 100%; }
.footer-pub { background: var(--ink); color: rgba(255,255,255,.7); padding: 2rem 0; }
.auth-wrap { min-height: 100vh; display: grid; grid-template-columns: 1fr 1fr; }
.auth-side { background: linear-gradient(145deg, #2c2470, var(--indigo) 55%, #00a094); color: #fff; padding: 3rem; display:
flex; flex-direction: column; justify-content: center; }
.auth-form { display: flex; align-items: center; justify-content: center; padding: 2rem; background: var(--bg); }
.auth-card { width: 100%; max-width: 440px; }
@media (max-width: 860px) { .auth-wrap { grid-template-columns: 1fr; } .auth-side { display: none; } }
.nav-link-x .label { overflow: hidden; text-overflow: ellipsis; min-width: 0; } .nav-link-x .tag { flex: none; }
.nav-link-x .tag { width: 8px; height: 8px; padding: 0; border-radius: 50%; background: var(--amber); font-size: 0; } .nav-link-
x.active .tag { background: var(--amber); }

/* ===== Responsive complémentaire ===== */
.table-responsive { -webkit-overflow-scrolling: touch; }
@media (max-width: 767.98px) {
  .page-head { flex-direction: column; align-items: stretch; } .page-head .btn { width: 100%; }
  .topbar h1 { font-size: 1.05rem; } .breadcrumb { flex-wrap: nowrap; overflow: hidden; white-space: nowrap; }
  .card-pad { padding: 1rem; } .stat .num { font-size: 1.6rem; }
  .cal td { height: 56px; padding: .25rem; font-size: .7rem; } .cal .ev { display: none; } .cal { border-spacing: 3px; }
```

```
  .modal-dialog { margin: 1rem; }
}
```

12.4 JavaScript : static/js/hub.js

static/js/hub.js

```
// Interactions purement visuelles (aucune donnée n'est modifiée ici).
(function () {
  const body = document.body;
  const safe = (fn) => { try { return fn(); } catch (e) { return null; } };

  // Sidebar : repliable (ordinateur) / tiroir (mobile)
  if (safe(() => localStorage.getItem('sidebar') === 'collapsed')) body.classList.add('sidebar-collapsed');
  document.querySelectorAll('[data-sidebar-toggle]').forEach((b) => b.addEventListener('click', () => {
    if (window.innerWidth < 1200) return body.classList.toggle('sidebar-open');
    const c = body.classList.toggle('sidebar-collapsed');
    safe(() => localStorage.setItem('sidebar', c ? 'collapsed' : 'open'));
  }));
  document.querySelectorAll('.sidebar-backdrop').forEach((b) => b.addEventListener('click', () =>
body.classList.remove('sidebar-open')));

  // Modale de suppression partagée
  const modal = document.getElementById('deleteModal');
  if (modal) modal.addEventListener('show.bs.modal', (e) => {
    const t = e.relatedTarget;
    modal.querySelector('form').action = t.dataset.url;
    modal.querySelector('[data-name]').textContent = t.dataset.name;
  });

  // Formulaire d'exercice : afficher les propositions selon le type
  const type = document.getElementById('id_type_exercice');
  if (type) {
    const props = document.getElementById('id_propositions').closest('.field');
    const hint = document.getElementById('answer-hint');
    const area = document.getElementById('id_bonne_reponse');
    const propsArea = document.getElementById('id_propositions');
    const sel = document.createElement('select');
    sel.className = 'form-select';
    area.after(sel);
    sel.addEventListener('change', () => { area.value = sel.value; });
    const fill = (choices) => {
      const current = area.value.trim().toLowerCase();
      sel.innerHTML = '<option value="">--- Choisir la bonne réponse ---</option>';
      choices.forEach((c) => {
        const o = new Option(c, c);
        o.selected = c.toLowerCase() === current;
        sel.add(o);
      });
      area.value = sel.value;
    };
    const sync = () => {
      const t = type.value;
      props.style.display = t === 'qcm' ? '' : 'none';
      hint.textContent = { qcm: 'Choisissez la bonne réponse parmi les propositions.', vf: 'Choisissez Vrai ou Faux.', ouverte:
'Réponse attendue (modèle de correction).' }[t];
      area.style.display = t === 'ouverte' ? '' : 'none';
      sel.style.display = t === 'ouverte' ? 'none' : '';
      if (t === 'vf') fill(['Vrai', 'Faux']);
      if (t === 'qcm') fill(propsArea.value.split('\n').map((p) => p.trim()).filter(Boolean));
    };
    type.addEventListener('change', sync);
    propsArea.addEventListener('input', sync);
    sync();
  }

  // Filtres de démonstration (onglets + recherche visuelle)
  const cards = [...document.querySelectorAll('[data-kind]')];
  let kind = 'all'; let term = '';
  const apply = () => cards.forEach((c) => {
    const ok = (kind === 'all' || c.dataset.kind === kind) && c.textContent.toLowerCase().includes(term);
    c.classList.toggle('d-none', !ok);
  });
  document.querySelectorAll('[data-filter]').forEach((b) => b.addEventListener('click', () => {
    document.querySelectorAll('[data-filter]').forEach((x) => x.classList.remove('active'));
    b.classList.add('active'); kind = b.dataset.filter; apply();
  }));
```

```
  const s = document.querySelector('[data-demo-search]');
  if (s) s.addEventListener('input', () => { term = s.value.toLowerCase(); apply(); });
})();
```

1. **Sidebar** : sur ordinateur, le bouton ☰ replie la sidebar (état mémorisé dans `localStorage`) ; sur écran étroit, il ouvre/ferme le tiroir.
2. **Modale de suppression** : à l'ouverture, copie `data-url` dans l'action du formulaire et `data-name` dans le texte.
3. **Formulaire d'exercice** : liste déroulante de la bonne réponse (voir 9.8).
4. **Filtres de démonstration** : onglets et recherche des ressources (visuel seulement).

13. Tests effectués

Contrôle	Résultat
`python manage.py check`	Aucune erreur
Parcours automatique (client de test Django) : 63 requêtes sur toutes les pages avec les 3 rôles	Statuts attendus (200, 302, 403)
Apprenant : créer/supprimer un cours ou accéder à l'administration	Refusé (403)
Enseignant : accéder aux utilisateurs/rôles	Refusé (403)
QCM avec réponse hors propositions	Refusé avec message d'erreur
QCM valide (liste déroulante) enregistré puis supprimé	OK
Suppression du rôle système « Apprenant »	Refusée
Vérification de réponse (QCM, Vrai/Faux, question ouverte)	Messages « Bonne réponse » / « Ce n'est pas la bonne réponse » corrects
Navigateur réel : 13 pages, console JavaScript	13 × statut 200, 0 erreur
Responsive : 8 pages × 6 largeurs (380 à 1280 px)	Aucun débordement, sidebar en tiroir sous 1200 px

Limites connues : Bootstrap, polices et Chart.js sont chargés par Internet (CDN). Le mode DEBUG est actif (développement) : avant un déploiement il faudrait désactiver DEBUG, définir SECRET_KEY et ALLOWED_HOSTS.

14. Questions probables du professeur (avec réponses)

1. **Pourquoi Django ?**
   C'est un framework Python « batteries incluses » : ORM, authentification, formulaires, administration, sécurité (CSRF, hachage des mots de passe) sont fournis. Il suit l'architecture MVT.

2. **Où sont stockées les données ?**
   Dans `db.sqlite3`. Les tables sont créées par les migrations à partir des modèles Python. Pas de SQL écrit à la main : on utilise l'ORM (`Cours.objects.filter(...)`).

3. **Comment les rôles fonctionnent-ils ?**
   Un rôle est un `Group` Django auquel on associe des permissions (`add_cours`, `view_user`…). Un utilisateur appartient à un groupe. Les vues vérifient les permissions avec `PermissionRequiredMixin`.

4. **Comment distinguer enseignant et apprenant ?**
   Par leur groupe (donc leurs permissions) : l'enseignant a add/change/delete/view sur cours et exercices, l'apprenant seulement view. En plus, un enseignant ne modifie que **ses** cours (`OwnerOrAdminMixin`).

5. **Comment est sécurisé le mot de passe ?**
   Django le hache (PBKDF2 + sel) avec `set_password` / `UserCreationForm`. Il n'est jamais stocké en clair. Les formulaires POST sont protégés par le jeton CSRF.

6. **Que se passe-t-il si un apprenant tape l'adresse d'ajout d'un cours ?**
   Il reçoit une erreur 403 « Accès refusé » : la vue exige la permission `pedagogie.add_cours`. Masquer le bouton ne suffit pas, la vraie protection est côté serveur.

7. **Pourquoi `PROTECT` sur l'enseignant du cours ?**
   Pour ne pas perdre des cours en supprimant un compte. On propose plutôt de **désactiver** le compte (`is_active = False`).

8. **Pourquoi les pages des collègues n'ont pas de modèles ?**
   Le travail est réparti : chacun développe ses entités. Ces pages sont des maquettes statiques (`demo.py`) clairement marquées « Démonstration », faciles à remplacer.

9. **Comment un collègue intègre-t-il son CRUD ?**
   Il écrit ses modèles dans son app, fait `makemigrations` + `migrate`, remplace la vue statique par des vues CRUD (même modèle que pedagogie), réutilise `base.html` et les composants. Voir `GUIDE_INTEGRATION.md`.

10. **Qu'est-ce qu'une migration ?**
    Un fichier Python qui décrit un changement de structure de la base (créer une table, ajouter une colonne). `makemigrations` le génère, `migrate` l'applique.

11. **Comment fonctionne la recherche et le filtre ?**
    Le formulaire envoie les critères dans l'URL (GET). La vue lit `request.GET` et enrichit le `QuerySet` avec `filter()` et `Q()`. Le `QueryStringMixin` garde ces critères dans la pagination.

12. **Pourquoi une modale pour supprimer ?**
    Pour confirmer avant une action irréversible. La suppression se fait en POST (jamais en GET) avec jeton CSRF.

13. **Le site est-il responsive ?**
    Oui : Bootstrap (grille) + règles CSS : sidebar en tiroir sous 1200 px, tableaux défilants, colonnes qui s'empilent.